

width=!,height=!,

Déclaration sur l'utilisation de l'Intelligence Artificielle Générative

Nous déclarons par la présente avoir utilisé des outils d'intelligence artificielle générative (IAG) comme suit :

1. **Reformulation** : Nous avons utilisé **ChatGPT** pour reformuler et affiner notre texte, en garantissant sa clarté et son originalité. Nous avons systématiquement vérifié et parfois modifié les résultats de l'IAG.
2. **Débogage de code** : Nous avons utilisé **Claude** comme assistant de développement pour identifier les erreurs potentielles, optimiser la structure du code et suggérer des améliorations algorithmiques. Chaque suggestion a été soigneusement évaluée et adaptée à nos besoins spécifiques.

Ces outils d'IAG ont été utilisés comme technologies de support pour améliorer notre rapport, les processus fondamentaux de recherche, de rédaction et de résolution de problèmes demeurant les nôtres.

Dédicace

*À mes parents, mes frères, ma sœur, ma famille et mes
amis, ...*

À tous ceux qui me sont chers

Remerciements

Avant tout, nous remercions Dieu de nous avoir accordé la force et la volonté d'accomplir ce travail. Nous tenons également à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à sa réalisation.

Nous souhaitons adresser nos vifs remerciements à notre encadrante académique, Mme Hamida Amdouni, pour le temps qu'elle nous a consacré, sa patience, ses orientations précieuses et son soutien indéfectible tout au long de cette expérience.

Nous exprimons également notre reconnaissance à l'ensemble de l'équipe de **Yes Internet** pour leur accueil chaleureux et pour nous avoir offert un cadre professionnel enrichissant, propice au développement de nos compétences. Nous remercions par ailleurs les membres du jury d'avoir accepté d'évaluer ce travail et de lui accorder de leur précieux temps.

Enfin, nos sincères remerciements vont à tous les enseignants et au personnel de l'École Supérieure d'Économie Numérique pour la qualité de la formation dispensée et leur engagement constant envers la réussite de leurs étudiants.

Résumé

Ce projet de fin d'études a été réalisé dans le cadre d'un stage au sein de la société **Yes Internet**. Il consiste à concevoir et développer une application web permettant la gestion centralisée de plusieurs agences de location de véhicules.

L'objectif principal de cette plateforme est de digitaliser et de faciliter la gestion des activités liées à la location de voitures, notamment la gestion des véhicules, des clients et des contrats de location. Le système intègre également un mécanisme de scoring client permettant d'évaluer la fiabilité des utilisateurs à partir de leur historique de location.

La solution propose aussi un module de gestion des paiements permettant d'enregistrer et de suivre les transactions effectuées par les clients. Des tableaux de bord interactifs sont mis à la disposition des administrateurs et des agences afin de visualiser les informations importantes et d'améliorer le suivi de l'activité.

Sur le plan technique, l'application repose sur une architecture web moderne basée sur le framework Laravel pour le développement du backend, React pour l'interface utilisateur et MySQL pour la gestion de la base de données. Cette architecture garantit une solution performante, évolutive et adaptée aux besoins des agences de location de véhicules.

Table des matières

Liste des tableaux

Table des figures

Introduction

Au cours des dernières années, le développement rapide des technologies numériques a profondément transformé de nombreux secteurs, favorisant la digitalisation des processus et l'amélioration de l'expérience utilisateur. Le domaine de la location de véhicules s'inscrit pleinement dans cette dynamique, où l'intégration de solutions numériques innovantes constitue un levier essentiel pour optimiser la gestion des agences, améliorer la satisfaction des clients et assurer un meilleur suivi des activités.

Cependant, malgré ces avancées, la gestion des opérations de location au sein de plusieurs agences demeure souvent complexe lorsqu'elle repose sur des méthodes manuelles ou des systèmes non centralisés. Cette situation peut engendrer des erreurs, des retards ainsi que des pertes d'informations, notamment dans le suivi des clients, des réservations et des transactions financières. Par conséquent, les agences de location éprouvent un besoin croissant de disposer d'une solution centralisée, fiable et performante permettant de gérer efficacement l'ensemble du processus de location.

Dans ce contexte, et dans le cadre de l'obtention de notre diplôme de licence en Business Computing, spécialité E-Business, à l'**École Supérieure d'Économie Numérique**, nous avons été amenés, lors de notre stage au sein de la société **Yes Internet**, à concevoir et développer une **plateforme web centralisée de gestion de location de véhicules**. Cette solution vise à digitaliser les processus métier, faciliter la gestion des véhicules, des clients et des réservations, assurer un suivi rigoureux des paiements, ainsi qu'à offrir des outils décisionnels à travers des tableaux de bord interactifs.

Ce rapport est structuré en plusieurs chapitres :

- **Chapitre 1 – Présentation du projet** : expose le contexte général, analyse les limites des méthodes actuelles et justifie la nécessité d'une solution numérique adaptée aux besoins des agences.
- **Chapitre 2 – Planification et architecture** : décrit l'identification des besoins fonctionnels, la modélisation des cas d'utilisation et l'organisation du projet en

sprints de développement.

- **Chapitre 3 – Sprint 1 : Gestion des utilisateurs** : présente le déroulement du premier sprint, incluant la planification et le développement des fonctionnalités de gestion des clients et des administrateurs.
- **Chapitre 4 – Sprint 2 : Gestion des locations et paiements** : détaille la mise en œuvre des fonctionnalités liées aux véhicules, aux réservations et au suivi des paiements.
- **Chapitre 5 – Phase de clôture et déploiement** : met en lumière les outils et technologies utilisés pour le développement de la plateforme, ainsi que les résultats obtenus.

En conclusion, ce projet vise à moderniser la gestion des locations de véhicules en offrant une solution web centralisée, performante et intuitive. Sa réalisation nous permet d'acquérir, en tant que binôme, des compétences techniques avancées tout en répondant aux besoins concrets du secteur de la location automobile.

Étude de Projet

1.1 Introduction

Dans le cadre de la réussite d'un projet, la réalisation d'une analyse préliminaire approfondie s'avère indispensable. Cette étape permet d'étudier les différentes dimensions du projet et d'en apprécier les impacts potentiels. Elle vise également à établir un cadre de référence en mettant en évidence les atouts et les limites des solutions existantes, tout en identifiant des axes d'amélioration pertinents. À cet effet, nous entamerons cette étude par une présentation générale du projet, suivie d'une analyse critique de l'existant et de la solution proposée. Enfin, nous exposerons les méthodologies adoptées ainsi que les technologies et outils mis en œuvre.

1.2 Présentation de l'Organisme d'Accueil

"Yes Internet est une société tunisienne spécialisée dans le développement de solutions numériques et de services informatiques. Elle aide ses clients à transformer numériquement leurs activités en leur proposant des applications web sur mesure, des solutions de gestion ainsi que des services d'hébergement."[yesinternet]



FIGURE 1.1 – Logo de Yes Internet

Yes Internet propose une gamme de services adaptés aux besoins des entreprises tunisiennes, notamment :

- Création d'applications web et mobiles sur mesure
- Conseil et accompagnement en matière digitale
- Services d'hébergement et de maintenance informatique
- Intégration de solutions de gestion métier

1.2.1 Structure organisationnelle

La Figure ?? présente l'organigramme de la société Yes Internet.

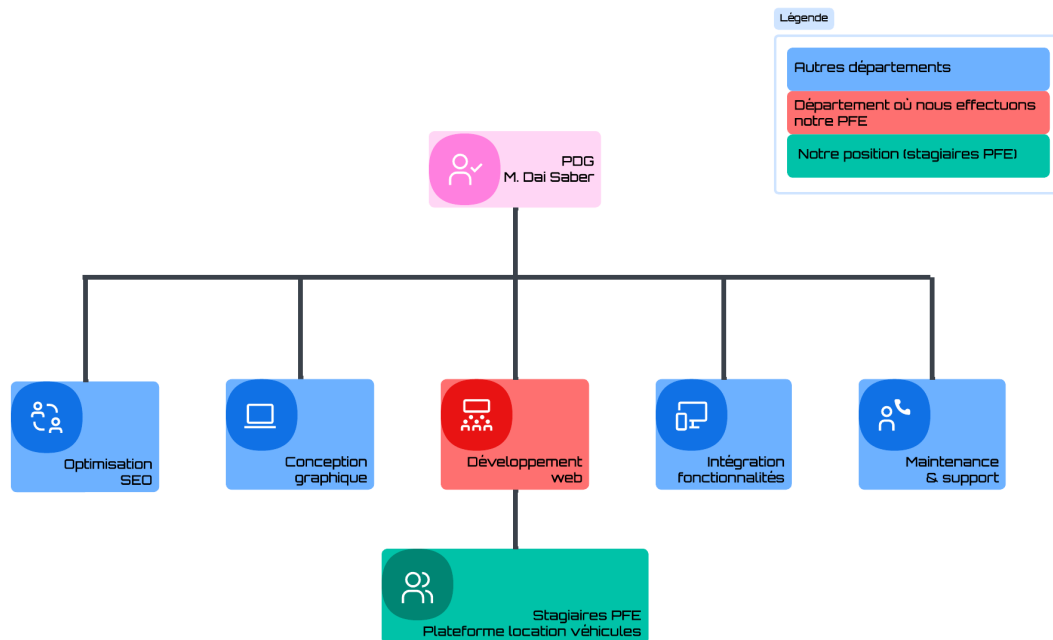


FIGURE 1.2 – Organigramme de Yes Internet

Dans le cadre de notre projet de fin d'études, nous sommes intégrés au sein du **département Développement Web**, sous la supervision directe du responsable technique. Notre mission consiste à concevoir et développer la plateforme intelligente de gestion de location de véhicules multi-agences.

1.3 Cadre et Présentation du Projet

Dans le cadre de notre projet de fin d'études, l'entreprise **Yes Internet** nous a confié la réalisation d'un projet intitulé *Plateforme Intelligente de Gestion de Location de Véhicules Multi-Agences*. Ce projet constitue une opportunité de mobiliser et de mettre en application l'ensemble des connaissances théoriques et des compétences pratiques acquises tout au long de notre formation à l'École Supérieure d'Économie Numérique. Il s'inscrit également dans la perspective de l'obtention du diplôme de licence en informatique de gestion.

1.4 Analyse de l'Existant

L'analyse de l'existant vise à enrichir l'étude des pistes innovantes d'un projet en cours de conception, avant sa mise en œuvre. Cette étude préalable permet de se faire une idée de la pertinence du projet, de sa faisabilité et de sa continuité. [etude_faisabilite_cairn]

D'après notre analyse, le secteur de la gestion et suivie de location de véhicules dispose de plusieurs solutions à l'échelle internationale, mais les agences locales tunisiennes restent largement sous-équipées sur le plan numérique. Il devient donc crucial de réaliser une évaluation approfondie des systèmes actuels tant au niveau national qu'international.

1.4.1 Étude des solutions existantes au niveau national

Après avoir effectué des recherches liées aux applications numériques utilisées pour gérer le processus de location de véhicules en Tunisie, nous avons constaté que les pratiques adoptées par les agences locales varient en fonction des moyens et des outils disponibles.

Selon certaines études, le secteur de la location de voitures en Tunisie compte plusieurs centaines d'agences, ce qui témoigne de l'importance de cette activité et de la diversité des modes de gestion adoptés [letemps_news].

- **Processus de location et de réservation en Tunisie :**

Le client se rend directement à l'agence pour louer un véhicule. L'agent collecte les informations nécessaires, telles que les coordonnées du client, le type de véhicule souhaité, la durée de location et les documents requis (permis de conduire, carte d'identité, etc.). Le contrat de location est ensuite établi à partir de modèles standards utilisés dans le domaine [scribd_contrat_standard]. Les informations relatives à la réservation, au contrat et à la disponibilité des véhicules sont consignées dans des supports internes tels que des registres ou des fichiers de suivi.

- **Suivi opérationnel et gestion de flotte :**

Les agences utilisent des fiches de suivi ou des supports de pointage pour enregistrer les mouvements des véhicules (mise à disposition, retour, état du véhicule).

- **Contrats et gestion financière :**

Les contrats de location, les paiements, les factures et les cautions sont enregistrés à l'aide des supports manuelles, fin de constituer un historique des opérations effectuées.

Exemple ci-dessous :

Contrats de location papier : Les agences utilisent fréquemment des formulaires papier pré-imprimés pour établir les contrats de location, nécessitant une saisie manuelle des informations client et véhicule, puis ces contrats sont archivés sous forme papier, servant à établir un historique des opérations de l'agence.

(Voir Figure [modele-contrat-location-vehicule]).

— **Utilisation d'outils informatiques standards :**

Certaines agences s'appuient sur des outils tels que des tableurs pour organiser les réservations et suivre les informations liées aux clients et aux véhicules [youtube_excel_gestion]. Exemple ci-dessous :

Gestion de flotte via Excel : Certaines agences utilisent des feuilles de calcul Excel pour gérer leur flotte, les réservations et la facturation, ce qui est sujet aux erreurs et manque d'automatisation. (Voir Figure [gestion-de-location-par_excel]).

Insérer votre logo
d'entreprise ici

CONTRAT DE LOCATION, N°[Le numéro de votre location], DU [Date de début] [heure] AU [Date de fin] [heure]

LOCATAIRE	VEHICULE	LOCATION
Locataire :	Marque/Modèle :	Début : [Date de début] [heure] à [Lieu]
Date de naissance :	Immatriculation :	Fin : [Date de fin] [heure] à [Lieu]
Tel :	Montant de la franchise :	Prix total :
	Montant de la caution :	Options :
		Durée :

CONDUCTEURS				
Prénom	Nom	Date de naissance	N° de permis	Date d'obtention

DEPART	
Kms compteur	
Carburant	
Commentaire :	
Le Client	Le loueur

RETOUR	
Kms compteur	
Carburant	
Commentaire :	
Le Client	Le loueur





En signant le contrat de location, le client accepte les conditions générales de location fournies par le loueur professionnel.

[Nom de l'entreprise, Adresse, code postale, ville, téléphone - Mail - Siren]
Fiéetee Ce contrat vous a été offert par fiéetee.io

FIGURE 1.3 – Modèle de contrat de location de véhicule.
[modele-contrat-location-vehicule]

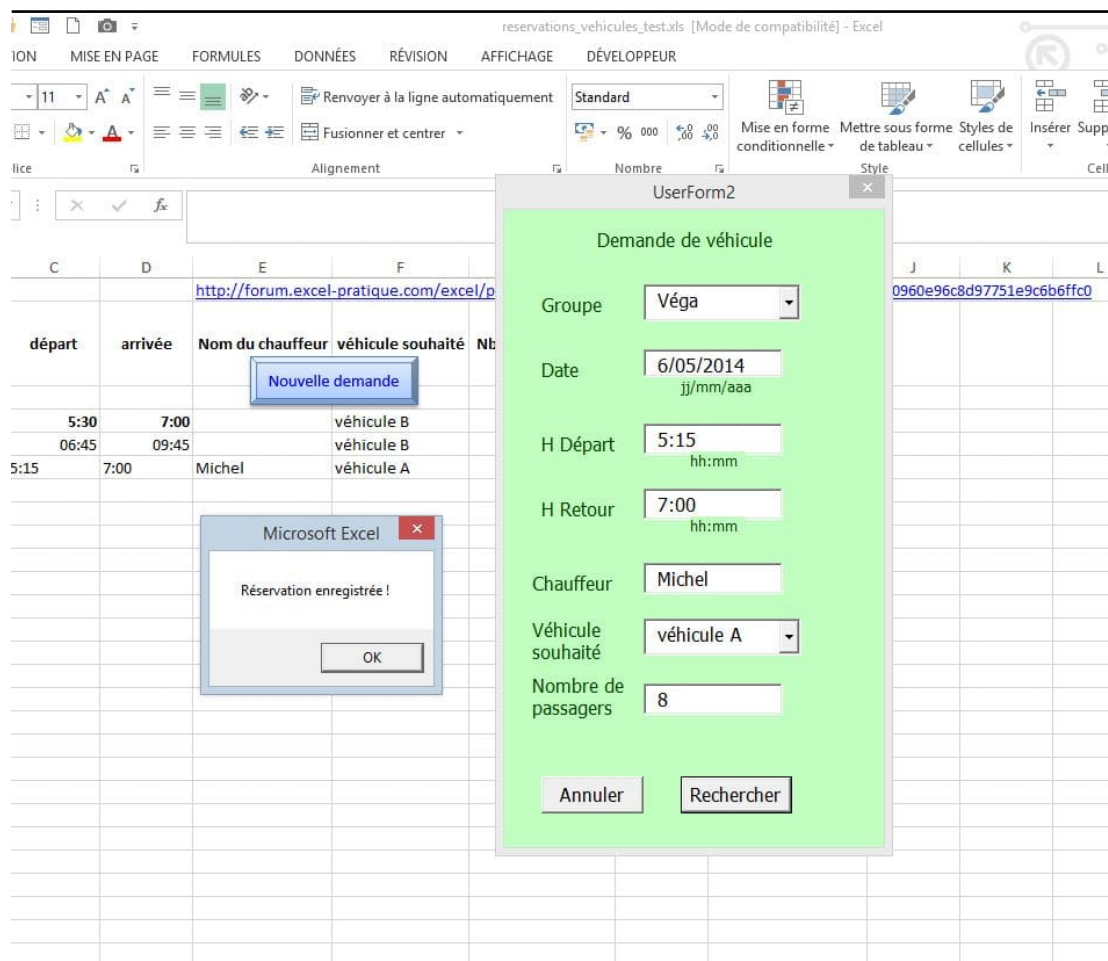


FIGURE 1.4 – Gestion de locations des voitures par Excel.[gestion-de-location-par_excel]

— **Solutions numériques spécialisées pour la location :**

En parallèle, il existe des solutions numériques dédiées à la gestion de la location de véhicules. **Des logiciels spécialisés** proposent des fonctionnalités telles que la gestion des réservations, le suivi de la flotte automobile et la facturation [renthub_tarifs][Logiciel_de_gestion_des_locations]. De même, certaines **plateformes en ligne** permettent de consulter les offres de location et d'effectuer des réservations à distance [tunisiadrive_benefits].

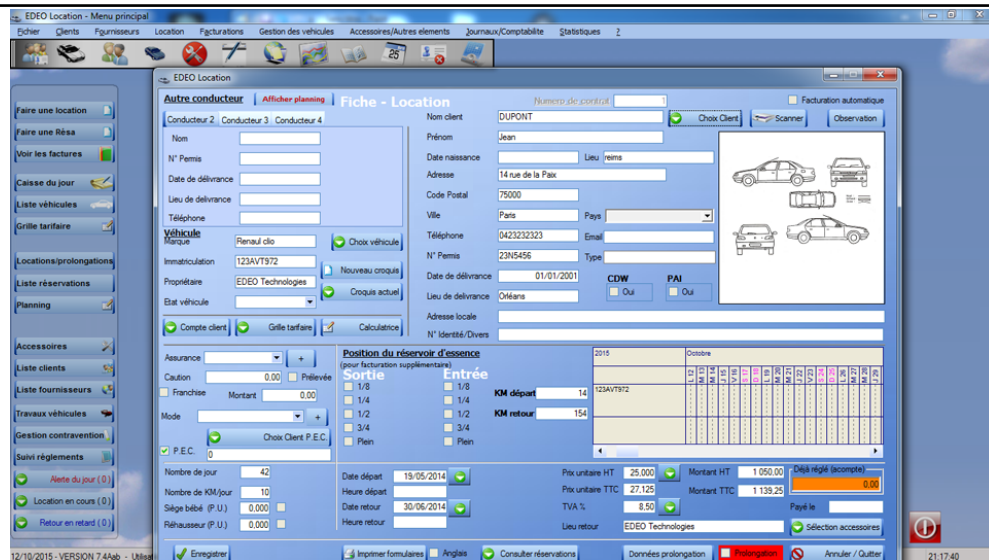


FIGURE 1.5 – EDEO : Logiciel interne de gestion des locations.[Logiciel_de_gestion_des_locations]

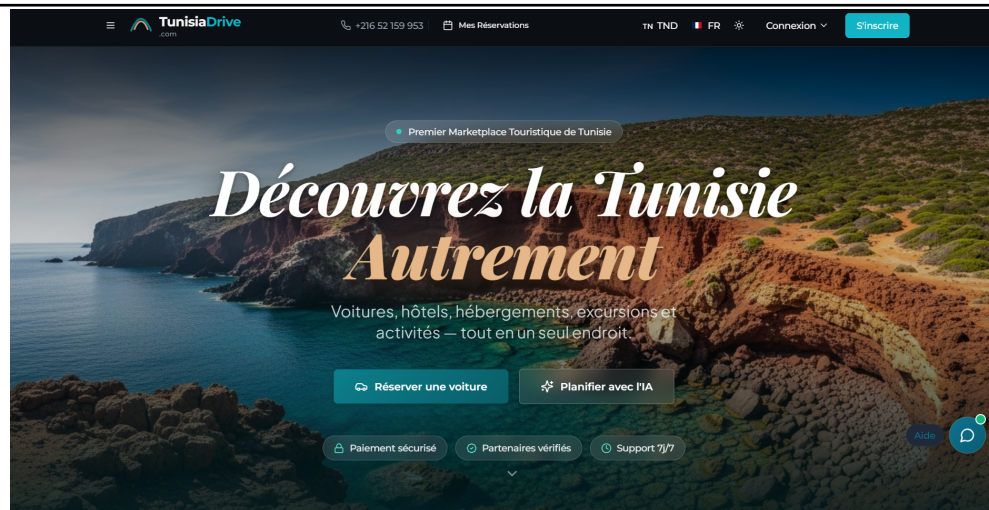


FIGURE 1.6 – Plateforme de locations des voitures en Tunisie.[tunisiadrive_benefits]

Ainsi, ces observations montrent que les pratiques de gestion dans le domaine de la location de véhicules en Tunisie reposent sur une diversité de solutions, allant de l'utilisation de supports internes et d'outils génériques à l'intégration de solutions numériques dédiées au secteur.

1.4.2 Étude des solutions existantes au niveau international

Nous procédons ici à une analyse approfondie d'une solution de gestion de location des véhicules disponible à l'international, retenue pour sa représentativité, sa maturité et sa large adoption sur le marché mondial.

1.4.2.1 Analyse de la plateforme Rent Centric

Rent Centric est une plateforme cloud de référence dans le domaine de la gestion de location de véhicules, fondée en 2002 et utilisée par des centaines d'agences à travers le monde, notamment en Amérique du Nord, en Europe et au Moyen-Orient. Elle s'adresse aussi bien aux petites agences indépendantes qu'aux grandes entreprises multi-sites [rc]. La plateforme propose un ensemble de fonctionnalités permettant de couvrir l'intégralité du processus de location :

- **Réservation en ligne** : réservations en temps réel avec confirmation automatique et gestion des disponibilités.
- **Gestion de flotte** : suivi complet des véhicules, historique d'utilisation et planification de maintenance.
- **Gestion des contrats** : génération automatique des contrats, prolongations et gestion des retours.
- **Tarification dynamique** : grilles tarifaires configurables selon la saison, la durée et le profil client.
- **Gestion multi-agences** : administration centralisée avec tableaux de bord personnalisés par site.
- **Rapports et analytique** : rapports détaillés sur les performances et le taux d'occupation de la flotte.

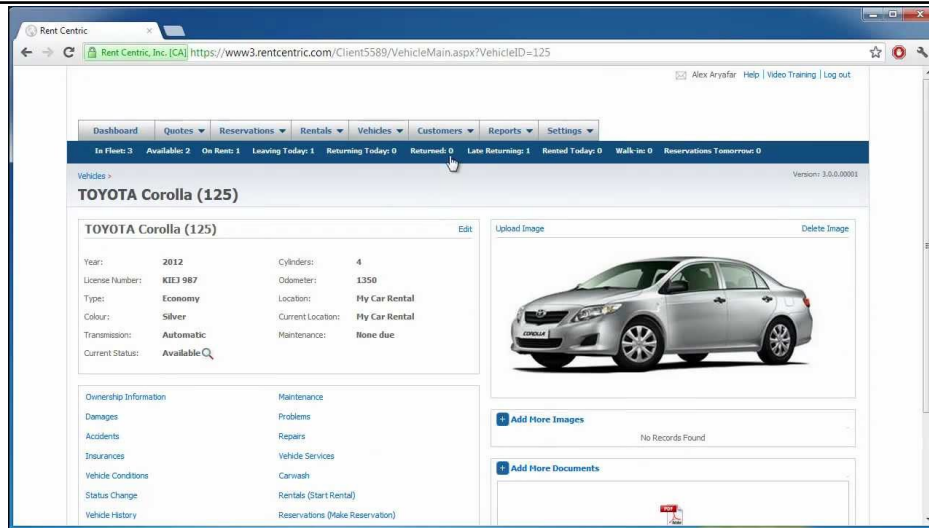


FIGURE 1.7 – Aperçu des fonctionnalités de Rent Centric [rc]

1.4.2.2 Analyse de HQ Rental Software

HQ Rental Software est une solution complète de gestion de flotte de véhicules, conçue pour optimiser l'efficacité opérationnelle et améliorer l'expérience client. Elle offre une gamme étendue de fonctionnalités pour les entreprises de location de voitures de toutes tailles.[hq_features]

- **Réservations et paiements en ligne** : Permet aux clients de réserver et de payer directement via un plugin de site web convivial, augmentant ainsi la portée du marché et améliorant l'expérience client.
- **Gestion de flotte** : Offre des outils pour gérer la disponibilité des véhicules, les réparations et les finances, réduisant la dépendance aux feuilles de calcul manuelles.
- **Dématérialisation** : Génère des contrats de location personnalisables qui peuvent être signés numériquement, avec la possibilité de télécharger des photos du véhicule via l'application mobile HQ.
- **Solutions pour divers marchés de location** : S'adapte à la location de motos, de bateaux, d'équipements et propose des solutions pour la location à long terme et le leasing.
- **Rapports avancés** : Fournit des tableaux de bord personnalisés pour suivre les indicateurs de performance clés et exporter les données nécessaires.
- **Paiements intégrés et comptabilité** : Automatise le processus de vente et la gestion des transactions financières.
- **Solution basée sur le cloud et accès mobile** : Accessible depuis n'importe

quel appareil et n'importe où dans le monde, avec des applications Android et iOS dédiées.

Vehicle Key	Vin	License Plate	Status	Vehicle Type	Current Renter	Available Date	Make	Vehicle Model	Vehicle Class	Current Location	Odometer	Fuel Level	Available?	Last Location Update	Telematics Connected
0001	1L8UZZ3C9Q8UTEN	1	Available	Car	Not Assigned		Chevrolet	Lumina 3.6 S	Economic Manual	Unknown	166,632	6/8	Yes		Yes
0002	ZZ34D0VZ8MCCYZLT	11	Available	Car	Not Assigned		SAFETY CHECK	CLEVIS PIN RIN 80321	Economic Manual	Unknown	49,025	6/8	Yes		Yes
0003	VNNEEQ4SACUJAZ5A0	21	Available	Car	Not Assigned		THOMAS BUILT	F565	Economic Manual	Unknown	158,093	6/8	Yes		Yes
0004	7F3D92TH5YNTLULO	31	Available	Car	Not Assigned		Toyota	Sprinter KE 40	Economic Manual	Unknown	94,731	6/8	Yes		Yes
0005	LV7THR6R8BAROWPCG	41	Available	Car	Not Assigned		Simca	1308	Economic Manual	Unknown	177,251	6/8	Yes		No
0006	3K1MHLKTHZPHVZ	51	Available	Car	Not Assigned		Nissan	Sports DC-3	Economic Manual	Unknown	159,806	6/8	Yes		No
0007	8HTD35FK5VZ3HMYC	61	Available	Car	Not Assigned		MALONE AUTO RACKS	MICROSPORT MPC460	Economic Automatic	Unknown	169,874	6/8	Yes		No
0008	LTD0K14QBIEKRCN	71	Available	Car	Not Assigned		DUNLOP	L6014	Economic Automatic	Unknown	88,843	6/8	Yes		No
0009	2WEKETW55YWBATV3	81	Available	Car	Not Assigned		Mazda	3 Grand	Economic	Unknown	155,339	6/8	Yes		No

FIGURE 1.8 – Tableau de bord de HQ Rental Software. [hq_features]

1.4.2.3 Analyse de VEVs Car Rental platform

VEVS Car Rental Software est une solution logicielle et de site web intégrée pour les entreprises de location de voitures, visant à automatiser les processus, à rationaliser la gestion et à augmenter l'efficacité opérationnelle.[vevs_features]

- **Réservations en ligne :** Permet aux clients de réserver des véhicules en ligne, avec des confirmations par e-mail automatiques et une page « My Booking » pour le suivi des réservations.
- **Gestion de flotte :** Outils pour gérer la disponibilité des véhicules, les réparations et les finances.
- **Tarification flexible et taxes configurables :** Permet de définir des options de tarification flexibles basées sur la durée de location et la demande, et de configurer les taxes (TVA, taxes touristiques, etc.).
- **Vente additionnelle d'extras et codes promotionnels :** Génère des revenus supplémentaires en proposant des extras (GPS, sièges enfant) et en créant des codes promotionnels pour les campagnes marketing.
- **Tarifs saisonniers et autres frais :** Permet de générer des structures de prix saisonnières distinctives et d'ajouter des frais supplémentaires (relais, jeunes conducteurs, kilométrage supplémentaire).
- **Solution basée sur le cloud et accès mobile :** Permet de gérer l'entreprise depuis n'importe quel appareil et n'importe où dans le monde.

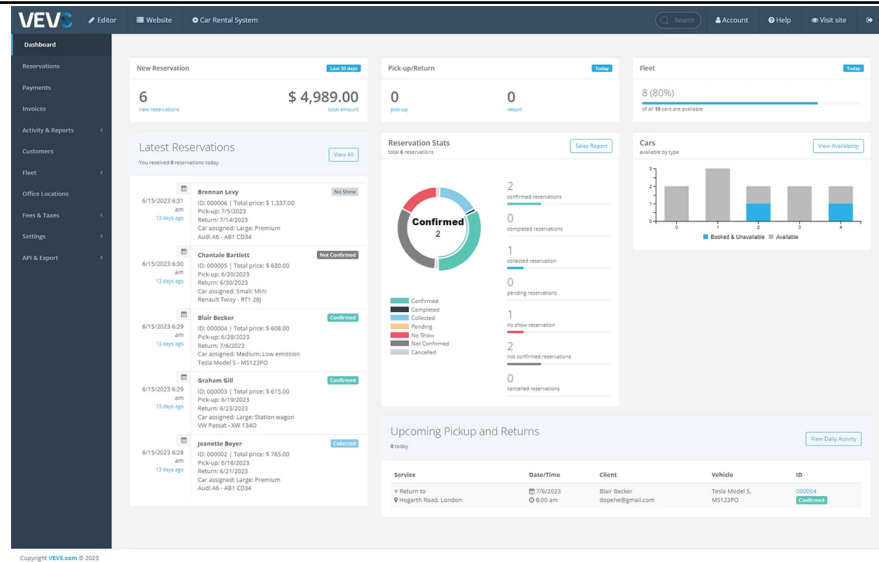


FIGURE 1.9 – Tableau de bord de VEVs Car Rental platform. [vevs_features]

Cette analyse nous a permis d'identifier les fonctionnalités clés à intégrer dans notre solution, tout en ciblant les lacunes auxquelles notre projet se propose de répondre.

1.5 Critique de l'Existant

L'analyse du marché actuel révèle plusieurs insuffisances notables, résumées ci-dessous :

1.5.1 Critique des solutions existantes au niveau national

- **Fragmentation du secteur** : Selon une étude récente, la Tunisie compte plus de 700 sociétés de location de voitures, ce qui traduit un marché fortement fragmenté et peu structuré. Cette dispersion complique la mise en place de solutions centralisées et favorise des pratiques hétérogènes entre les agences [letemps_news].
- **Faible digitalisation des agences** : Malgré l'existence de plateformes d'information et de réservation en ligne, plusieurs agences continuent d'utiliser des outils traditionnels (Excel, registres papier). Cela limite l'automatisation des processus et engendre une perte d'efficacité opérationnelle [drivotunisie].
- **Absence de plateformes unifiées** : Contrairement à d'autres marchés, il existe peu de plateformes nationales intégrées permettant de centraliser les offres de

plusieurs agences. Les solutions existantes sont souvent limitées à des vitrines ou à des sites informatifs sans réelle interconnexion des données.

- **Manque de visibilité globale** : Les plateformes locales disponibles ne proposent généralement pas de tableaux de bord analytiques avancés. Cela empêche les agences de suivre en temps réel leurs performances ou d'exploiter les données pour optimiser leur activité.
- **Expérience utilisateur limitée** : Les systèmes actuels offrent une expérience client fragmentée (réservation, paiement, suivi). L'absence d'intégration complète (réservation en ligne, paiement sécurisé, suivi du véhicule) réduit l'attractivité des services proposés.
- **Difficulté d'adaptation technologique** : Les agences rencontrent des difficultés à intégrer des technologies modernes (paiement en ligne, géolocalisation, gestion automatisée). Cela est dû au manque de solutions locales adaptées et au coût des alternatives internationales.

1.5.2 Critique des solutions existantes au niveau international

- **Coût élevé et accessibilité limitée** : Les plateformes et logiciels internationaux de gestion de location (SaaS : Software as a Service) impliquent des coûts d'abonnement élevés, ce qui constitue une barrière importante pour les petites et moyennes agences tunisiennes.
- **Complexité d'intégration** : Ces solutions nécessitent souvent une intégration technique complexe (API, configuration avancée), difficile à mettre en œuvre dans des environnements locaux disposant de ressources techniques limitées.
- **Orientation SaaS plutôt que plateforme** : La majorité des solutions internationales sont conçues comme des outils de gestion interne (SaaS) et non comme des plateformes collaboratives multi-agences. Elles ne favorisent donc pas la mutualisation des offres ni la visibilité globale du marché.
- **Manque d'adaptation au contexte local** : Ces solutions ne prennent pas en compte les spécificités tunisiennes (réglementation, fiscalité, modes de paiement locaux). Leur utilisation nécessite souvent des adaptations coûteuses et complexes.
- **Fonctionnalités surdimensionnées** : Certaines plateformes proposent des fonctionnalités avancées (ERP complets) qui dépassent les besoins réels des agences locales, entraînant une complexité inutile et une faible adoption.
- **Dépendance au fournisseur** : L'utilisation de solutions SaaS implique une dépendance forte vis-à-vis du fournisseur (hébergement, sécurité, mises à jour), ce qui peut poser des risques en cas d'interruption du service.
- **Expérience utilisateur non localisée** : Même si certaines plateformes sont multilingues, elles ne sont pas toujours adaptées au contexte culturel et linguistique tunisien, ce qui peut nuire à leur adoption par les utilisateurs locaux.
- **Absence de logique marketplace locale** : Contrairement à des plateformes globales comme les marketplaces internationales, les solutions existantes ne permettent pas de fédérer plusieurs agences tunisiennes au sein d'un écosystème numérique commun.

1.6 Solution Proposée

Pour remédier aux limites identifiées dans l'analyse du marché actuel, nous proposons le développement d'une plateforme web intelligente de gestion de location de véhicules multi-agences, entièrement adaptée au contexte tunisien. Notre solution présentera plusieurs avantages distinctifs :

- **Une architecture multi-agences centralisée** permettant de gérer l'ensemble du système depuis une interface unique
- **Une gestion complète du processus de location** (véhicule disponible, réservé, en cours d'utilisation, retourné ou en maintenance)
- **Une expérience utilisateur intégrée** couvrant l'ensemble du processus de location (recherche, réservation, paiement et suivi) via une interface intuitive et ergonomique
- **Un système de sécurité renforcé** assurant l'authentification des utilisateurs, la gestion des accès et la protection des données sensibles
- **Un système de paiement en ligne sécurisé** permettant aux clients d'effectuer leurs transactions directement via la plateforme
- **Une gestion automatisée des contrats de location** incluant leur génération, leur archivage et leur consultation
- **Un suivi du statut des réservations** avec un historique complet par agence et par client
- **Un système de score de fiabilité client** calculé automatiquement selon le comportement historique de chaque client (retards, annulations, incidents)
- **Des tableaux de bord analytiques** offrant une visibilité complète sur les indicateurs clés de performance
- **Une architecture ouverte** basée sur des API facilitant l'intégration avec des systèmes externes
- **Une adaptation aux spécificités du contexte tunisien**, notamment en termes de réglementation et modes de paiement locaux
- **Une solution accessible et adaptée aux petites et moyennes agences** grâce à une conception optimisée et des coûts réduits
- **Une interface simple et facile à prendre en main** favorisant une adoption rapide par les agences
- **Une architecture modulaire et évolutive** basée sur des technologies modernes garantissant performance et maintenabilité
- **Une architecture scalable** capable de supporter un grand nombre d'agences et d'utilisateurs simultanément

Enfin, notre plateforme sera conçue en s'inspirant des meilleures pratiques des solutions

analysées, tout en restant adaptée aux besoins et contraintes du marché tunisien.

1.7 Méthodologie et Modélisations

L'implémentation de stratégies de gestion performantes est l'une des clés de la réussite d'un projet et d'une bonne coordination entre les différentes phases. Cela implique le déploiement d'une bonne méthodologie de gestion de projet et de langages de modélisation adaptés.

1.7.1 Les méthodes agiles

La méthode Agile se base sur un cycle de développement itératif qui place le client au centre du processus [atlassian]. Contrairement aux méthodes classiques qui suivent une approche linéaire et séquentielle avec des spécifications figées dès le départ, les méthodes agiles privilégient la flexibilité, la collaboration et la livraison fréquente de versions fonctionnelles. Grâce à cette approche, le demandeur obtient une meilleure visibilité sur la gestion des travaux, et l'équipe bénéficie d'un feedback régulier pour appliquer directement les changements nécessaires. Les données présentées dans le Tableau ?? illustrent les principales différences entre les méthodes classiques et les méthodes agiles.

TABLE 1.1 – Tableau comparatif entre méthodes classiques et méthodes agiles

Méthode classique	Méthode agile
Approche linéaire et séquentielle	Approche itérative et incrémentale
Moins de flexibilité	Très flexible et adaptable
Une seule livraison à la fin du projet	Des livraisons fréquentes et régulières

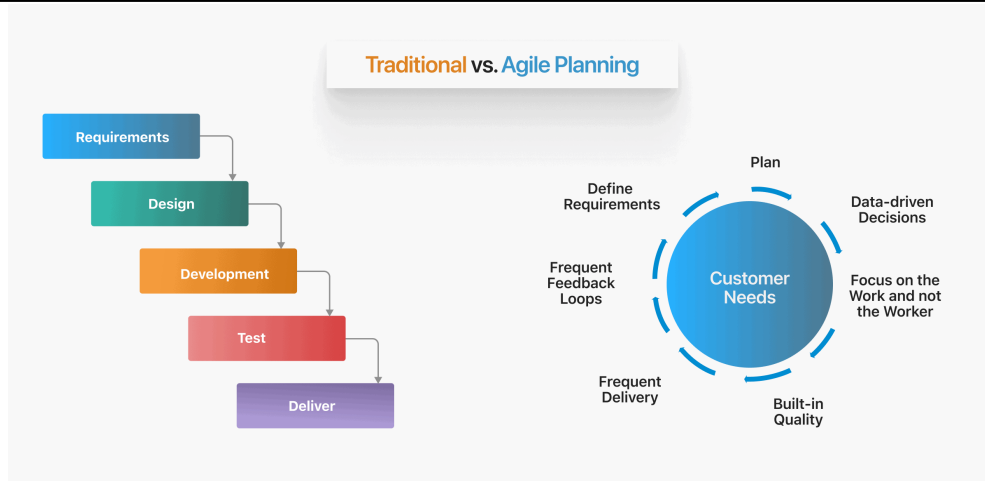


FIGURE 1.10 – Agile vs approches classiques

Parmi les méthodes agiles les plus reconnues dans le domaine du génie logiciel, on trouve Scrum, Kanban, Extreme Programming (XP), SAFe et Crystal. En vue de mener notre projet de manière efficace et de fournir un produit de qualité dans les délais impartis, nous avons opté pour la méthodologie **Scrum**.

1.7.2 La méthodologie Scrum

"SCRUM est un cadre de travail dans lequel les personnes peuvent aborder des problèmes complexes et adaptatifs tout en livrant de manière efficace et créative des produits de la plus grande valeur possible." [scrum]

Il s'agit d'une approche itérative et progressive qui évite l'effet « tunnel », où les résultats ne sont visibles qu'à la clôture du projet. C'est également une approche collaborative dans laquelle tous les membres de l'équipe sont actifs et contribuent aux décisions. Scrum s'articule autour de trois acteurs principaux :

- **Product Owner** : Il détient la responsabilité du produit ; c'est le client qui communique ses attentes et approuve le travail lors des revues de sprint.
- **Scrum Master** : Il supervise le bon déroulement de la mission et résout les obstacles qui entravent l'avancement du projet.
- **Scrum Team** : Ensemble d'individus en charge d'exécuter les exigences, incluant développeurs, concepteurs et testeurs.

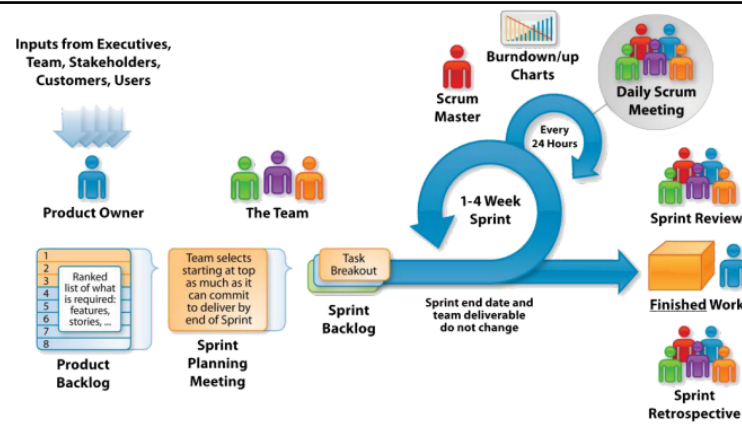


FIGURE 1.11 – Les caractéristiques du modèle Scrum

1.7.3 Product Backlog et Sprint Backlog

Dans la méthodologie Scrum, deux artefacts essentiels structurent la gestion des tâches et des priorités tout au long du projet.

Le **Product Backlog** est une liste ordonnée et exhaustive de toutes les fonctionnalités, exigences et améliorations souhaitées pour le produit final. Il est géré et priorisé par le Product Owner selon la valeur métier de chaque élément. Il évolue continuellement tout au long du projet en fonction des retours des parties prenantes et des besoins émergents. Chaque élément du Product Backlog est exprimé sous forme de **User Stories**, décrivant une fonctionnalité du point de vue de l'utilisateur.

Le **Sprint Backlog**, quant à lui, représente la sélection des User Stories issues du Product Backlog que l'équipe s'engage à réaliser durant un sprint donné. Contrairement au Product Backlog qui couvre l'ensemble du projet, le Sprint Backlog est limité à une itération précise (2 à 4 semaines) et appartient à l'équipe de développement. Il est défini lors du Sprint Planning et ne doit pas être modifié en cours de sprint.

TABLE 1.2 – Différence entre Product Backlog et Sprint Backlog

Critère	Product Backlog	Sprint Backlog
Portée	Ensemble du projet	Un seul sprint
Propriétaire	Product Owner	Équipe de développement
Contenu	Toutes les User Stories	User Stories sélectionnées
Évolution	Continu et dynamique	Figé pendant le sprint
Priorité	Définie par valeur métier	Définie par le Sprint Planning

1.8 Langage de Modélisation : UML

Pour la conception de notre système, nous avons adopté l'**UML** (Unified Modeling Language), un langage graphique composé de diagrammes offrant différentes perspectives sur le système en cours de réalisation.



FIGURE 1.12 – Logo UML

L'UML est un standard reconnu internationalement qui présente plusieurs atouts : il est sans ambiguïtés, universel, adaptable à tout langage orienté objet, et facilite la communication entre les membres de l'équipe et les parties prenantes. Dans le cadre de notre projet, nous utiliserons principalement le **diagramme de cas d'utilisation** pour modéliser les interactions entre les acteurs et le système, ainsi que le **diagramme de séquence** pour décrire le déroulement des scénarios fonctionnels.[uml]

1.9 Conclusion

Dans ce premier chapitre, nous avons présenté notre organisme d'accueil Yes Internet ainsi que le cadre général du projet. Nous avons analysé les solutions existantes au niveau national et international, résumé leurs lacunes, et proposé notre solution adaptée au contexte tunisien.

Enfin, nous avons présenté la méthodologie de gestion de projet utilisée ainsi que le langage de modélisation qui guideront la conduite du projet.

Dans le chapitre qui suit, nous aborderons la planification de notre projet.

Planification et architecture

Introduction

La planification constitue une étape essentielle dans la mise en œuvre d'un projet. Ainsi, dans ce chapitre, nous allons identifier et analyser **les différentes parties prenantes** du système.

Ensuite, nous présenterons **les besoins fonctionnels et non fonctionnels**, **les diagrammes de cas d'utilisation**, ainsi que l'organisation du projet en **sprints** Scrum.

2.1 Identification des acteurs

Notre application web implique cinq acteurs principaux :

- **Internaute** : L'internaute peut consulter les actualités, interagir avec un Chatbot et créer un compte sur la plateforme pour accéder aux système.
- **Utilisateur** : L'utilisateur, qui peut être un client, une agence ou un administrateur, a la possibilité de gérer son profil, consulter les offres et voir son score client, après authentification.
- **Client** : Le client est un utilisateur souhaitant louer un véhicule auprès d'une agence digitale .Suite à l'authentification, il peut consulter ses réservations en cours, contacter une agence et voir son score de fiabilité.
- **Agence** : L'agence est une entreprise de location de véhicules utilisant la plateforme pour gérer son activité.Dès son authentification, elle peut créer et gérer sa vitrine automobile, suivre les réservations et les réclamations, valider ou refuser des demandes et accéder aux statistiques.
- **Administrateur** :Une fois authentifié, l'administrateur supervise l'ensemble de

la plateforme pour garantir son bon fonctionnement. Il peut gérer les agences et les utilisateurs, superviser les véhicules, consulter les KPI, détecter les anomalies et configurer les paramètres système.

2.2 Diagramme de contexte statique

Le diagramme de contexte statique permet de mettre en évidence les interactions au sein des différentes entités impliquées dans le système, tout en précisant le nombre d'exemplaires existant pour chaque intervenant connecté à un instant donné.[itttrip]

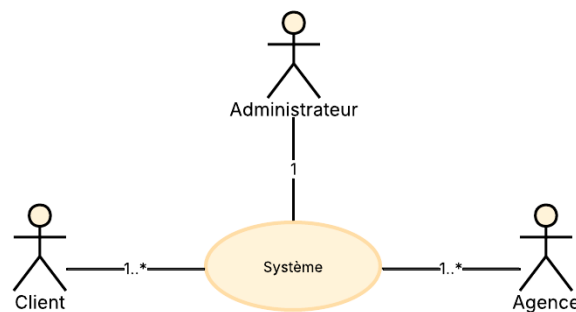


FIGURE 2.1 – Diagramme de contexte statique

2.3 Identification et analyse des besoins

Dans cette section, nous identifions les différentes parties prenantes du projet et analysons leurs besoins, qu'ils soient fonctionnels ou non fonctionnels. Cette étape permet de définir et d'organiser l'ensemble des exigences du système sous forme d'un **Product Backlog**, afin de faciliter leur gestion et leur priorisation pour les étapes de développement suivantes.

2.3.1 Identification des besoins fonctionnels

Le besoin peut être exprimé de manière fonctionnelle, mettant en évidence les fonctions de services (*À quoi ça sert ?*) et les fonctions techniques (*Comment cela peut marcher ?*). Ces fonctions doivent être ordonnées, hiérarchisées et quantifiées sous la forme de valeurs de performance attendues.[ISO]

A. Les besoins fonctionnels de l'acteur "Internaute"

-
- **Consulter actualités** : L'internaute peut accéder aux informations publiées sur la plateforme.
 - **S'inscrire** : L'application permet à l'internaute de s'inscrire en fournissant ses informations personnelles.
 - **Interagir avec chatbot** : L'internaute peut poser des questions et obtenir des réponses automatiques.
 - **Contacteur administrateur** : L'internaute peut envoyer un message à l'administrateur pour obtenir plus d'informations .

B. Les besoins fonctionnels de l'acteur "Utilisateur"

En plus des besoins de l'acteur "Utilisateur" :

- **S'authentifier** : L'utilisateur doit saisir un login et un mot de passe pour accéder aux fonctionnalités de système.
- **Gérer profil** : L'utilisateur peut modifier ses informations personnelles et ses préférences.
- **Consulter véhicules** : L'utilisateur peut consulter les véhicules disponibles ainsi que leurs détails.
- **Voir score client** : L'utilisateur peut consulter son indice de fiabilité.

C. Les besoins fonctionnels de l'acteur "Client"

En plus des besoins de l'acteur "Utilisateur" :

- **Effectuer réservation** : Le client peut réserver un véhicule auprès d'une agence.
- **Consulter réservations en cours** : Le client peut suivre l'état de ses réservations.
- **Contacteur agence** : Le client peut communiquer directement avec l'agence pour assistance ou négociation.

D. Les besoins fonctionnels de l'acteur "Agence"

En plus des besoins de l'acteur "Utilisateur" :

- **Gérer vitrine** : L'agence peut publier et mettre à jour ses véhicules disponibles.
- **Traiter réservations** : L'agence peut accepter ou refuser les demandes de réservation ainsi que traiter les réclamations des clients..
- **Consulter statistiques** : L'agence peut analyser ses performances via des indicateurs.

E. Les besoins fonctionnels de l'acteur "Administrateur"

En plus des besoins de l'acteur "Utilisateur" :

- **Consulter agences** : L'administrateur peut visualiser la liste des agences ainsi que leurs informations détaillées. À partir de cette interface de consultation, il peut également effectuer des actions de gestion telles que l'ajout d'une nouvelle agence, la modification des informations ou le blocage d'une agence en cas de non-conformité.
- **Consulter utilisateurs** : L'administrateur peut consulter les comptes la liste des utilisateurs ainsi que leurs informations.
Depuis cette interface, il peut aussi gérer les utilisateurs en ajoutant de nouveaux comptes (en plus de l'inscription autonome), en modifiant leurs informations ou en bloquant leurs comptes si nécessaire.
- **Gérer messages** : L'administrateur peut consulter les messages reçus et y répondre afin d'assurer un bon suivi de la communication.

-
- **Consulter statistiques :** L'administrateur peut suivre les indicateurs de performance globaux afin de détecter les anomalies et d'améliorer le système.

2.3.2 Identification des besoins non fonctionnels

Les exigences non fonctionnelles représentent la qualité prévue de l'application et les contraintes à satisfaire lors de la réalisation des besoins fonctionnels. Notre application doit satisfaire un ensemble d'exigences non fonctionnelles :

A. Performance

- Optimiser le temps de chargement des différentes pages de l'application.

B. Sécurité

- Protéger les données personnelles des agences et des clients.
- Sécuriser tous les accès par authentification (login et mot de passe).

C. Fiabilité

- Garantir un fonctionnement fluide et sans erreurs critiques.

D. Maintenabilité

- Organiser le code de façon propre et structurée pour faciliter les évolutions futures.

E. Ergonomie

- Offrir des interfaces intuitives, épurées et faciles à prendre en main pour tous les profils d'utilisateurs.

2.4 Diagramme des cas d'utilisation global

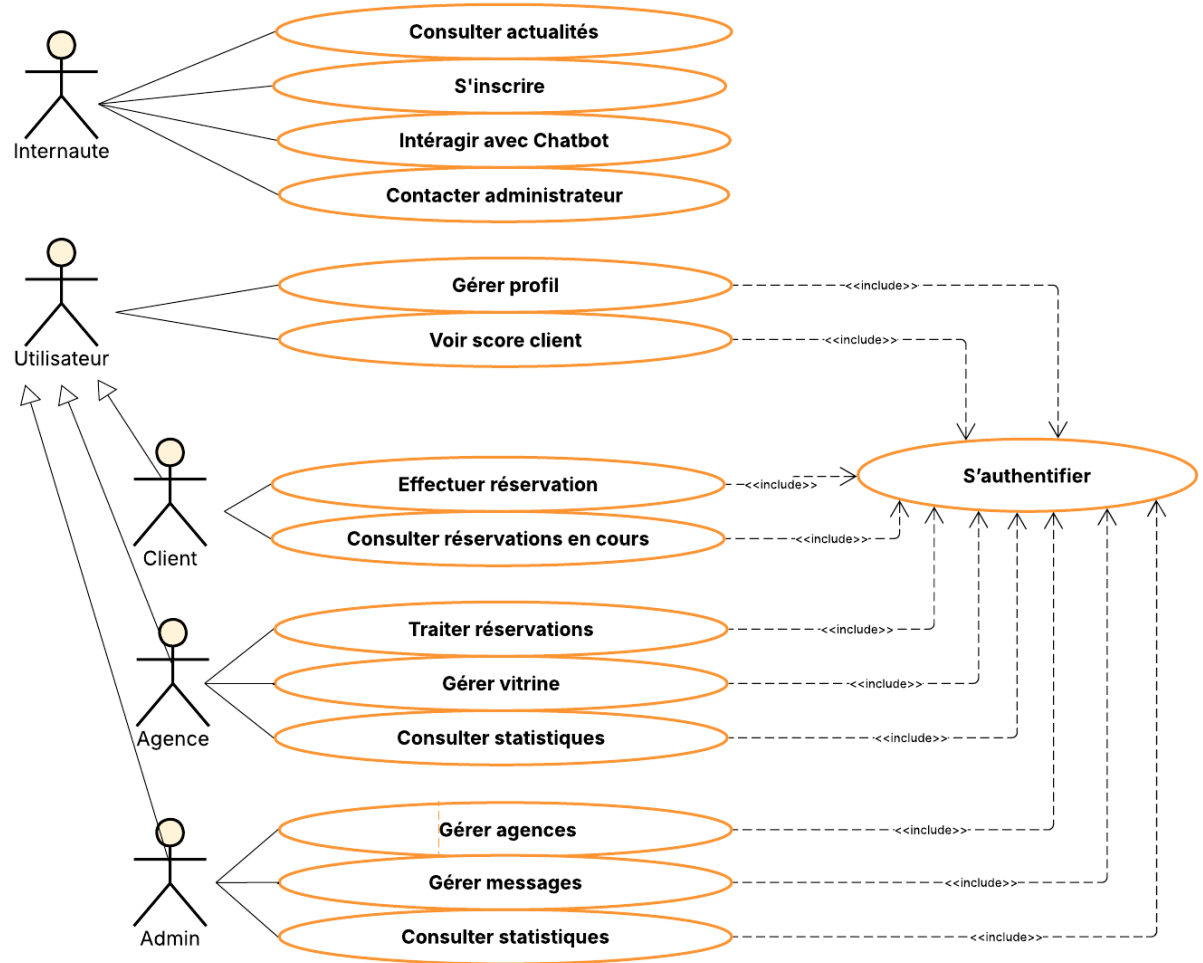


FIGURE 2.2 – Diagramme de cas d'utilisation global

Le diagramme de cas d'utilisation global présente une vue d'ensemble des interactions entre les différents acteurs (internautes, utilisateurs, clients, agences et administrateur) et notre application. Il permet de mettre en évidence les fonctionnalités principales offertes par le système ainsi que les relations hiérarchiques entre les acteurs. [Lucidchart]

2.5 Identification de l'équipe Scrum

Dans le cadre d'une approche Scrum, il est essentiel de définir clairement les responsabilités de chaque membre de l'équipe. Les rôles Scrum structurent le travail collaboratif et garantissent une organisation efficace, assurant la transparence, la communication et l'adaptation continue tout au long du projet.



FIGURE 2.3 – Les rôles Scrum

2.6 Le Backlog produit

Le *Product Backlog* constitue une liste hiérarchisée de tâches qui définissent les caractéristiques attendues d'un produit. Il s'agit d'un élément fondamental de la méthodologie Scrum [scrum]. Dans le cadre de notre projet, le backlog est structuré autour des champs suivants :

- **ID** : identifiant unique attribué à chaque *User Story*, permettant de la distinguer et de la référencer facilement dans le backlog.
- **Thème** : résumé synthétique décrivant la fonctionnalité concernée, servant de regroupement logique pour plusieurs *User Stories* liées.
- **User Story** : description du contenu fonctionnel attendu, exprimée du point de vue de l'utilisateur, sous forme de scénario simple.
- **Priorité** : niveau d'importance défini par le *Product Owner*, indiquant l'ordre de traitement et la valeur métier associée à la fonctionnalité.

TABLE 2.1 – Backlog produit

	ID	Thème	En tant que	User Story (Je veux...)	Priorité	
US01	S'inscrire			Internaute	s'inscrire afin d'accéder aux services	Haute
US02	Authentification			Utilisateur	m'authentifier afin d'accéder aux fonctionnalités du système	Haute
US03	Gérer profil			Utilisateur	gérer mon profil afin de modifier mes informations personnelles et préférences	Haute
US04	Gérer vitrine			Agence	gérer la vitrine afin de publier et mettre à jour les véhicules disponibles	Haute
US05	Gérer agences			Admin	gérer les agences afin de visualiser, ajouter, modifier ou bloquer les agences	Haute
US06	Consulter actualités			Internaute	consulter les actualités afin d'accéder aux informations publiées sur la plateforme	Moyenne
US07	Effectuer réservation			Client	effectuer une réservation afin de louer un véhicule auprès d'une agence	Moyenne
US08	Consulter réservations en cours			Client	consulter mes réservations en cours afin de suivre leur état	Moyenne
US09	Traiter réservations et réclamations			Agence	traiter les réservations et réclamations afin d'accepter/refuser les demandes et gérer les litiges	Moyenne
US10	Voir score client			Utilisateur	voir mon score client afin de connaître mon indice de fiabilité	Moyenne

	ID	Thème	En tant que	User Story (Je veux...)	Priorité
US11		Contacter administrateur		Internaute contacter l'administrateur afin d'obtenir plus d'informations	Moyenne
US12		Gérer messages		Admin gérer les messages afin de consulter et répondre aux utilisateurs	Moyenne
US13		Consulter statistiques(Agence)		Agence consulter les statistiques afin d'analyser les performances	Moyenne
US14		Consulter statistiques (Admin)		Admin consulter les statistiques globales afin d'analyser les performances du système	Moyenne
US15		Interagir avec chatbot		Internaute interagir avec le chatbot afin d'obtenir des réponses automatiques	Moyenne

2.7 La planification des sprints

La planification de sprint permet d'identifier les tâches qui seront accomplies pendant le sprint et la manière dont elles seront réalisées.

Sprint 1	Sprint 2
• S'inscrire • S'authentifier • Gérer profil • Gérer vitrine • Gérer agence • Consulter actualités • Contacter administrateur • Gérer messages	• Effectuer réservation • Consulter réservations en cours • Traiter réservations • Voir score client • Consulter statistiques(Admin) • Consulter statistiques(Agence) • Interagir avec chatbot

FIGURE 2.4 – Sprint 1

2.8 Diagramme de Gantt

Le diagramme de Gantt illustre la répartition temporelle des quatre phases du projet sur une durée totale de douze semaines.

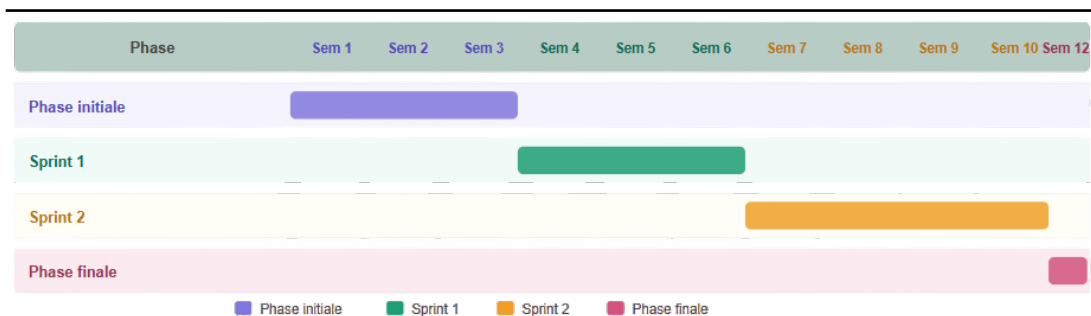


FIGURE 2.5 – Diagramme de Gantt

2.9 Conclusion

Au cours de ce chapitre, nous avons identifié les parties prenantes et défini le backlog du produit en précisant les demandes fonctionnelles et techniques de notre projet.

Ensuite, nous avons achevé la première étape de la méthodologie Scrum en constituant l'équipe, en élaborant le diagramme global des cas d'utilisation et en établissant la liste

des tâches produit ainsi que la planification des sprints.

Le chapitre suivant sera consacré à la réalisation du **Sprint 1**, couvrant la gestion des agences, des clients et de la vitrine.

Chapitre 3

Réalisation du premier sprint

3.1 Introduction

En se basant sur la planification établie dans le chapitre précédent, ce chapitre est consacré à la réalisation du Sprint 1. Nous aborderons successivement la description fonctionnelle, la conception, l'implémentation et la phase de tests.

3.2 Backlog produit du premier sprint

Le Sprint Backlog est constitué d'une série de tâches définies par l'équipe et devant être complétées avant la clôture du sprint.[\[atlassian\]](#)

TABLE 3.1 – Backlog produit du premier sprint

Acteur	ID tâche	Thème	User Story	Tâche	Estimation (Jours)
Internaute	US01	S'inscrire	S'inscrire afin d'accéder aux services	Rédiger les spécifications et diagrammes du cas «S'inscrire».	4
				Implémenter l'inscription.	
				Tester.	
Utilisateur	US02	Authentification	S'authentifier afin d'accéder au système	Rédiger les diagrammes du cas «S'authentifier».	4
				Implémenter l'authentification et tester.	
	US03	Gérer profil	Gérer mon profil	Implémenter la consultation et modification du profil.	4
				Tester.	
Agence	US04	Gérer vitrine	Gérer la vitrine	Implémenter la gestion de la vitrine.	4
				Tester.	
Administrateur	US05	Gérer agences	Gérer les agences afin de visualiser, ajouter, modifier ou bloquer	Implémenter la gestion des agences.	4
				Tester.	
				Estimation Totale (Jours)	

3.3 Les prototypes des interfaces

Prototyper aide à identifier forces, faiblesses et à limiter les risques.



FIGURE 3.1 – Prototypage de l'interface d'inscription

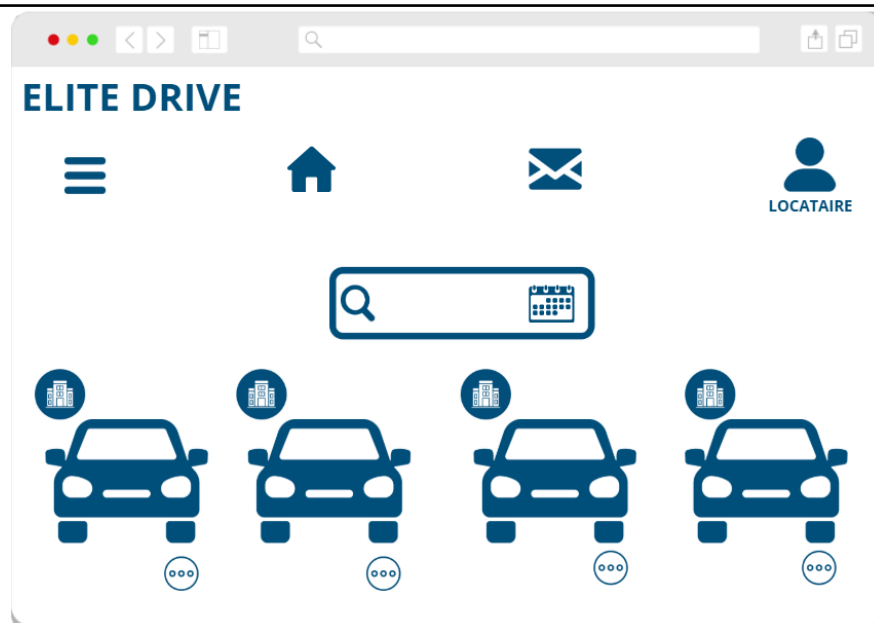


FIGURE 3.2 – Prototypage de l'interface «Utilisateur»

3.4 La spécification fonctionnelle des besoins

On décide d'étudier la spécification fonctionnelle en analysant les diagrammes de cas d'utilisation détaillés du premier sprint, ainsi que les descriptions textuelles et les diagrammes système associés à chaque fonctionnalité.

3.5 Diagramme de cas d'utilisation du premier sprint

La figure suivante présente le diagramme des cas d'utilisation du premier sprint.

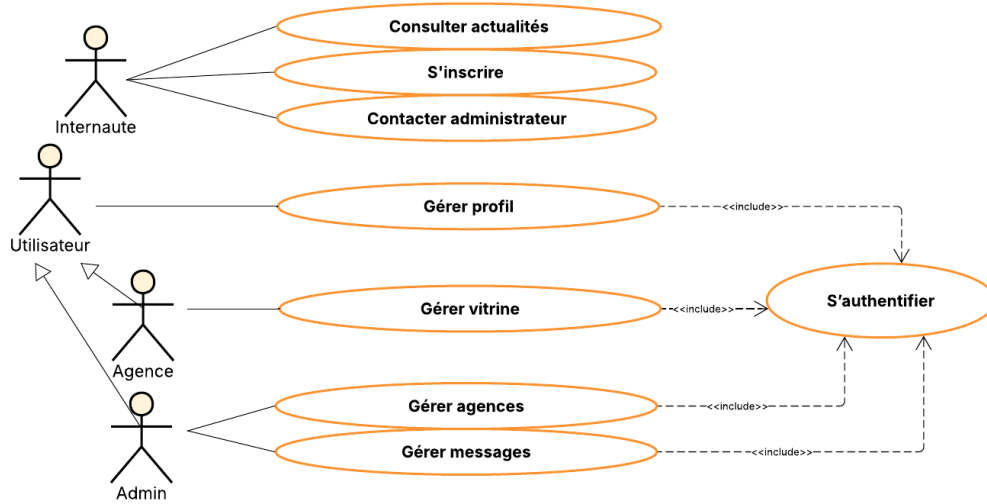


FIGURE 3.3 – Diagramme de cas d'utilisation du premier sprint

3.6 Analyse des cas d'utilisation du premier sprint

À chaque cas d'utilisation, nous allons associer une description textuelle pour détailler le scénario ainsi que les éventuelles situations exceptionnelles.

3.7 Analyse des cas d'utilisation de l'acteur «Internaute»

3.7.1 Analyse de cas d'utilisation «S'inscrire»

Afin de mieux illustrer le déroulement du scénario, nous proposons ci-après une description textuelle accompagnée du diagramme de séquence du cas d'utilisation «**S'inscrire**».

TABLE 3.2 – Description textuelle du cas d'utilisation «S'inscrire»

Élément	Description
Acteur	Internaute
Pré-condition	L'internaute consulte le site de l'application.
Post-condition	L'internaute est enregistré.
Scénario nominal	1. L'internaute lance l'application. 2. L'internaute sélectionne l'option «S'inscrire». 3. Le système affiche le formulaire d'inscription. 4. L'internaute remplit le formulaire. 5. Le système vérifie les données saisies. 6. L'internaute valide l'inscription. 7. Le système vérifie l'unicité de l'e-mail. 8. Le système redirige l'internaute vers le formulaire de connexion.
Scénario alternatif	5.1 Données manquantes ou incorrectes. 5.1.1 Le système affiche un message d'erreur. 5.1.2 Retour à l'étape 4 du scénario nominal. 5.2 Données invalides. 5.2.1 Le système affiche un message d'erreur. 5.2.2 Retour à l'étape 4 du scénario nominal. 7.1 Internaute déjà inscrit. 7.1.1 Le système affiche un message d'erreur. 7.1.2 Retour à l'étape 4 du scénario nominal.

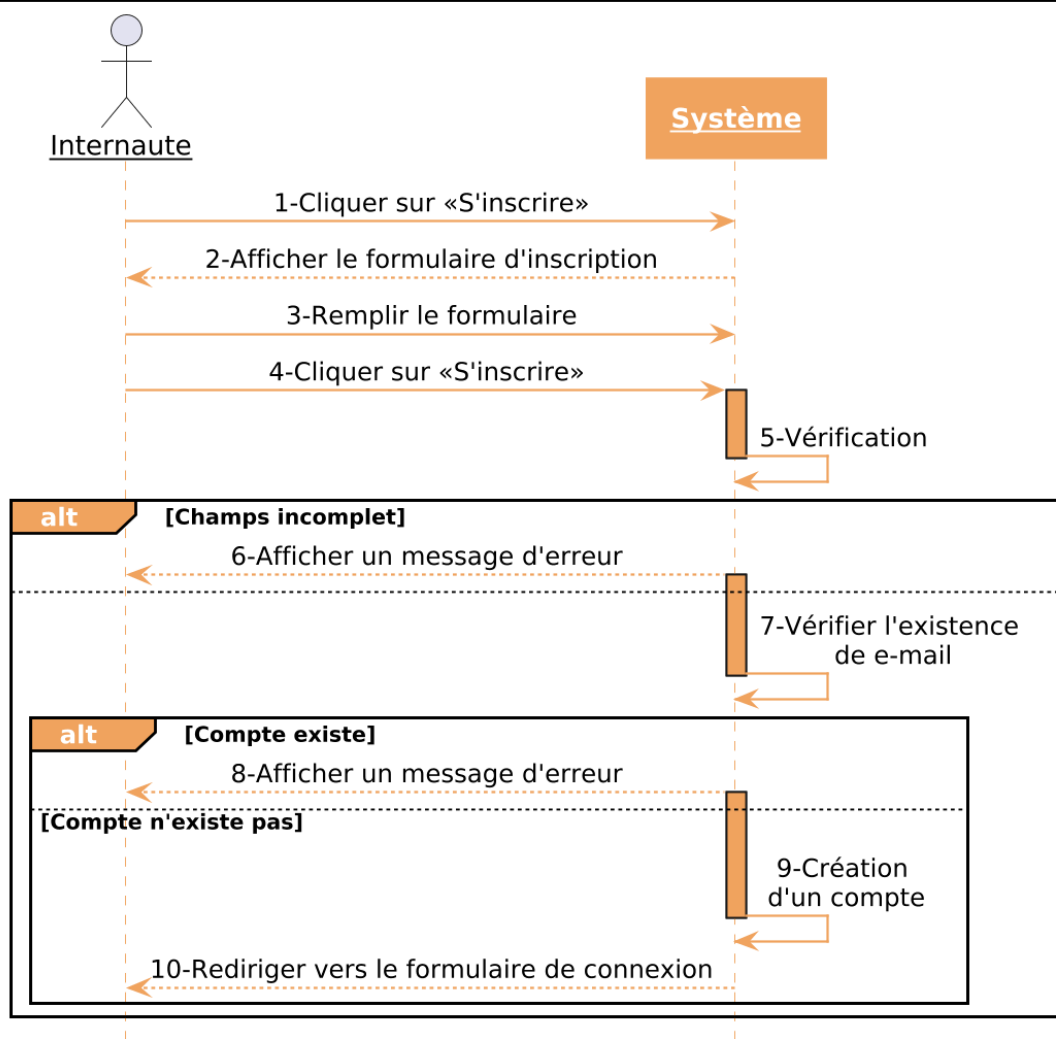


FIGURE 3.4 – Diagramme de séquence système de cas d'utilisation «S'inscrire»

3.8 Analyse des cas d'utilisation de l'acteur «Utilisateur»

3.8.1 Analyse de cas d'utilisation «S'authentifier»

Afin de mieux illustrer le déroulement du scénario, nous proposons ci-après une description textuelle accompagnée du diagramme de séquence du cas d'utilisation «S'authentifier».

TABLE 3.3 – Description textuelle du cas d'utilisation «S'authentifier»

Cas d'utilisation	S'authentifier
Acteur	Utilisateur
Pré-condition	L'utilisateur a déjà un compte.
Post-condition	Utilisateur authentifié.
Scénario nominal	1. L'utilisateur lance l'application. 2. L'utilisateur sélectionne l'option «Se connecter». 3. Le système fait apparaître le formulaire de connexion. 4. L'utilisateur entre son adresse de courrier électronique et son mot de passe. 5. Le système vérifie les données saisies. 6. L'utilisateur sélectionne l'option «Se connecter». 7. Le système examine la présence des informations saisies. 8. Le système redirige l'utilisateur vers son espace personnel.
Scénario alternatif	5.1 L'utilisateur insère des données manquantes et/ou incorrectes. 5.1.1 Le système fait apparaître un message d'erreur. 5.1.2 Retour à l'étape 3 du scénario nominal. 5.2 Le compte est inexistant. 5.2.1 Le système fait apparaître un message d'erreur. 5.2.2 Retour à l'étape 3 du scénario nominal.

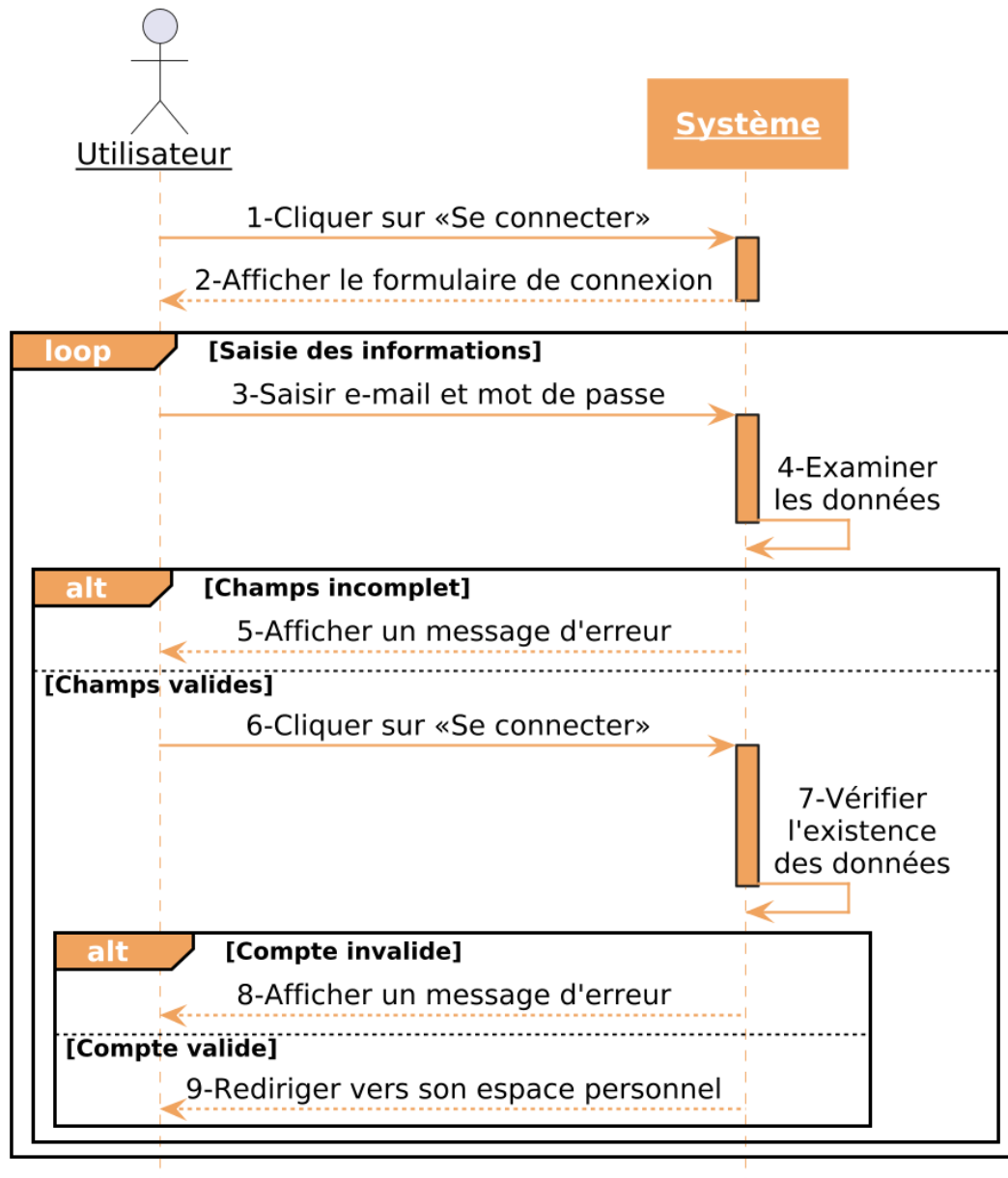


FIGURE 3.5 – Diagramme de séquence système de cas d'utilisation «S'authentifier»

3.8.2 Analyse de cas d'utilisation «Gérer profil»

La section suivante présente le raffinement du cas d'utilisation «Gérer profil» de l'acteur *Utilisateur*.

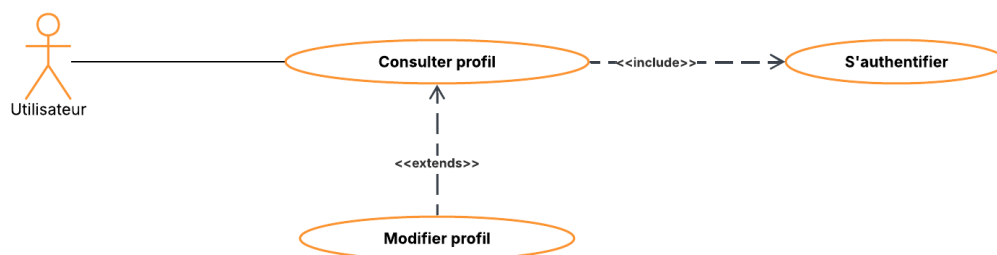


FIGURE 3.6 – Raffinement du cas d'utilisation «Gérer profil»

3.8.2.1 Analyse de cas d'utilisation «Consulter profil»

La section suivante propose une description textuelle et un diagramme de séquence, auxquels s'ajoute l'analyse du cas d'utilisation «**Consulter profil**» de l'acteur **Utilisateur**.

TABLE 3.4 – Description textuelle du cas d'utilisation «Consulter profil»

Élément	Description
Nom	Consulter profil
Acteur	Utilisateur
Description brève	Permet à un utilisateur de consulter son profil.
Précondition	L'utilisateur est authentifié. Le système est accessible.
Postcondition	Ensemble des données affichés.
Enchaînement principal	1. L'utilisateur clique sur «Profil». 2. Le système affiche les informations du profil.
Scénarios alternatifs	Pas d'exceptions

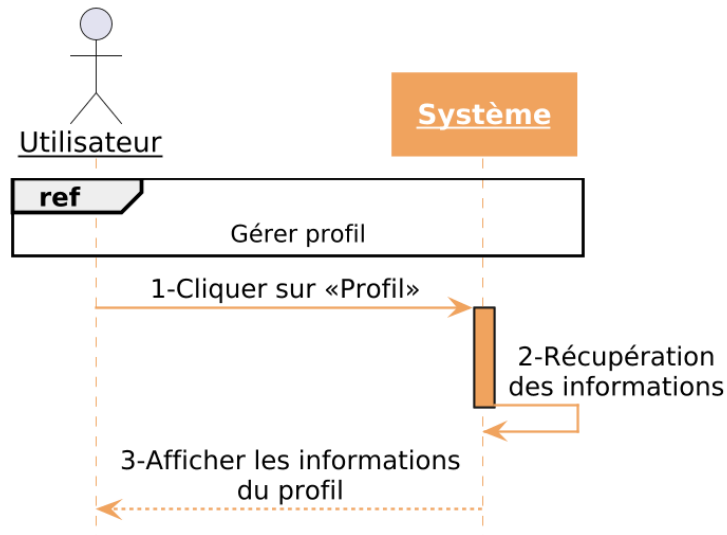


FIGURE 3.7 – Diagramme de séquence système de cas d'utilisation «Consulter profil»

3.8.2.2 Analyse de cas d'utilisation «Modifier profil»

La section suivante propose une description textuelle et un diagramme de séquence, auxquels s'ajoute l'analyse du cas d'utilisation «**Modifier profil**» de l'acteur **Utilisateur**.

TABLE 3.5 – Description textuelle du cas d'utilisation «Modifier profil»

Élément	Description
Nom	Modifier profil
Acteur	Utilisateur
Description brève	Permet à un utilisateur de modifier ses informations personnelles (nom, email, mot de passe, numéro de téléphone).
Précondition	L'utilisateur est authentifié. Le système est accessible.
Postcondition	Les informations du profil sont mises à jour. Les nouvelles données sont enregistrées.
Enchaînement principal	1. L'utilisateur accède à la page «profil». 2. Le système affiche les informations actuelles du profil. 3. L'utilisateur modifie ses informations personnelles. 4. Le système vérifie les données et effectue la modification. 5. Le système affiche un message de confirmation.
Scénarios alternatifs	4.1 Informations invalides. 4.1.1 Le système affiche un message d'erreur. 5.1 Échec lors de l'enregistrement. 5.1.1 Le système affiche un message d'échec.

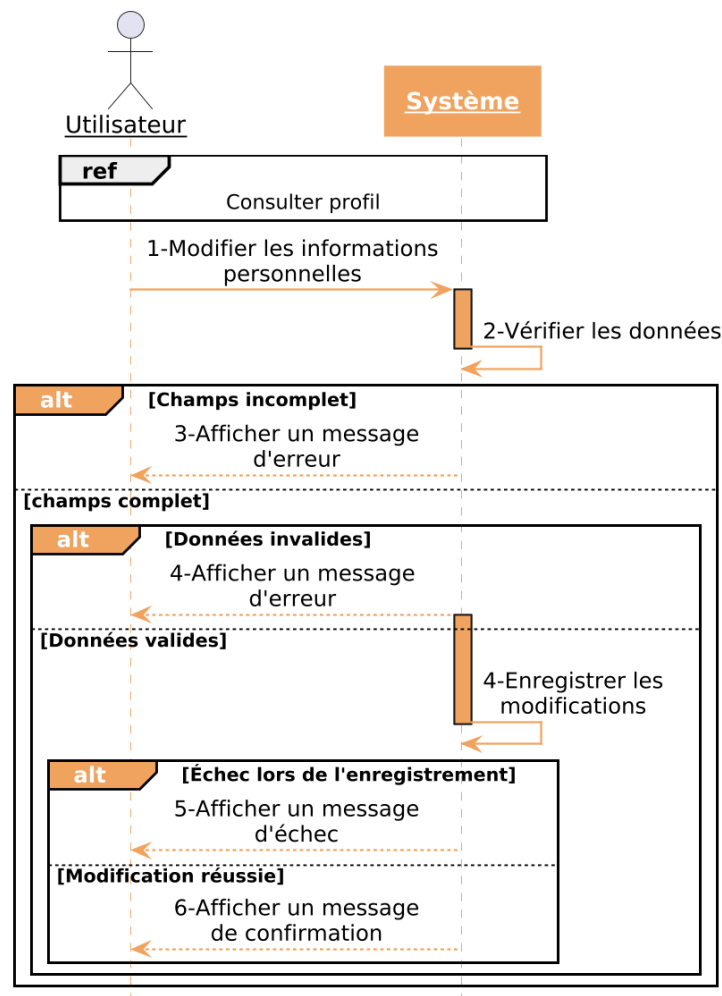


FIGURE 3.8 – Diagramme de séquence système de cas d'utilisation «Modifier profil»

3.9 Analyse des cas d'utilisation de l'acteur «Administrateur»

3.9.1 Analyse de cas d'utilisation «Gérer agences»

Afin de mieux illustrer le déroulement du scénario, nous proposons ci-après un raffinement du cas d'utilisation «**Gérer agences**» de l'acteur *Administrateur*.

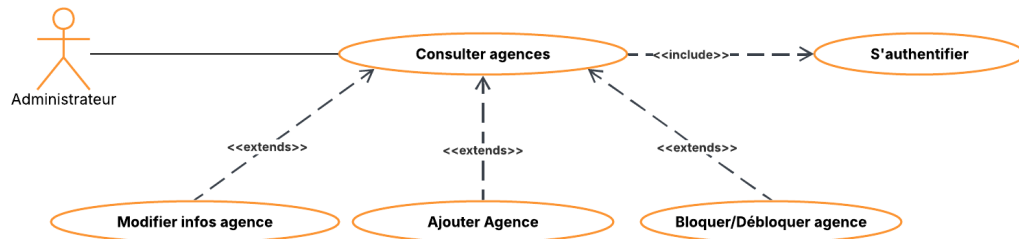


FIGURE 3.9 – Raffinement du cas d'utilisation «Gérer agences»

3.9.1.1 Analyse de cas d'utilisation «Consulter agences»

La section suivante propose une description textuelle et un diagramme de séquence, auxquels s'ajoute l'analyse du cas d'utilisation «**Consulter agences**» de l'acteur **Administrateur**.

TABLE 3.6 – Description textuelle du cas d'utilisation «Consulter agences»

Élément	Description
Nom	Consulter agences
Acteur	Administrateur
Description brève	Permet à un administrateur de consulter les informations des agences.
Précondition	L'administrateur est authentifié. Le système est accessible.
Postcondition	Ensemble des données affichés.
Enchaînement principal	1. L'administrateur clique sur «Agences». 2. Le système affiche les informations des agences.
Scénarios alternatifs	Pas d'exceptions

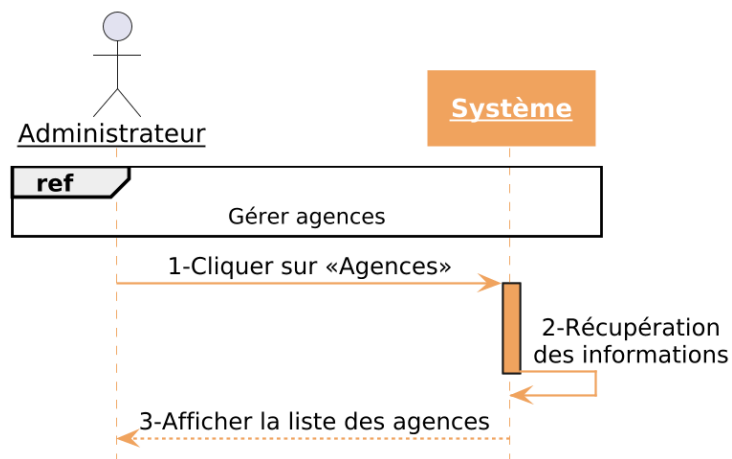


FIGURE 3.10 – Diagramme de séquence système de cas d'utilisation «Consulter agences»

3.9.1.2 Analyse de cas d'utilisation «Modifier infos agence»

La section suivante propose une description textuelle et un diagramme de séquence, auxquels s'ajoute l'analyse du cas d'utilisation «**Modifier infos agence**» de l'acteur **Administrateur**.

TABLE 3.7 – Description textuelle du cas d'utilisation «Modifier infos agence»

Élément	Description
Nom	Modifier infos agence
Acteur	Administrateur
Description brève	Permet à un administrateur de modifier les informations d'une agence.
Précondition	L'administrateur est authentifié. Le système est accessible.
Postcondition	Les informations de l'agence sont mises à jour. Les nouvelles données sont enregistrées.
Enchaînement principal	1. L'administrateur accède à la page «gestion des agences». 2. Le système affiche les informations actuelles de l'agence. 3. L'administrateur modifie les informations de l'agence. 4. Le système vérifie les données et effectue la modification. 5. Le système affiche un message de confirmation.
Scénarios alternatifs	4.1 Informations invalides. 4.1.1 Le système affiche un message d'erreur. 5.1 Échec lors de l'enregistrement. 5.1.1 Le système affiche un message d'échec.

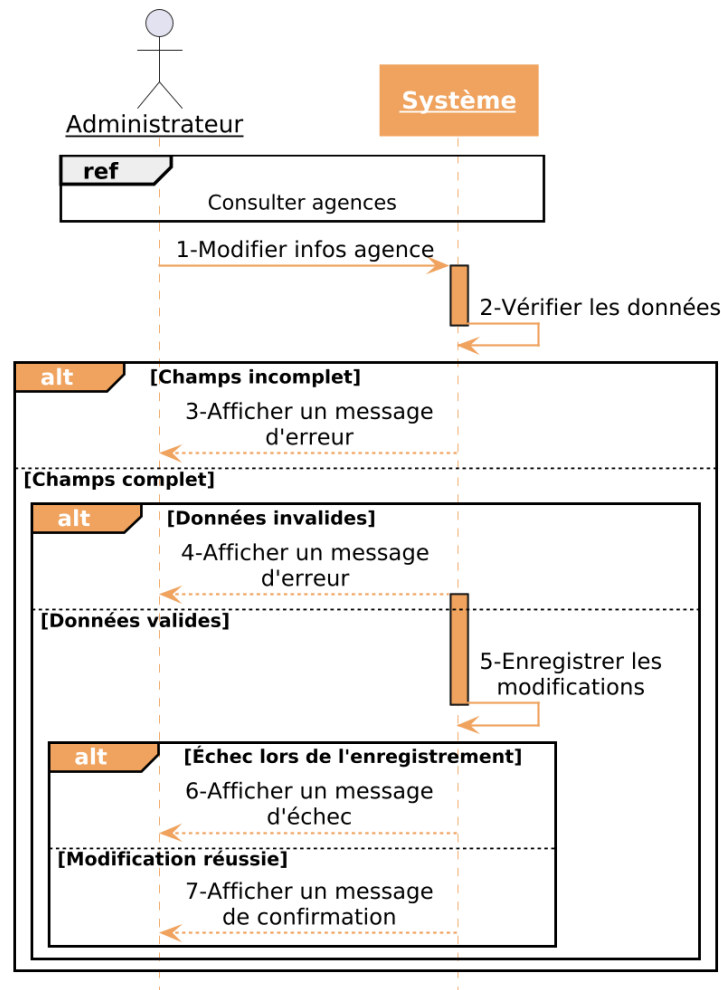


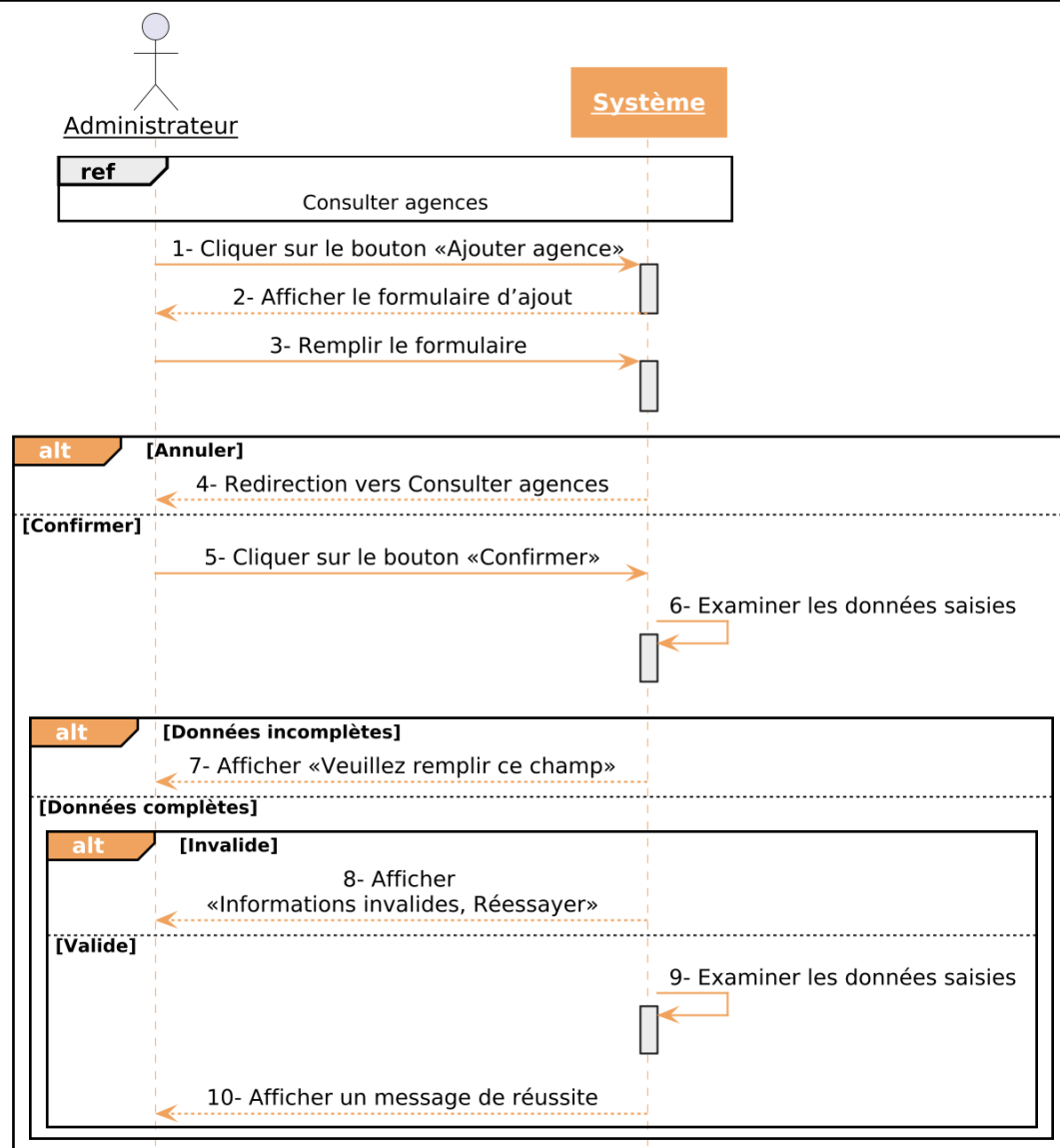
FIGURE 3.11 – Diagramme de séquence système de cas d'utilisation «Modifier infos agence»

3.9.2 Analyse de cas d'utilisation «Ajouter agence»

Dans la suite, nous présentons une description textuelle accompagnée d'un diagramme de séquence du cas d'utilisation «Ajouter agence» de l'acteur **Administrateur**.

TABLE 3.8 – Description textuelle du cas d'utilisation «Ajouter agence»

Élément	Description
Nom	ajouter agence
Acteur	Administrateur
Description brève	Permet à l'administrateur d'ajouter une agence.
Précondition	L'administrateur est authentifié. Le système est accessible.
Postcondition	Une agence est ajoutée.
Enchaînement principal	1. L'administrateur accède à la page de gestion. 2. Le système affiche la liste des agence. 3. L'administrateur clique sur «ajouter une agence». 4. Le système affiche le formulaire d'ajout. 5. L'administrateur saisit les informations nécessaires. 6. Le système vérifie les données. 7. Le système enregistre l'agence. 8. Le système affiche un message de confirmation.
Scénario alternatif	6.1 Informations invalides. 6.1.1 Le système affiche un message d'erreur. 7.1 Erreur lors de l'enregistrement. 7.1.1 Le système affiche un message d'échec.



3.9.3 Analyse de cas d'utilisation «Bloquer/Débloquer agence»

Dans la suite, nous présentons une description textuelle accompagnée d'un diagramme de séquence du cas d'utilisation «**Bloquer/Débloquer agence**» de l'acteur **Administrateur**.

TABLE 3.9 – Description textuelle du cas d'utilisation «Bloquer/Débloquer agence»

Élément	Description
Nom	Bloquer/Débloquer agence
Acteur	Administrateur
Description brève	Permet à l'administrateur de bloquer ou de débloquent une agence afin de désactiver ou d'activer son accès.
Précondition	L'administrateur est authentifié. L'agence existe dans le système.
Postcondition	L'agence est bloquée/Débloquée et peut/ ne peut plus accéder au système.
Enchaînement principal	1. L'administrateur consulte la liste des agences. 2. L'administrateur sélectionne une agence. 3. L'administrateur clique sur «Bloquer/Débloquer». 4. Le système demande confirmation. 5. L'administrateur confirme. 6. Le système bloque/débloquent l'agence. 7. Le système affiche un message de confirmation.
Scénario alternatif	5.1 Annulation de l'action. 5.1.1 Aucune modification n'est effectuée. 6.1 Erreur lors du blocage/déblocage. 6.1.1 Le système affiche un message d'échec.

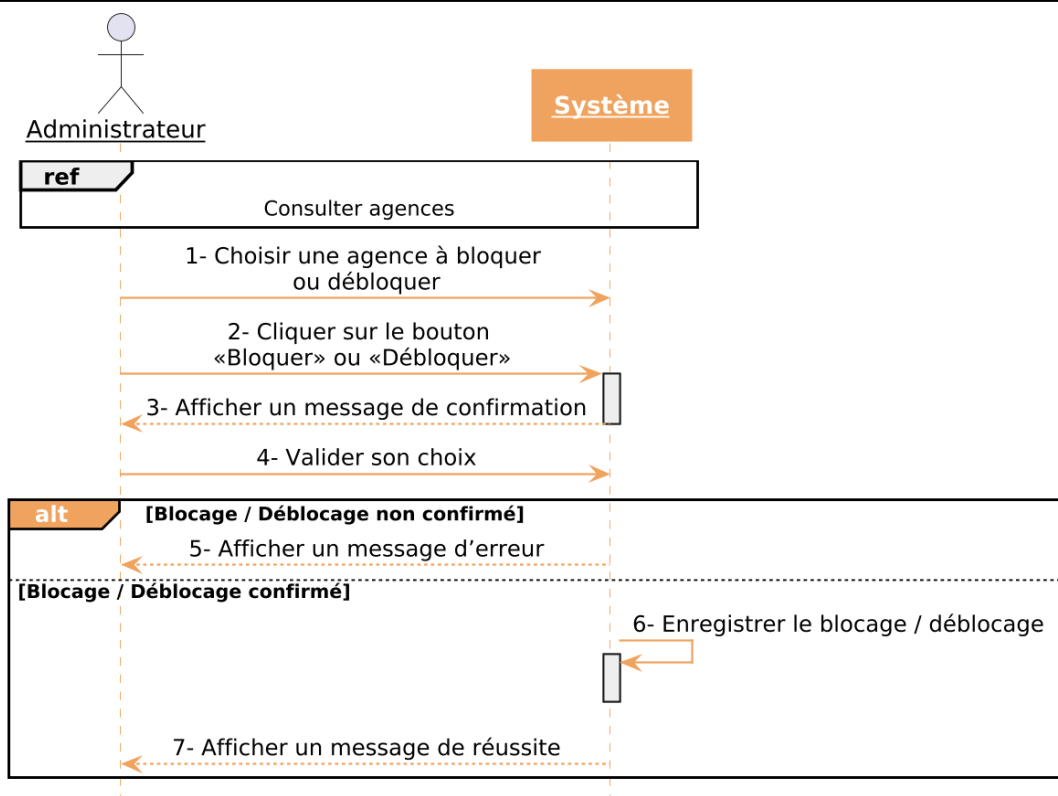


FIGURE 3.13 – Diagramme de séquence système de cas d'utilisation «Bloquer agences»

3.10 Analyse des cas d'utilisation de l'acteur «Agence»

3.10.1 Analyse de cas d'utilisation «Gérer vitrine»

Pour mieux analyser le déroulement du scénario, nous proposons ci-après un raffinement accompagné d'une description textuelle et un diagramme de séquence du cas d'utilisation «**Gérer vitrine**» de l'acteur *Agence*.

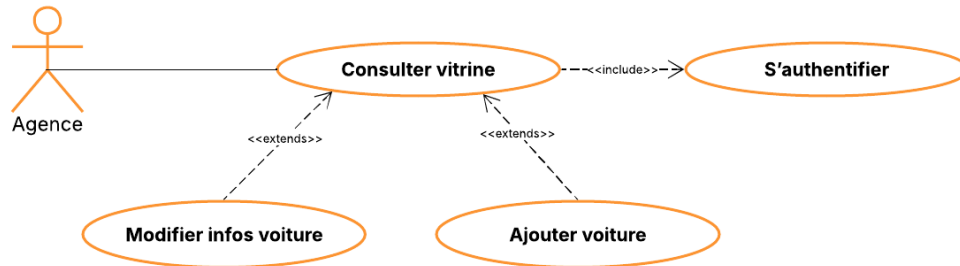


FIGURE 3.14 – Raffinement du cas d'utilisation «Gérer vitrine»

3.10.1.1 Analyse de cas d'utilisation «Consulter vitrine»

La section suivante propose une description textuelle et une diagramme de séquence, auxquels s'ajoute l'analyse du cas d'utilisation «**Consulter vitrine**» de l'acteur *Agence*.

TABLE 3.10 – Description textuelle du cas d'utilisation «Consulter vitrine»

Élément	Description
Nom	Consulter vitrine
Acteur	Agence
Description brève	Permet à une agence de consulter sa vitrine.
Précondition	L'agence est authentifié. Le système est accessible.
Postcondition	La liste des voitures est affichés ainsi que leurs informations.
Enchaînement principal	1. L'Agence clique sur «vitrine». 2. Le système affiche les voitures enregistrées dans la vitrine ainsi que leurs informations.
Scénarios alternatifs	pas d'exceptions

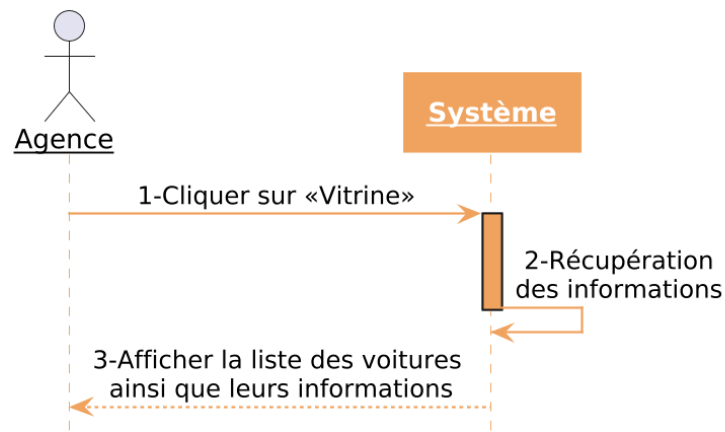


FIGURE 3.15 – Diagramme de séquence système de cas d'utilisation «Consulter vitrine»

3.10.1.2 Analyse de cas d'utilisation «Modifier informations voiture»

La section suivante propose une description textuelle et un diagramme de séquence, auxquels s'ajoute l'analyse du cas d'utilisation «**Modifier informations voiture**» de l'acteur **Agence**.

TABLE 3.11 – Description textuelle du cas d'utilisation «Modifier informations voiture»

Élément	Description
Nom	Modifier informations voiture
Acteur	Agence
Description brève	Permet à un Agence de consulter et de modifier ses informations voiture (nom, email, adresse, numéro de téléphone).
Précondition	L'agence est authentifié. Le système est accessible.
Postcondition	Les informations du voiture sont mises à jour. Les nouvelles données sont enregistrées.
Enchaînement principal	1. L'agence accède à la page «vitrine». 2. Le système affiche les voiture actuelles. 3. L'agence modifie les informations d'une voiture. 4. Le système vérifie les données et effectue l'action. 5. Le système affiche un message de confirmation.
Scénarios alternatifs	4.1 Informations invalides. 4.1.1 Le système affiche un message d'erreur. 5.1 Échec lors de l'enregistrement. 5.1.1 Le système affiche un message d'échec.

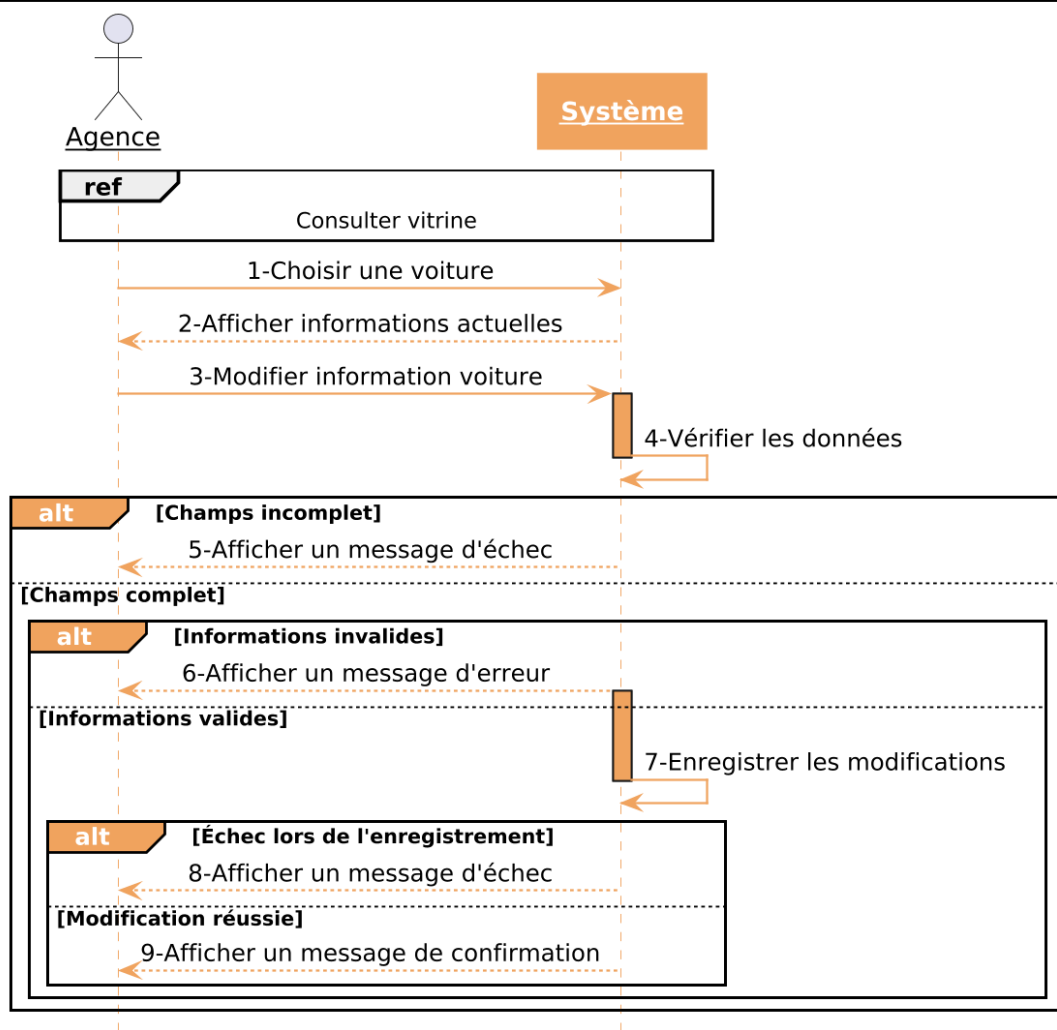


FIGURE 3.16 – Diagramme de séquence système de cas d'utilisation «Modifier infos voiture»

3.10.2 Analyse de cas d'utilisation «Ajouter voiture»

Dans la suite, nous présentons une description textuelle accompagnée d'un diagramme de séquence du cas d'utilisation «**Ajouter voiture**» de l'acteur *Agence*.

TABLE 3.12 – Description textuelle du cas d'utilisation «Ajouter voiture»

Élément	Description
Nom	Ajouter voiture
Acteur	Agence
Description brève	Permet à l'agence d'ajouter une voiture.
Précondition	L'agence est authentifié. Le système est accessible.
Postcondition	Une voiture est ajoutée.
Enchaînement principal	1. L'agence accède à la page «Vitrine». 2. Le système affiche la liste des voitures. 3. L'agence choisit d'ajouter une voiture. 4. L'agence saisit les informations nécessaires. 5. Le système vérifie les données. 6. Le système enregistre l'voiture. 7. Le système affiche une message de confirmation.
Scénario alternatif	5.1 Informations invalides. 5.1.1 Le système affiche une message d'erreur. 6.1 Erreur lors de l'enregistrement. 6.1.1 Le système affiche une message d'échec.

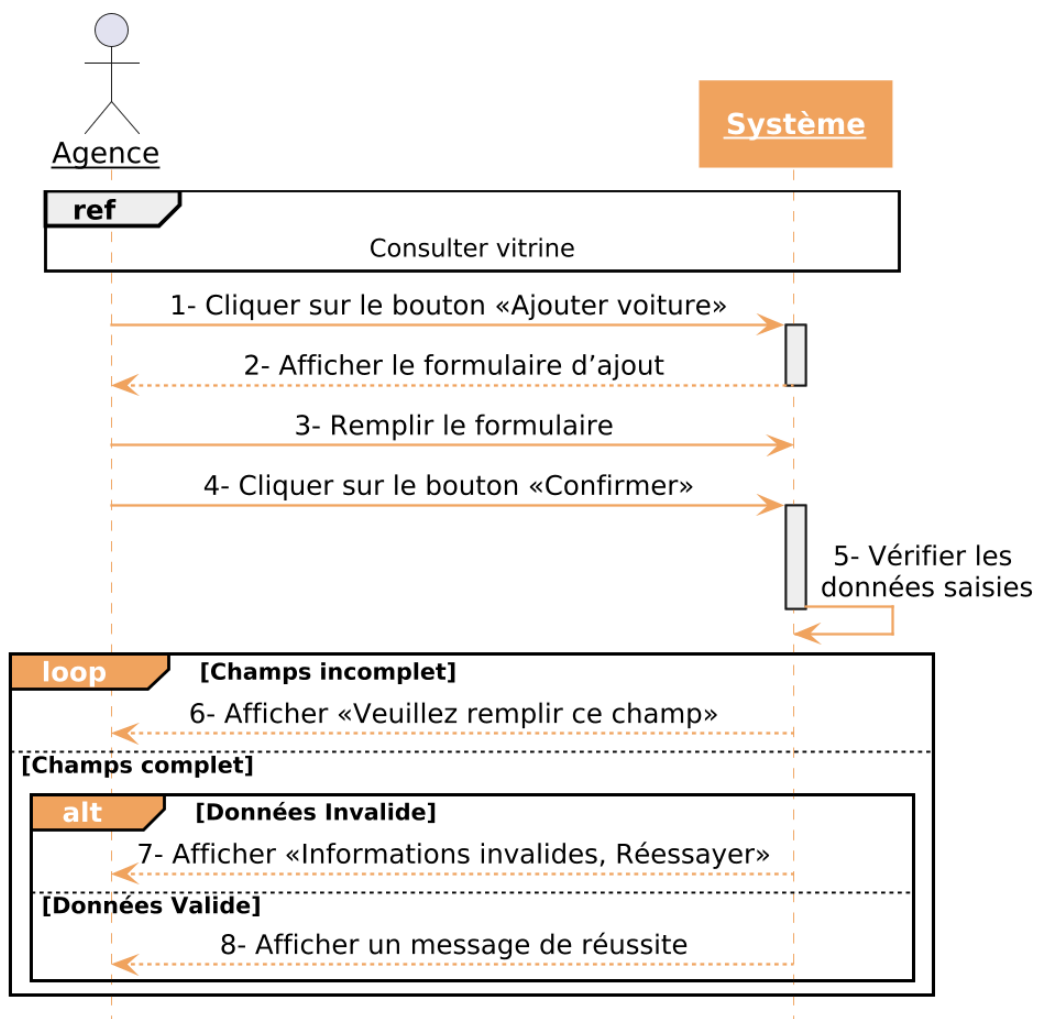


FIGURE 3.17 – Diagramme de séquence système de cas d'utilisation «Ajouter voiture»

3.11 Conception des cas d'utilisation

Nous illustrerons dans cette section la conception de ce premier sprint à travers la réalisation d'un diagramme des classes pour chaque module, ainsi que d'un diagramme de séquence détaillé correspondant à chaque cas d'utilisation analysé

3.11.1 Diagramme des classes participantes

Un diagramme des classes participantes est une représentation UML utilisée dans la phase d'analyse pour décrire la réalisation d'un cas d'utilisation spécifique. Il met en évidence les différentes classes impliquées dans l'interaction, ainsi que leurs responsabilités et leurs relations, afin de structurer le comportement du système selon une approche orientée objet.[[stackoverflow](#)]

Ce diagramme repose généralement sur trois catégories principales de classes d'analyse :

1. **Les classes frontières (ou classes de dialogue)** : Elles assurent l'interaction entre l'utilisateur et le système. Elles contiennent des attributs correspondant aux données affichées ou saisies (champs de formulaires, résultats, etc.) et des opérations représentant les actions effectuées par l'utilisateur via l'interface homme-machine (IHM).[[stackoverflow](#)]
2. **Les classes de contrôle** : Elles orchestrent le déroulement du cas d'utilisation. Elles implémentent la logique applicative, coordonnent les échanges entre les classes frontières et les classes entités, et encapsulent les règles métier ainsi que les traitements du système.[[stackoverflow](#)]
3. **Les classes entités** : Elles représentent les données métier du système. Ces classes contiennent des attributs persistants correspondant aux informations stockées (par exemple dans une base de données) et peuvent inclure des opérations liées à la gestion de ces données.[[stackoverflow](#)]

3.11.1.1 Diagramme des classes participantes par acteur «Internaute»

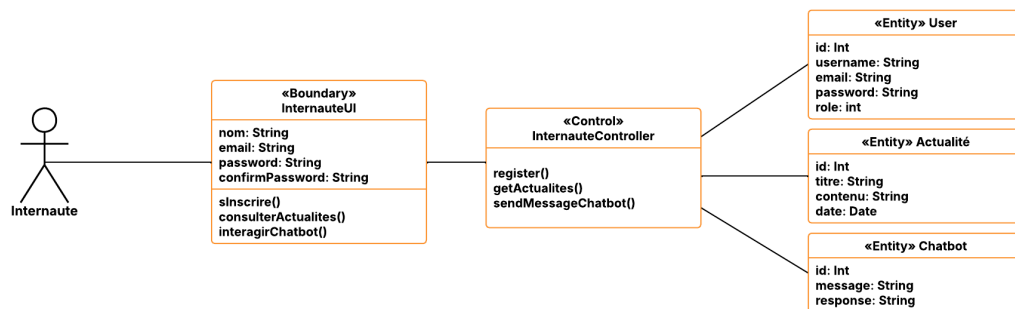


FIGURE 3.18 – Diagramme des classes participantes par acteur Internaute

3.11.1.2 Diagramme des classes participantes par acteur «Utilisateur»

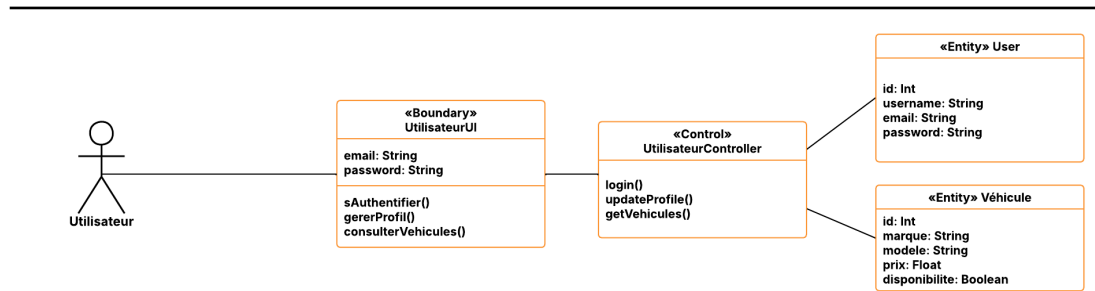


FIGURE 3.19 – Diagramme des classes participantes par acteur Utilisateur

3.11.1.3 Diagramme des classes participantes par acteur «Agence»

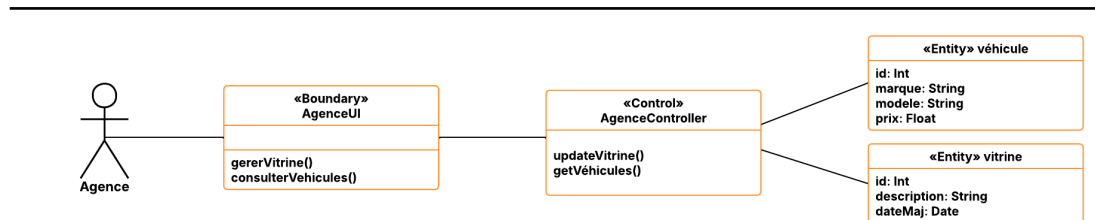


FIGURE 3.20 – Diagramme des classes participantes par acteur Agence

3.11.1.4 Diagramme des classes participantes par acteur «Administrateur»

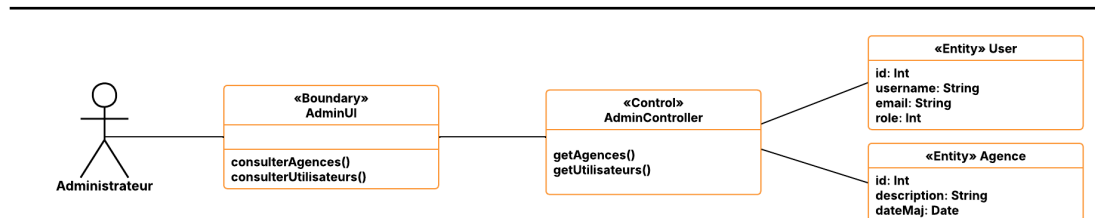


FIGURE 3.21 – Diagramme des classes participantes par acteur Administrateur

3.12 Diagrammes de séquence détaillés du premier sprint

"Les diagrammes de séquence sont un outil essentiel en génie logiciel pour visualiser les interactions entre les objets d'un système au fil du temps. Ils aident à comprendre le comportement dynamique d'un système, à concevoir son architecture et à documenter

des processus complexes. Ce guide couvrira les concepts clés, les recommandations, un exemple concret et une étude de cas pour vous aider à maîtriser les diagrammes de séquence.”[cybermedian]

3.12.1 Diagrammes de séquence détaillés de l’acteur Internaute

3.12.1.1 Diagrammes de séquence détaillé du cas d’utilisation « Consulter actualités »

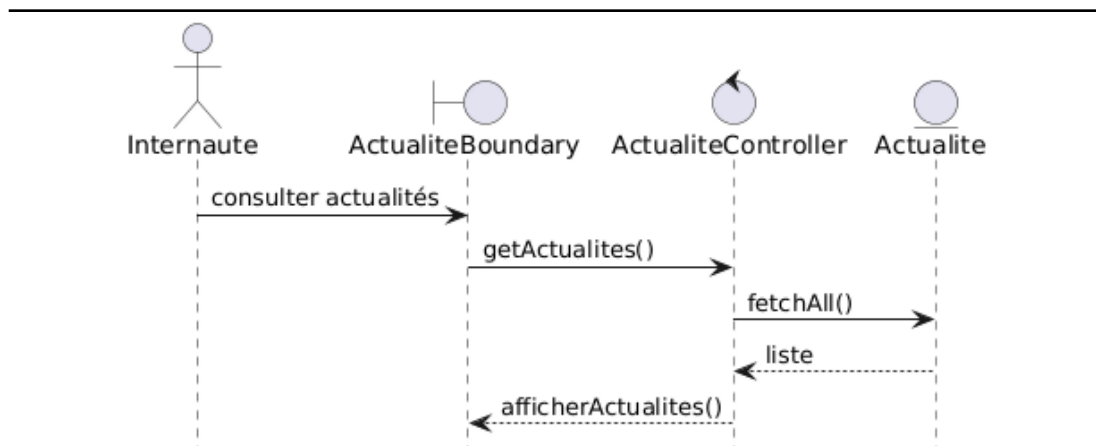


FIGURE 3.22 – Diagramme de séquence détaillé du cas d’utilisation « Consulter actualités »

3.12.1.2 Diagrammes de séquence détaillé du cas d'utilisation « S'inscrire »

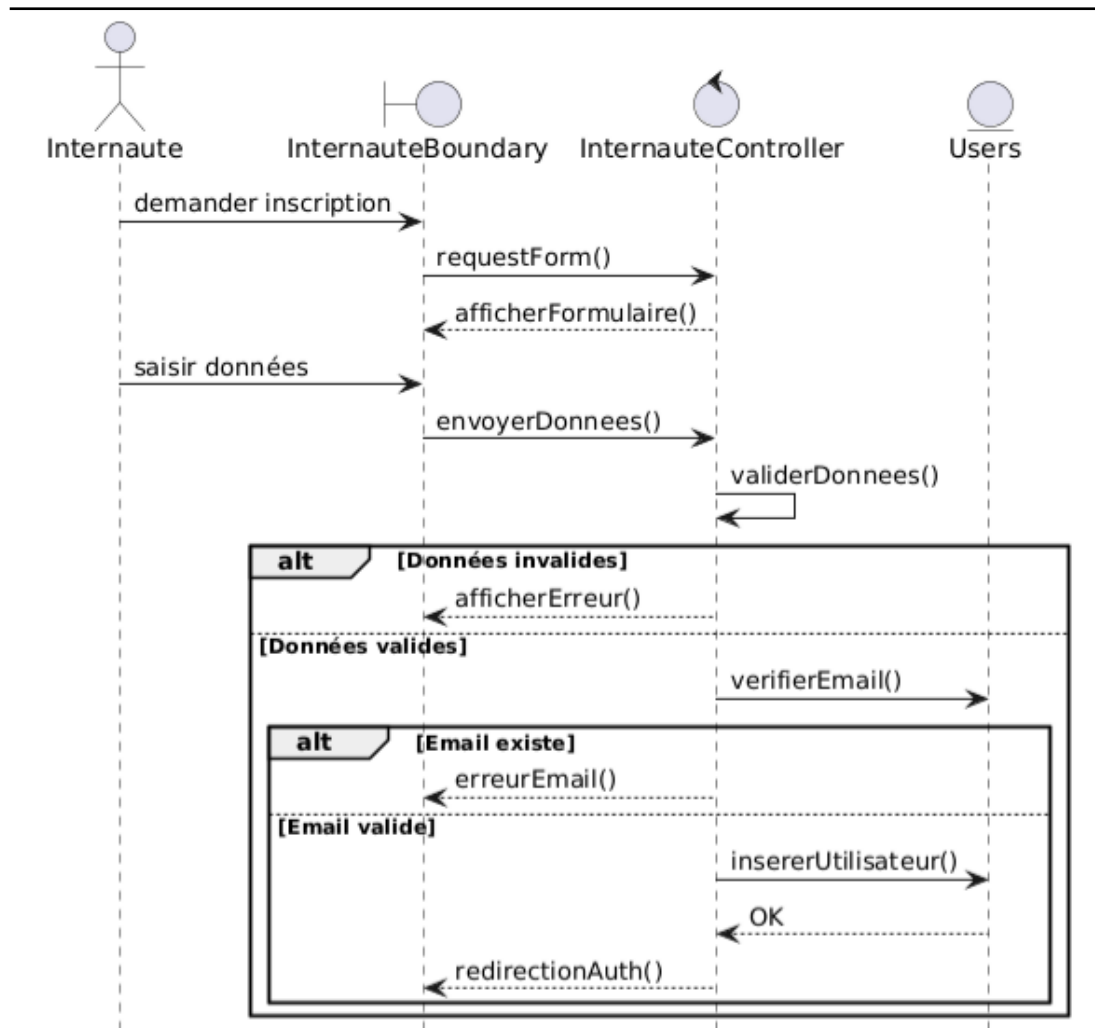


FIGURE 3.23 – Diagramme de séquence détaillé du cas d'utilisation « S'inscrire »

3.12.2 Diagrammes de séquence détaillés de l'acteur Utilisateur

3.12.2.1 Diagrammes de séquence détaillé du cas d'utilisation « S'authentifier »

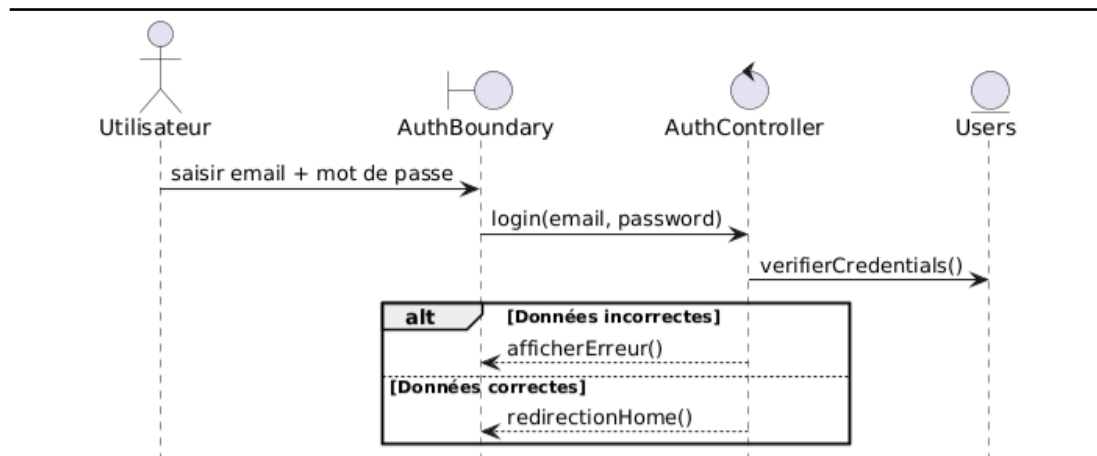


FIGURE 3.24 – Diagramme de séquence détaillé du cas d'utilisation « S'authentifier »

3.12.2.2 Diagrammes de séquence détaillé du cas d'utilisation « Interagir avec Chatbot »

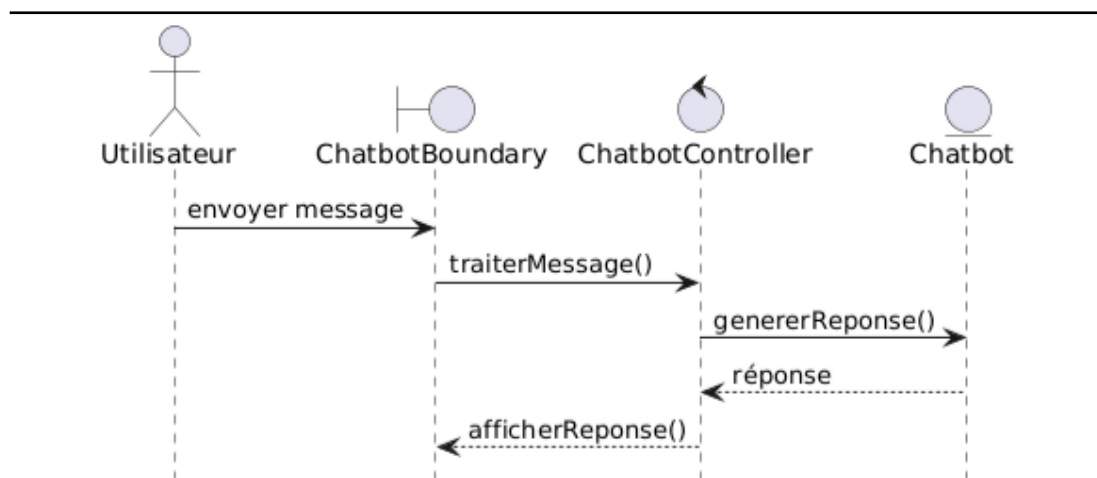


FIGURE 3.25 – Diagramme de séquence détaillé du cas d'utilisation « Interagir avec Chatbot »

3.12.2.3 Diagrammes de séquence détaillé du cas d'utilisation « Consulter profil »

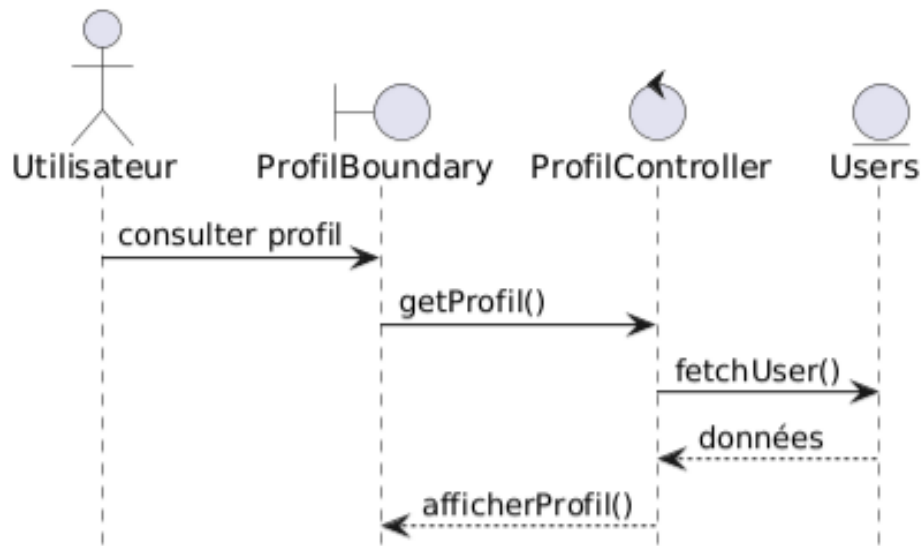


FIGURE 3.26 – Diagramme de séquence détaillé du cas d'utilisation « Consulter profil »

3.12.3 Diagrammes de séquence détaillés de l'acteur Administrateur

3.12.3.1 Diagrammes de séquence détaillé du cas d'utilisation « Ajouter agence »

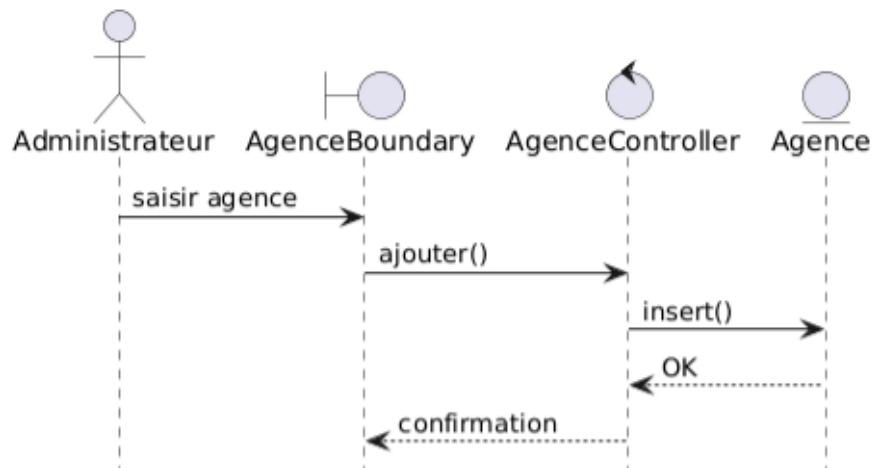


FIGURE 3.27 – Diagramme de séquence détaillé du cas d'utilisation « Ajouter agence »

3.12.3.2 Diagrammes de séquence détaillé du cas d'utilisation « Bloquer/Débloquer agence »

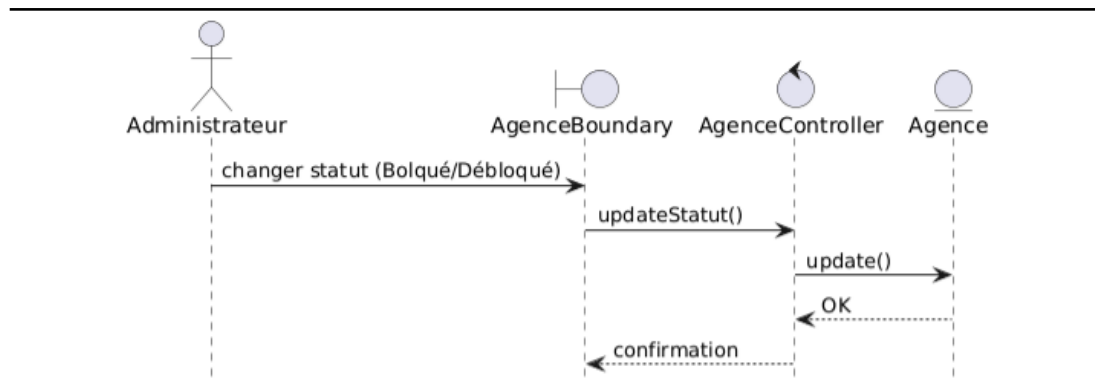


FIGURE 3.28 – Diagramme de séquence détaillé du cas d'utilisation « Bloquer/Débloquer agence »

3.12.3.3 Diagrammes de séquence détaillé du cas d'utilisation « Ajouter utilisateur »

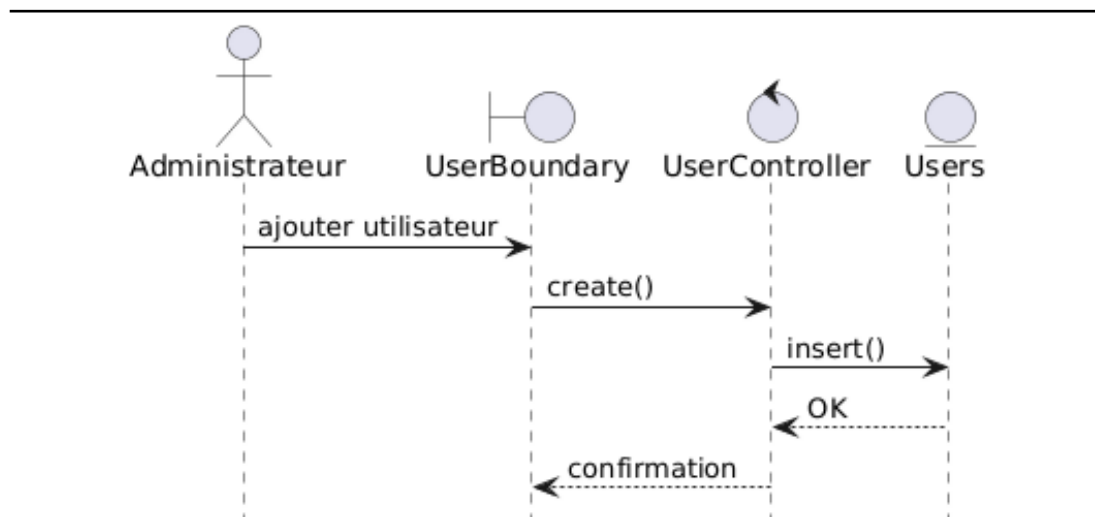


FIGURE 3.29 – Diagramme de séquence détaillé du cas d'utilisation « Ajouter utilisateur »

3.12.3.4 Diagrammes de séquence détaillé du cas d'utilisation « Bloquer/Débloquer utilisateur »

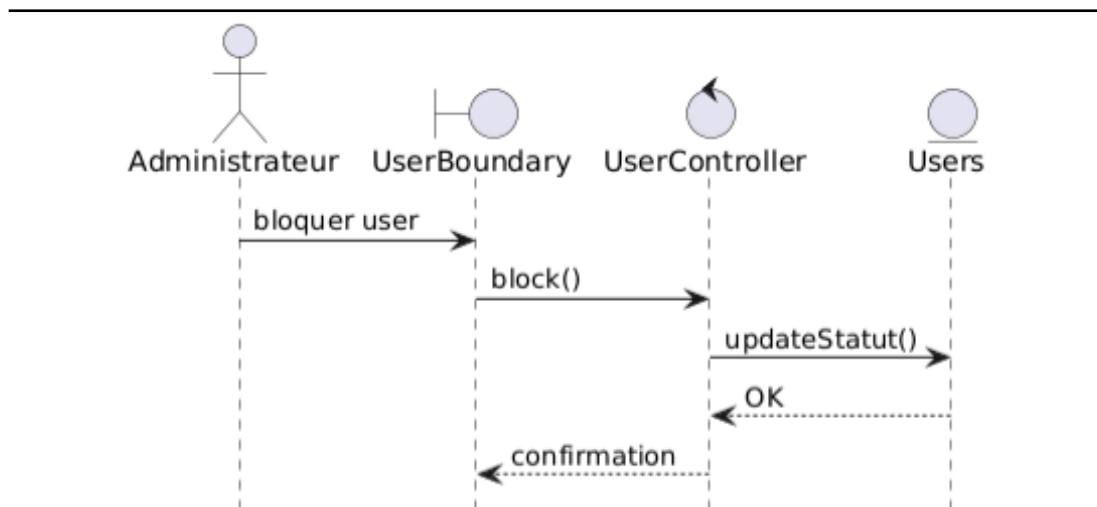


FIGURE 3.30 – Diagramme de séquence détaillé du cas d'utilisation « Bloquer/Débloquer utilisateur »

3.13 Diagramme de classe du premier sprint

Un diagramme de classes est une représentation graphique qui identifie les principales classes d'un système, leurs attributs, leurs méthodes et les relations qui les lient. Chaque classe est figurée par un rectangle divisé en trois parties (nom, attributs, opérations), et les liens entre classes traduisent des relations comme l'héritage, l'association ou la dépendance. Ce type de diagramme permet de structurer et de visualiser l'architecture logicielle, en offrant une vue claire des entités et de leurs interactions essentielles.[Lucidchart]

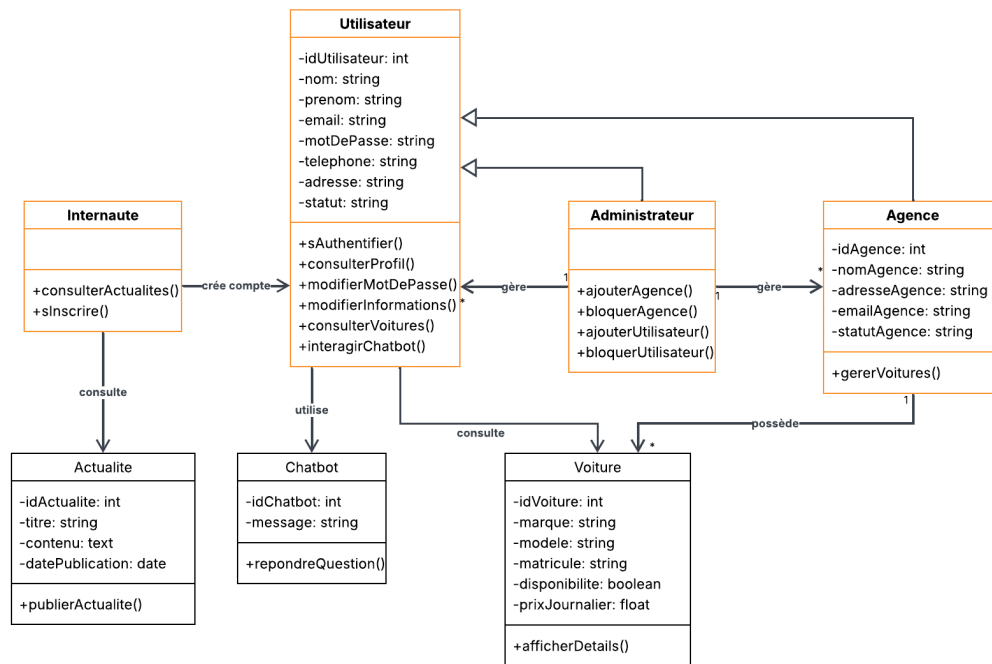


FIGURE 3.31 – Diagramme de classe du premier sprint

3.14 Implémentation

Le module d'implémentation d'un projet prévoit l'implémentation des fonctions qui ont été réalisées au cours du premier sprint.

3.14.1 Tables de la base de données

Le tableau ci-joint décrit le répertoire de données afin d'illustrer les attributs que l'on retrouve dans les différentes classes, ainsi que leurs étiquettes et leurs types.

3.14.1.1 Tableau « users »

TABLE 3.13 – Tableau « users »

Champs	Type	Libellé	Contraintes
id	Int	Identifiant unique de l'utilisateur	PRIMARY KEY, NOT NULL
name	String	Le nom de l'utilisateur	NOT NULL
email	String	L'email de l'utilisateur	NOT NULL, UNIQUE
email_verified_at	Timestamp	La date de vérification de l'email	NULLABLE
password	String	Le mot de passe de l'utilisateur	NOT NULL
must_change_password	Tinyint(1)	Indique si l'utilisateur doit changer son mot de passe	NOT NULL, DEFAULT 0
role	Enum	Le rôle de l'utilisateur (client / agency_admin / super_admin)	NOT NULL
agency_id	BigInt	Identifiant unique de l'agence	FOREIGN KEY, NULLABLE
phone	String	Le numéro de téléphone de l'utilisateur	NULLABLE
address	Text	L'adresse de l'utilisateur	NULLABLE
driver_license	String	Le permis de conduire de l'utilisateur	NULLABLE
is_suspended	Boolean	Indique si l'utilisateur est suspendu	NOT NULL, DEFAULT 0
suspension_reason	Text	La raison de la suspension de l'utilisateur	NULLABLE
suspended_at	Timestamp	La date de suspension de l'utilisateur	NULLABLE
remember_token	String	Token de mémorisation	NULLABLE
created_at	Timestamp	Date de création	NOT NULL
updated_at	Timestamp	Date de modification	NOT NULL

3.14.1.2 Tableau « vehicles »

TABLE 3.14 – Tableau « vehicles »

Champs	Type	Libellé	Contraintes
id	Int	Identifiant unique du véhicule	PRIMARY KEY, NOT NULL
brand	String	La marque du véhicule	NOT NULL
model	String	Le modèle du véhicule	NOT NULL
year	Int	L'année du véhicule	NOT NULL
mileage	Int	Le kilométrage du véhicule	NOT NULL
daily_price	Decimal(8,2)	Le prix journalier de location	NOT NULL
caution_amount	Decimal(10,2)	Le montant de la caution fixé par l'agence	NULLABLE
license_plate	String	La plaque d'immatriculation	NOT NULL, UNIQUE
color	String	La couleur du véhicule	NOT NULL
seats	Int	Le nombre de places	NOT NULL
transmission	Enum	Le type de transmission (manual / automatic)	NOT NULL
fuel_type	Enum	Le type de carburant (petrol / diesel / electric / hybrid)	NOT NULL
status	Enum	L'état du véhicule (available / reserved / in_use / returned)	NOT NULL, DEFAULT available
agency_id	BigInt UNSIGNED	Identifiant unique de l'agence	FOREIGN KEY, NOT NULL
created_at	Timestamp	Date de création	NULLABLE
updated_at	Timestamp	Date de modification	NULLABLE
images	Text	Les images du véhicule	NULLABLE

3.14.2 Réalisation des interfaces du premier sprint

Nous présentons dans cette section les interfaces graphiques développées lors du premier sprint.

3.14.2.1 Interface d'inscription :

Elite Drive

Rejoignez Elite Drive

Créez votre compte et profitez d'une expérience de location exceptionnelle

- Plus de 50 véhicules premium disponibles
- Réservation en ligne rapide et sécurisée
- Assistance client disponible 24/7
- Assurance complète incluse

© 2026 Elite Drive. Tous droits réservés.

Créer un compte
Commencez votre voyage avec Elite Drive

Prénom Nom

Adresse e-mail

Numéro de téléphone

Type de compte

Client Agence

Adresse (optionnel)

Permis de conduire (optionnel)

Mot de passe

Confirmer le mot de passe

☐ J'accepte les conditions d'utilisation et la politique de confidentialité

Créer mon compte

OU S'INSCRIRE AVEC

Vous avez déjà un compte? [Se connecter](#)

FIGURE 3.32 – Interface d'inscription

3.14.2.2 Interface d'authentification :

Elite Drive

Bienvenue sur Elite Drive

Louez les meilleurs véhicules premium en Tunisie avec un service exceptionnel

- ✓ Plus de 50 véhicules premium disponibles
- ✓ Réservation en ligne rapide et sécurisée
- ✓ Assistance client disponible 24/7
- ✓ Assurance complète incluse

© 2026 Elite Drive. Tous droits réservés.

Bon retour
Connectez-vous pour gérer vos réservations

Adresse e-mail
test@location.tn

Mot de passe
••••••••

☐ Se souvenir de moi [Mot de passe oublié?](#)

Se connecter

Vous n'avez pas de compte? [S'inscrire](#)

FIGURE 3.33 – Interface d'authentification

3.14.2.3 Interface profil :

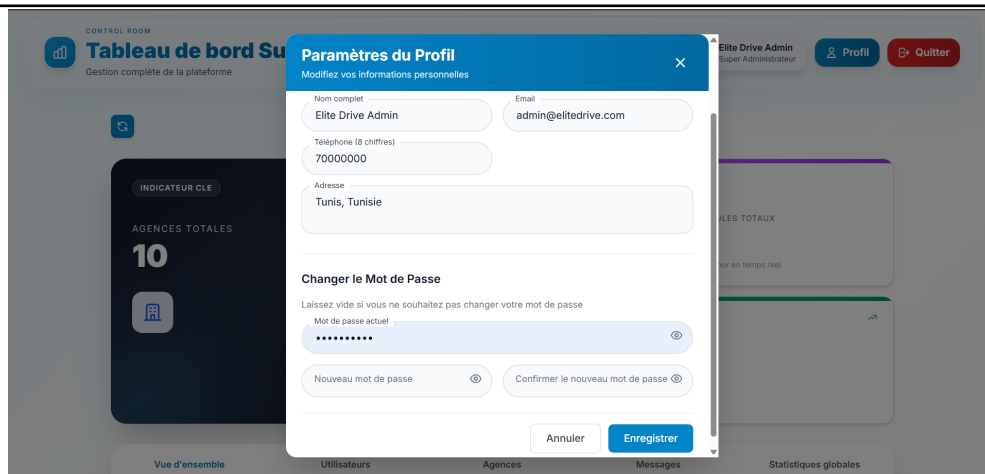


FIGURE 3.34 – Interface profil

3.14.2.4 Interface chatbot :

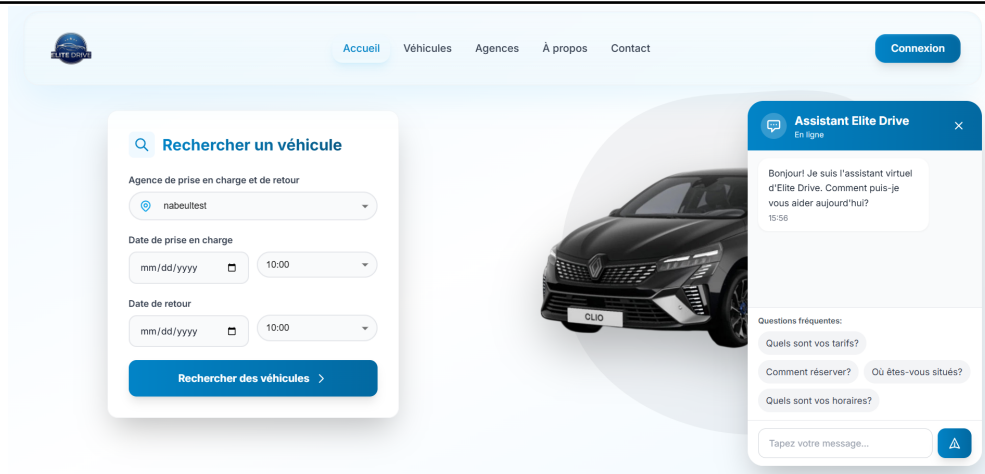


FIGURE 3.35 – Interface chatbot

3.14.2.5 Interface vitrine :

Vue d'ensemble

Réservations

Vitrine

Reclamations

Statistiques

Vitrine des véhicules

Gérez votre vitrine de véhicules disponibles

+ Ajouter un véhicule

MODÈLE	MARQUE	IMMATRICULATION	STATUT	RÉSERVATIONS	ACTIONS
208	Peugeot	234TU5678	Disponible	0	Historique Modifier Supprimer
Golf	Volkswagen	345TU6789	Disponible	0	Historique Modifier Supprimer

FIGURE 3.36 – Interface vitrine

3.14.2.6 Interface bloquer/débloquer une agence :

Vue d'ensemble

Clients

Agences

Messages

Statistiques globales

Gestion des Agences

+ Ajouter une agence

AGENCE	LOCALISATION	VÉHICULES	REVENU	STATUT	ACTIONS
Agence Tunis Centre	Tunis	0	0 DT	Active	Modifier Suspendre
Agence Aéroport Carthage	Tunis	0	0 DT	Active	Modifier Suspendre
Agence Sousse	Sousse	0	0 DT	Active	Modifier Suspendre
Agence Sfax	Sfax	0	0 DT	Active	Modifier Suspendre
Agence Hammamet	Hammamet	0	0 DT	Active	Modifier Suspendre
Agence Nabeul	Nabeul	0	0 DT	Active	Modifier Suspendre

FIGURE 3.37 – Interface bloquer/débloquer une agence

3.15 Test unitaire

3.15.1 Test unitaire du cas d'utilisation «S'authentifier»

Cette section se concentre sur la récupération du code pour le cas «S'authentifier».

— Code source de la méthode d'authentification :

```
vi.mock("../http", () => ({
  default: { post: vi.fn() },
}));

describe("Test unitaire du cas d'utilisation S'authentifier", () => {
  it("doit authentifier l'utilisateur avec succes", async () => {
    const creds = { email: "user@test.com", password: "secret" };
    const user = {
      id: 1,
      name: "Test User",
      email: "user@test.com",
      role: "client",
    };
    http.post.mockResolvedValue({ data: { success: true, data: { user } } });

    const store = {};
    global.localStorage = {
      getItem: (k) => store[k] ?? null,
      setItem: (k, v) => (store[k] = String(v)),
    };

    const result = await authService.login(creds);

    expect(http.post).toHaveBeenCalledWith("/login", creds);
    expect(result.user).toEqual(user);
    expect(localStorage.getItem("user")).toBe(JSON.stringify(user));
  });
});
```

FIGURE 3.38 – Test unitaire du cas d'utilisation «S'authentifier»

3.15.2 Test unitaire du cas d'utilisation «Bloquer ou Débloquer une agence»

Cette section se concentre sur la récupération du code pour le cas Bloquer ou Débloquer une agence.

- Code source de la méthode de blocage ou déblocage d'une agence :

```
describe("Test unitaire du cas d'utilisation Bloquer/Débloquer une agence", () => {
  it("doit bloquer puis débloquer une agence", async () => {
    const id = 7;

    http.put
      .mockResolvedValueOnce({
        data: {
          success: true,
          message: "Agence bloquée",
          data: { id, status: "inactive" },
        },
      })
      .mockResolvedValueOnce({
        data: {
          success: true,
          message: "Agence débloquée",
          data: { id, status: "active" },
        },
      })
      .call();

    const blocked = await adminService.suspendAgency(id, "inactive");
    const unblocked = await adminService.suspendAgency(id, "active");

    expect(http.put).toHaveBeenNthCalledWith(1, `/admin/agencies/${id}`, {
      status: "inactive",
    });
    expect(http.put).toHaveBeenNthCalledWith(2, `/admin/agencies/${id}`, {
      status: "active",
    });
    expect(blocked.data.status).toBe("inactive");
    expect(unblocked.data.status).toBe("active");
  });
});
```

FIGURE 3.39 – Test unitaire du cas d'utilisation «Bloquer ou Débloquer un agence»

3.16 Conclusion

Dans le cadre de ce chapitre, nous avons mené le Sprint 1 selon une séquence logique couvrant l'analyse des cas d'utilisation, la conception et les tests. La phase suivante concerne la mise en œuvre du Sprint 2, qui est dédié à la gestion des réservations, des réclamations et du tableau de bord.

Chapitre 4

Réalisation du deuxième sprint

4.1 Introduction

En se basant sur la planification établie dans le chapitre précédent, ce chapitre est consacré à la réalisation du Sprint 2. Nous aborderons successivement la description fonctionnelle, la conception, ainsi que les diagrammes de séquence et de classe associés aux fonctionnalités développées au cours de ce sprint. Ces fonctionnalités concernent principalement la gestion des réservations, le suivi du score client, le traitement des réclamations, la consultation des statistiques et la gestion des messages.

4.2 Backlog produit du deuxième sprint

TABLE 4.1 – Backlog produit du deuxième sprint

Acteur	ID	Thème	User Story	Tâche	Estimation (Jours)
Internaute	US06	Consulter actualités	Consulter les actualités	Implémenter l’affichage des actualités.	3
				Tester.	
	US15	Interagir avec chatbot	Interagir avec le chatbot	Intégrer le module chatbot.	3
				Tester.	
US11	Contacter administrateur	Contacter l’administrateur	Implémenter le formulaire de contact.	2	
			Tester.		
Client	US07	Effectuer réservation	Effectuer une réservation	Implémenter la réservation d’un véhicule.	4
				Tester.	
	US08	Consulter réservations en cours	Consulter les réservations en cours	Implémenter la consultation des réservations.	3
				Tester.	
Agence	US09	Traiter réservations et redamations	Traiter les réservations et redamations	Implémenter le traitement des réservations et réclamations.	6
				Tester.	
	US13	Consulter statistiques(Agence)	Consulter les statistiques	Implémenter l’affichage des statistiques de l’agence.	3
				Tester.	
Utilisateur	US10	Voir score client	Voir le score client	Implémenter l’affichage du score.	2
				Tester.	
Administrateur	US12	Gérer messages	Gérer les messages	Implémenter la gestion des messages.	2
				Tester.	
	US14	Consulter statistiques (Admin)	Consulter les statistiques globales	Implémenter l’affichage des statistiques globales.	2
Tester.					
Estimation Totale (Jours)					30

4.3 Les prototypes des interfaces

Prototyper aide à identifier forces, faiblesses et à limiter les risques.

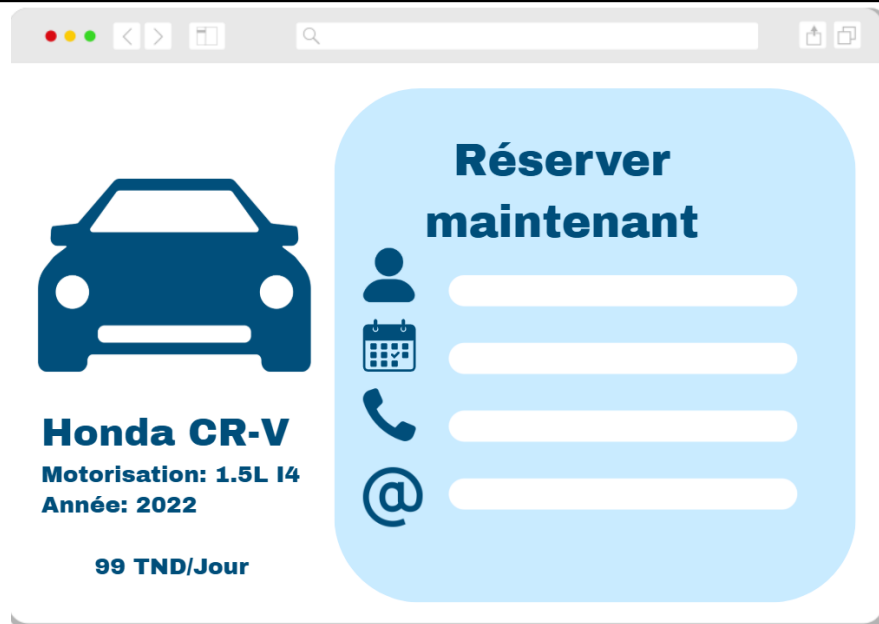


FIGURE 4.1 – Prototypage de l'interface "Effectuer réservation"



FIGURE 4.2 – Prototypage du tableau de bord global

4.4 La spécification fonctionnelle des besoins

On décide d'étudier la spécification fonctionnelle en analysant les diagrammes de cas d'utilisation détaillés du deuxième sprint, ainsi que les descriptions textuelles et les diagrammes système associés à chaque fonctionnalité.

4.5 Diagramme de cas d'utilisation du deuxième sprint

La figure suivante présente le diagramme des cas d'utilisation du deuxième sprint.

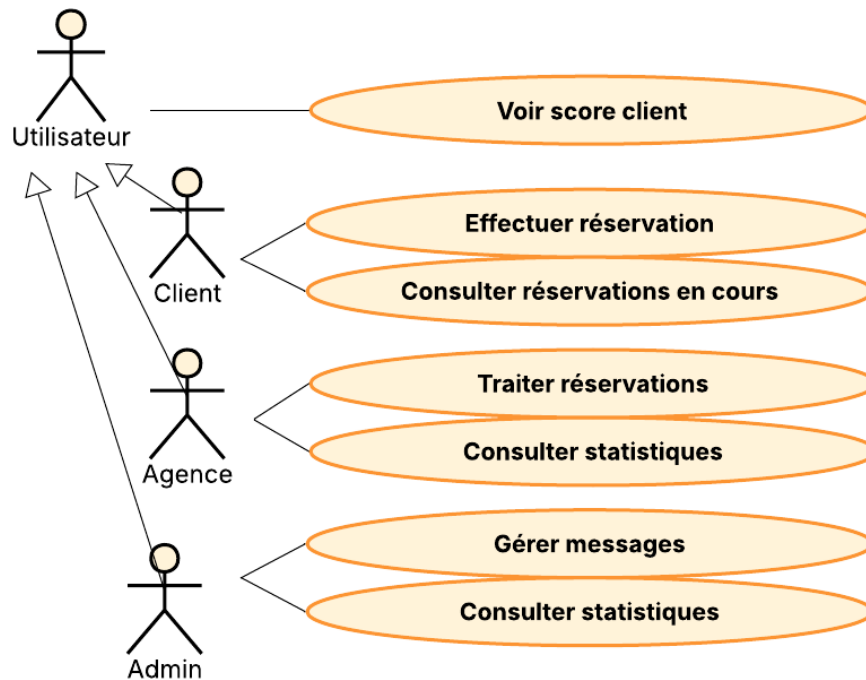


FIGURE 4.3 – Diagramme de cas d'utilisation du deuxième sprint

4.6 Analyse des cas d'utilisation du deuxième sprint

À chaque cas d'utilisation, nous allons associer une description textuelle pour détailler le scénario ainsi que les éventuelles situations exceptionnelles.

4.7 Analyse des cas d'utilisation de l'acteur "Internaute"

4.7.1 Analyse de cas d'utilisation «Consulter actualités»

Afin de mieux illustrer le déroulement du scénario, nous proposons ci-après une description textuelle accompagnée du diagramme de séquence du cas d'utilisation «**Consulter actualités**».

TABLE 4.2 – Description textuelle du cas d'utilisation «Consulter actualités»

Élément	Description
Acteur	Internaute
Description	Permet à l'internaute de consulter les dernières actualités publiées sur la plateforme.
Précondition	L'internaute accède à la plateforme. Des actualités sont disponibles.
Postcondition	L'internaute visualise les actualités. Il peut accéder au détail d'une actualité.
Scénario nominal	L'internaute accède à la section actualités. Le système affiche la liste des actualités. L'internaute sélectionne une actualité. Le système affiche le contenu détaillé.
Scénario alternatif	Si aucune actualité n'est disponible, un message informatif est affiché. Si une erreur survient, le système affiche un message d'erreur.

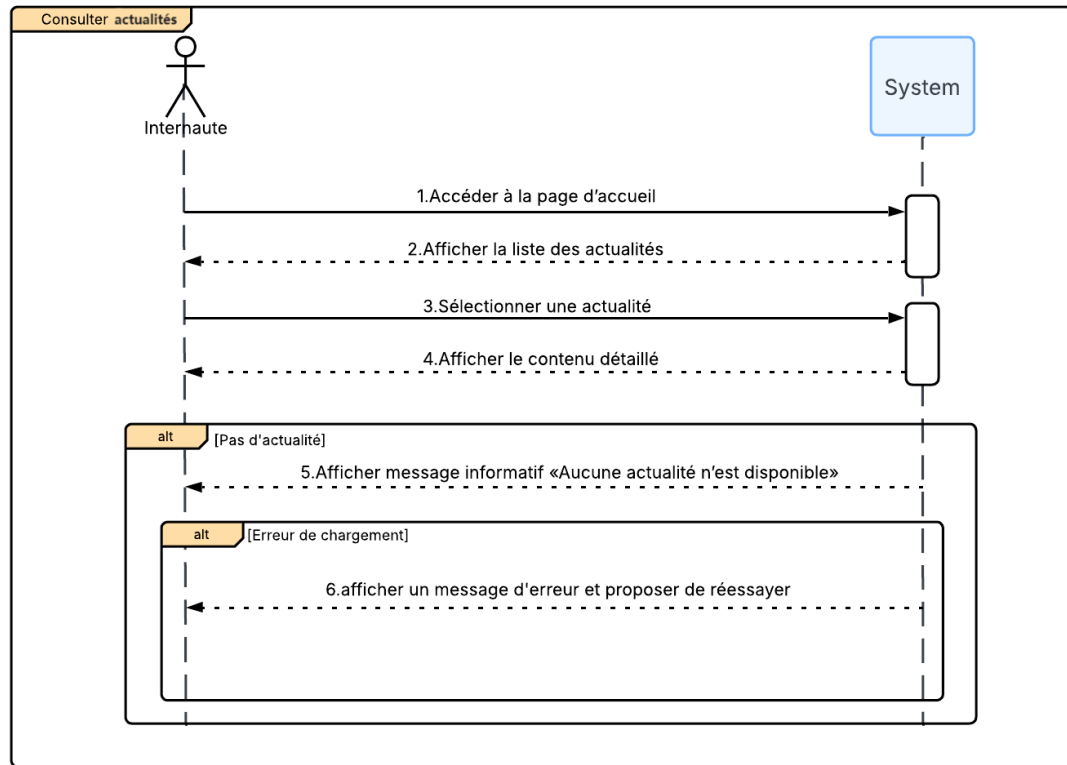


FIGURE 4.4 – Diagramme de séquence système de cas d'utilisation «Consulter actualités»

4.7.2 Analyse de cas d'utilisation «Interagir avec Chatbot»

Afin de mieux illustrer le déroulement du scénario, nous proposons ci-après une description textuelle accompagnée du diagramme de séquence du cas d'utilisation «**Interagir avec Chatbot**».

TABLE 4.3 – Description textuelle du cas d'utilisation «Interagir avec Chatbot»

Élément	Description
Acteur	Internaute
Description	Permet à l'internaute de communiquer avec un chatbot intelligent afin d'obtenir des informations ou de l'assistance.
Précondition	L'internaute accède à la plateforme. Le chatbot est disponible et fonctionnel.
Postcondition	L'internaute obtient une réponse pertinente à sa requête. L'interaction peut être enregistrée pour amélioration du service.
Scénario nominal	L'internaute ouvre l'interface du chatbot. Il saisit une question ou une demande. Le chatbot analyse la requête. Le chatbot génère une réponse adaptée. La réponse est affichée à l'internaute.
Scénario alternatif	Si la requête est incomprise, le chatbot demande une reformulation. Si le chatbot est indisponible, un message d'erreur est affiché.

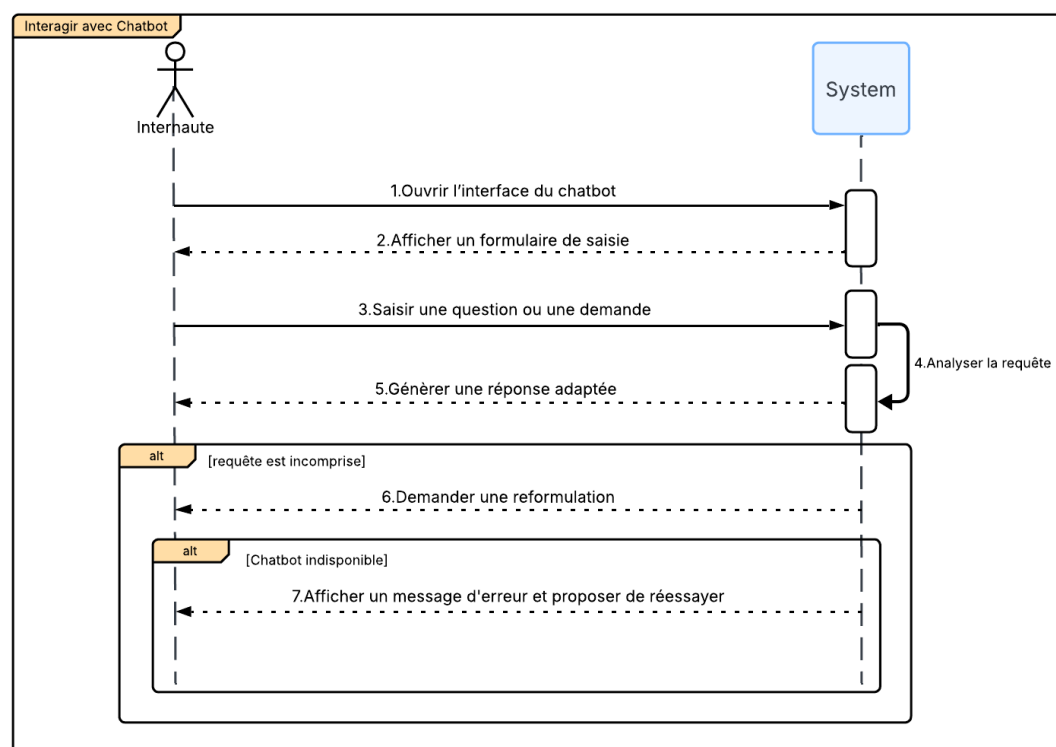


FIGURE 4.5 – Diagramme de séquence système de cas d'utilisation «Interagir avec Chatbot»

4.8 Analyse des cas d'utilisation de l'acteur "Utilisateur"

4.8.1 Analyse de cas d'utilisation "Voir score client"

La section suivante présente l'analyse du cas d'utilisation "Voir score client" de l'acteur utilisateur.

TABLE 4.4 – Description textuelle du cas d'utilisation "Voir score client"

Élément	Description
Nom	Voir score client
Acteur	Utilisateur
Description brève	Permet à l'utilisateur de consulter le score de fiabilité calculé automatiquement à partir de son historique de location.
Précondition	L'utilisateur est authentifié. Le système est accessible.
Postcondition	Le score de fiabilité de l'utilisateur est affiché ainsi que les critères ayant contribué à son calcul.
Enchaînement principal	1. L'utilisateur accède à son espace personnel. 2. Le système calcule le score à partir de l'historique (retards, annulations, incidents). 3. Le système affiche le score de fiabilité ainsi que son détail.
Scénario alternatif	2.1 Historique insuffisant pour calculer un score. 2.1.1 Le système affiche un score par défaut "100". 2.2 Erreur de calcul. 2.2.1 Le système affiche un message d'erreur.

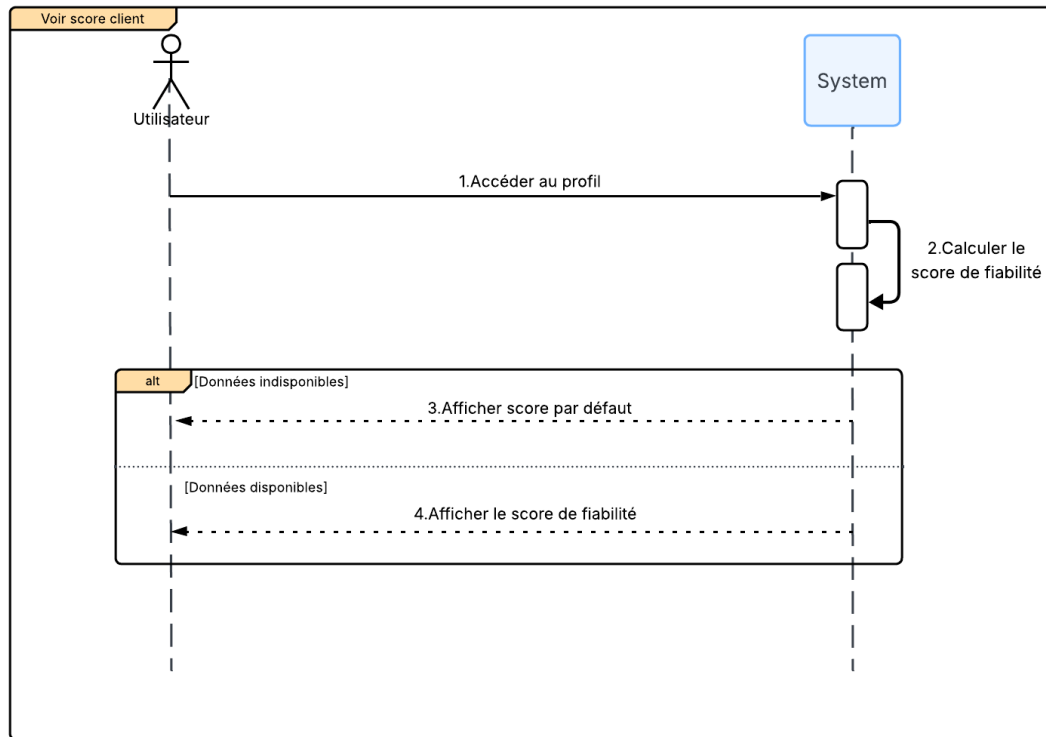


FIGURE 4.6 – Diagramme de séquence système de cas d'utilisation "Voir score client"

4.9 Analyse des cas d'utilisation de l'acteur "Client"

4.9.1 Analyse de cas d'utilisation "Effectuer réservation"

Afin de mieux illustrer le déroulement du scénario, nous proposons ci-après une description textuelle accompagnée du diagramme de séquence du cas d'utilisation "**Effectuer réservation**".

TABLE 4.5 – Description textuelle du cas d'utilisation "Effectuer réservation"

Élément	Description
Nom	Effectuer réservation
Acteur	Client
Description brève	Permet au client de réserver un véhicule disponible auprès d'une agence.
Précondition	Le client est authentifié. Le système est accessible. Au moins un véhicule est disponible.
Postcondition	La réservation est enregistrée et son statut est " En attente ". Le client reçoit une confirmation de sa demande.
Enchaînement principal	1. Le client accède à la liste des véhicules disponibles. 2. Le client sélectionne un véhicule. 3. Le système affiche les détails du véhicule et le formulaire de réservation. 4. Le client renseigne les dates de début et de fin de location. 5. Le système calcule le montant total. 6. Le client confirme la réservation. 7. Le système vérifie la disponibilité du véhicule. 8. Le système enregistre la réservation. 9. Le système affiche un message de confirmation.
Scénario alternatif	4.1 Dates invalides ou incohérentes. 4.1.1 Le système affiche un message d'erreur. 4.1.2 Retour à l'étape 4 du scénario nominal. 7.1 Le véhicule n'est plus disponible. 7.1.1 Le système affiche un message d'indisponibilité. 7.1.2 Retour à l'étape 1 du scénario nominal. 8.1 Erreur lors de l'enregistrement. 8.1.1 Le système affiche un message d'échec.

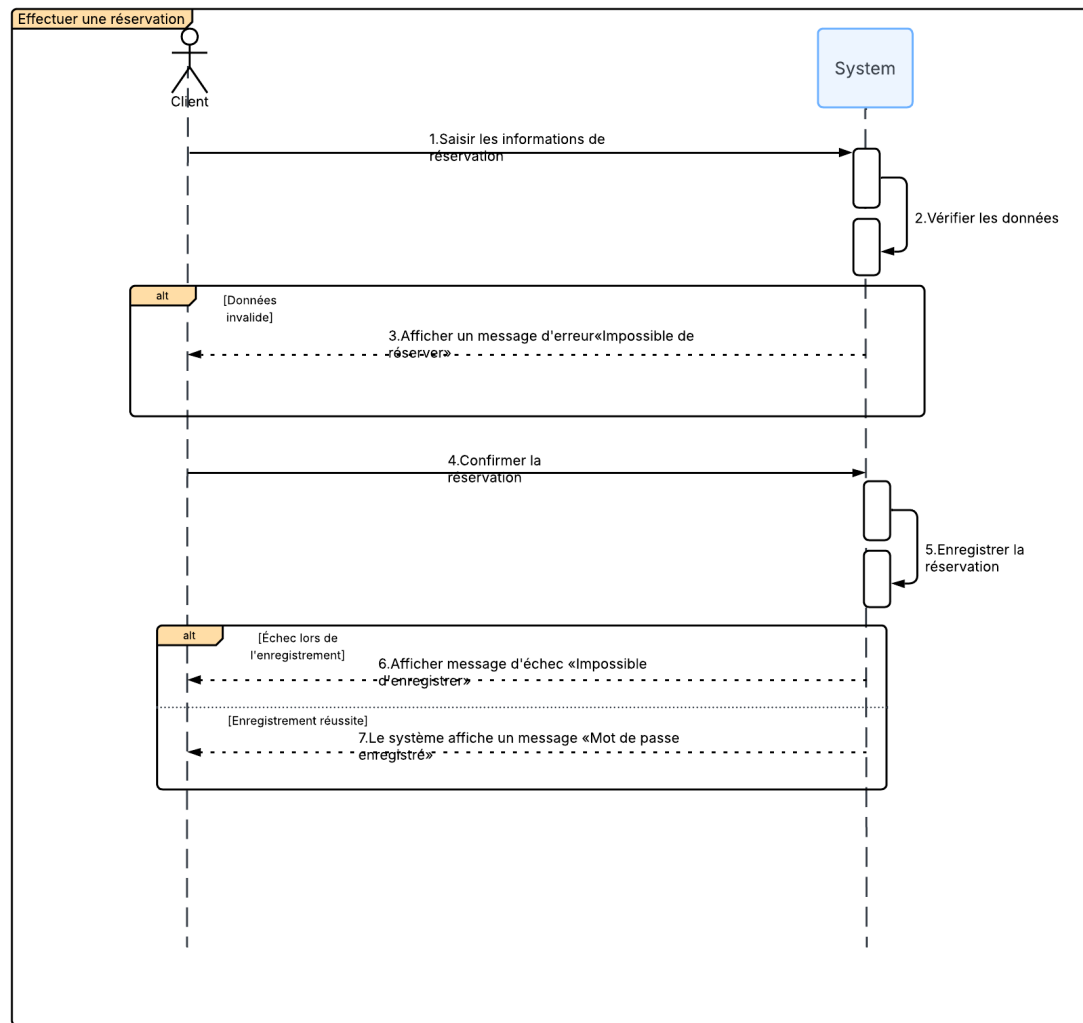


FIGURE 4.7 – Diagramme de séquence système de cas d'utilisation "Effectuer réservation"

4.9.2 Analyse de cas d'utilisation "Consulter réservations en cours"

Afin de mieux illustrer le déroulement du scénario, nous proposons ci-après une description textuelle accompagnée du diagramme de séquence du cas d'utilisation "**Consulter réservations en cours**".

TABLE 4.6 – Description textuelle du cas d'utilisation "Consulter réservations en cours"

Élément	Description
Nom	Consulter réservations en cours
Acteur	Client
Description brève	Permet au client de consulter l'état de ses réservations actives.
Précondition	Le client est authentifié. Le système est accessible.
Postcondition	La liste des réservations en cours est affichée avec leur statut respectif.
Enchaînement principal	1. Le client accède à la section "Mes réservations". 2. Le système récupère les réservations associées au compte client. 3. Le système affiche la liste des réservations avec leur statut (En attente, Confirmée, En cours, Terminée). 4. Le client sélectionne une réservation pour consulter les détails. 5. Le système affiche les informations détaillées de la réservation.
Scénario alternatif	2.1 Aucune réservation disponible. 2.1.1 Le système affiche le message "Aucune réservation en cours". 3.1 Erreur de chargement des données. 3.1.1 Le système affiche un message d'erreur et propose de réessayer.

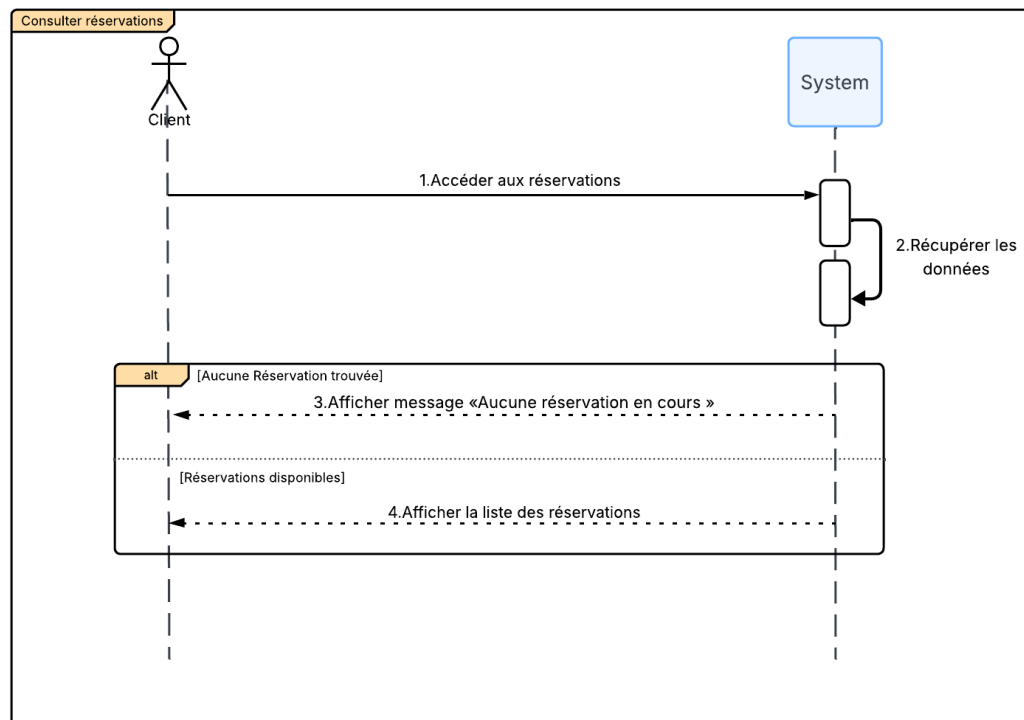


FIGURE 4.8 – Diagramme de séquence système de cas d'utilisation "Consulter réservations en cours"

4.10 Analyse des cas d'utilisation de l'acteur "Agence"

4.10.1 Analyse de cas d'utilisation "Traiter réservations"

Pour mieux analyser le déroulement du scénario, nous proposons ci-après une description textuelle et un diagramme de séquence du cas d'utilisation "**Traiter réservations**" de l'acteur Agence.

TABLE 4.7 – Description textuelle du cas d'utilisation "Traiter réservations"

Élément	Description
Nom	Traiter réservations
Acteur	Agence
Description brève	Permet à l'agence de consulter, accepter ou refuser les demandes de réservation des clients.
Précondition	L'agence est authentifiée. Des demandes de réservation existent dans le système.
Postcondition	La réservation est acceptée ou refusée. Le statut de la réservation est mis à jour. Le client est notifié de la décision.
Enchaînement principal	1. L'agence consulte les demandes de réservations. 2. Le système affiche la liste des demandes. 3. L'agence sélectionne une demande. 4. L'agence choisit d'accepter la réservation. 5. Le système met à jour le statut de la réservation. 6. Le système affiche un message "Réservation acceptée". 7. Le système envoie une notification d'acceptation au client.
Scénario alternatif	4.1 Refus de la réservation. 4.1.1 L'agence choisit de refuser la demande. 4.1.2 Le système confirme le refus. 4.1.3 Le système envoie une notification de refus au client.

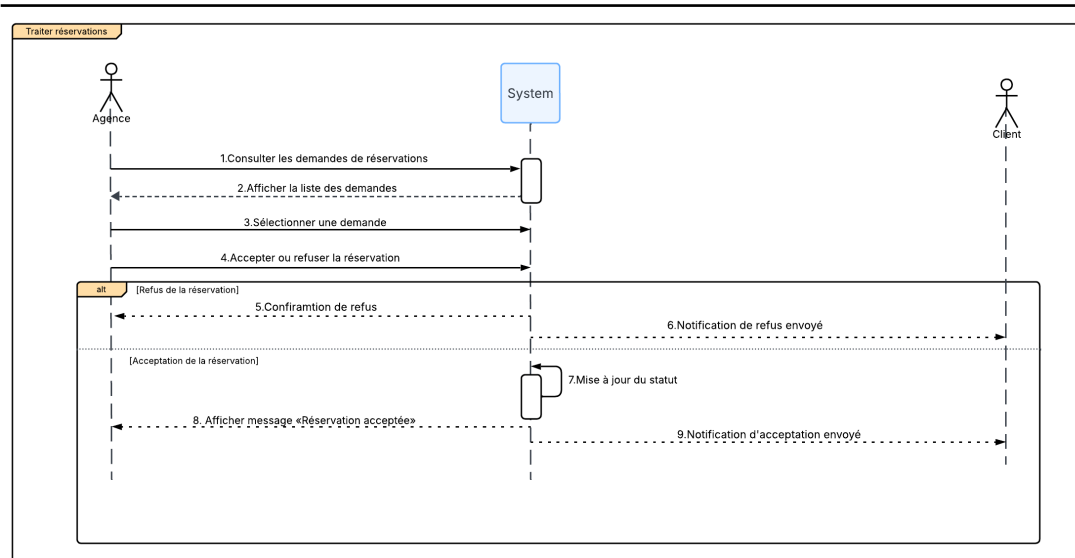


FIGURE 4.9 – Diagramme de séquence système de cas d'utilisation "Traiter réservations"

4.10.2 Analyse de cas d'utilisation "Consulter statistiques (Agence)"

Dans la suite, nous présentons une description textuelle accompagnée d'un diagramme de séquence du cas d'utilisation "**Consulter statistiques**" de l'acteur Agence.

TABLE 4.8 – Description textuelle du cas d'utilisation "Consulter statistiques (Agence)"

Élément	Description
Nom	Consulter statistiques
Acteur	Agence
Description brève	Permet à l'agence de visualiser ses indicateurs de performance à travers un tableau de bord interactif.
Précondition	L'agence est authentifiée. Le système est accessible.
Postcondition	Les indicateurs de performance sont affichés (nombre de réservations, taux d'occupation, chiffre d'affaires, satisfaction client).
Enchaînement principal	1. L'agence accède à la section " Statistiques ". 2. Le système collecte les données relatives à l'activité de l'agence. 3. Le système génère et affiche les indicateurs clés (graphiques, tableaux, synthèses). 4. L'agence consulte et analyse les données.
Scénario alternatif	2.1 Données insuffisantes pour générer des statistiques. 2.1.1 Le système affiche un message informatif. 3.1 Erreur de chargement. 3.1.1 Le système affiche un message d'erreur et propose de réessayer.

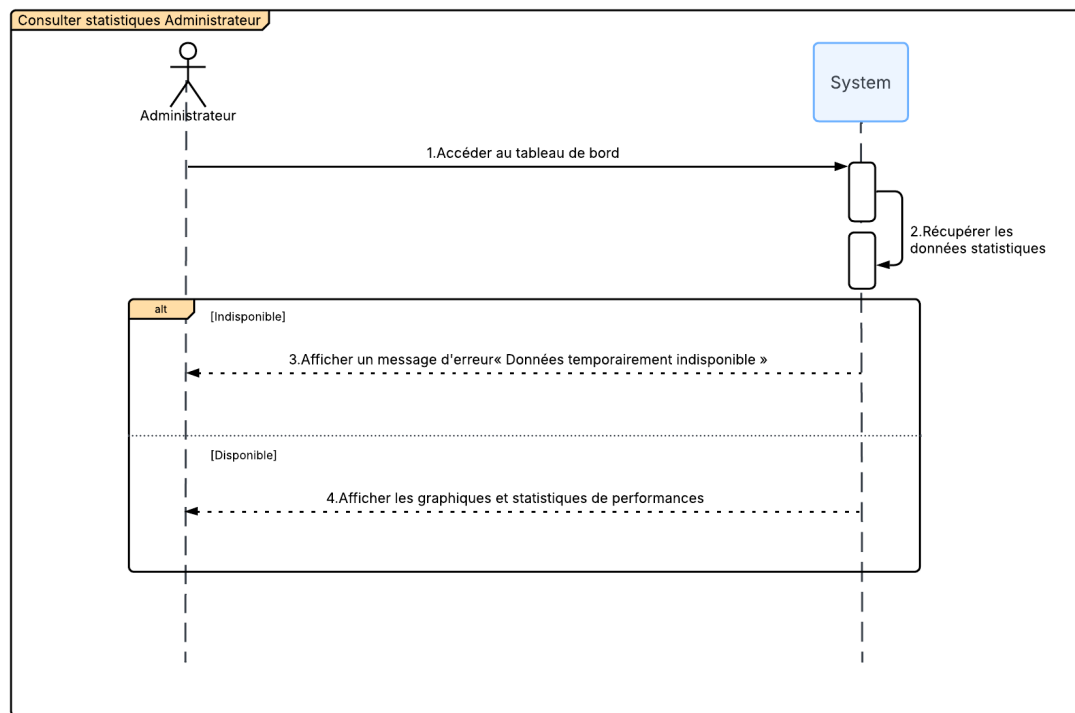


FIGURE 4.10 – Diagramme de séquence système de cas d'utilisation "Consulter statistiques (Agence)"

4.11 Analyse des cas d'utilisation de l'acteur "Administrateur"

4.11.1 Analyse de cas d'utilisation "Gérer messages"

Afin de mieux illustrer le déroulement du scénario, nous proposons ci-après une analyse du cas d'utilisation "**Gérer messages**" de l'acteur Administrateur.

TABLE 4.9 – Description textuelle du cas d'utilisation "Gérer messages"

Élément	Description
Nom	Gérer messages
Acteur	Administrateur
Description brève	Permet à l'administrateur de consulter les messages reçus des utilisateurs et d'y apporter une réponse.
Précondition	L'administrateur est authentifié. Le système est accessible.
Postcondition	Le message est traité et une réponse est envoyée à l'utilisateur concerné.
Enchaînement principal	1. L'administrateur accède à la section " Messages ". 2. Le système affiche la liste des messages reçus avec leur statut. 3. L'administrateur sélectionne un message. 4. Le système affiche le contenu du message et les informations de l'expéditeur. 5. L'administrateur rédige une réponse. 6. Le système enregistre et envoie la réponse. 7. Le système marque le message comme traité et affiche un message de confirmation.
Scénario alternatif	2.1 Aucun message disponible. 2.1.1 Le système affiche le message " Aucun message reçu ". 6.1 Erreur lors de l'envoi de la réponse. 6.1.1 Le système affiche un message d'échec.

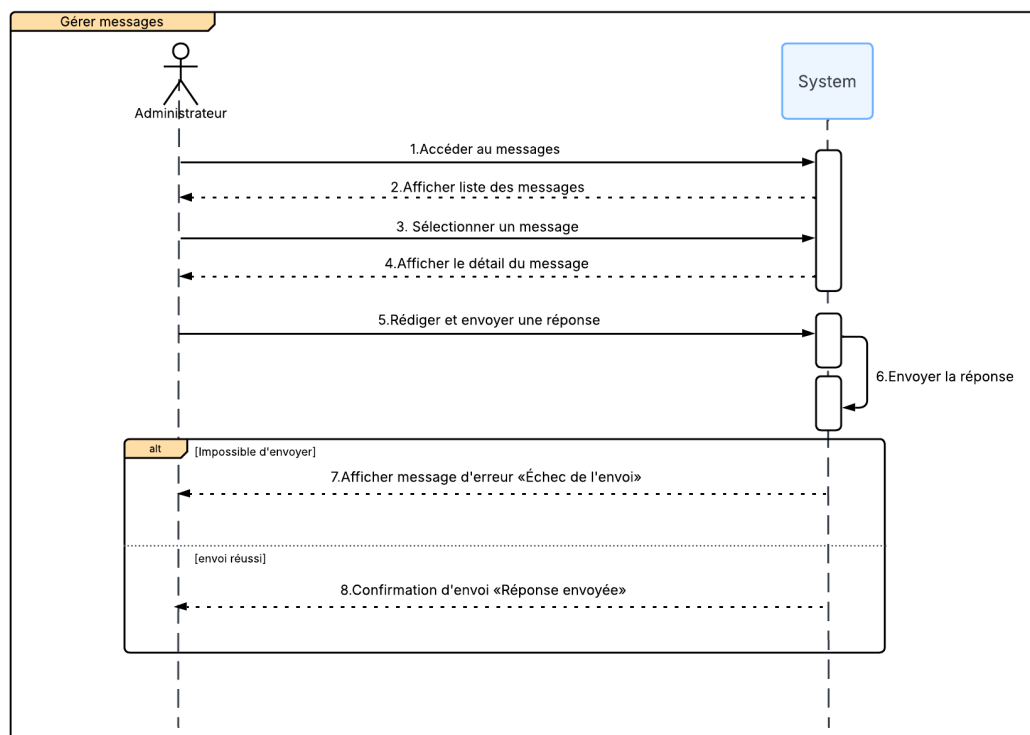


FIGURE 4.11 – Diagramme de séquence système de cas d'utilisation "Gérer messages"

4.11.2 Analyse de cas d'utilisation "Consulter statistiques (Administrateur)"

Dans la suite, nous présentons une description textuelle accompagnée d'un diagramme de séquence du cas d'utilisation "**Consulter statistiques globales**" de l'acteur Administrateur.

TABLE 4.10 – Description textuelle du cas d'utilisation "Consulter statistiques (Administrateur)"

Élément	Description
Nom	Consulter statistiques globales
Acteur	Administrateur
Description brève	Permet à l'administrateur de visualiser les indicateurs de performance globaux de l'ensemble de la plateforme afin de détecter les anomalies et d'améliorer le système.
Précondition	L'administrateur est authentifié. Le système est accessible.
Postcondition	Les indicateurs globaux sont affichés (nombre total d'agences, de réservations, de clients, score moyen, anomalies détectées).
Enchaînement principal	1. L'administrateur accède à la section " Tableau de bord ". 2. Le système collecte les données globales de toutes les agences. 3. Le système génère et affiche les indicateurs clés de performance (KPI). 4. L'administrateur consulte les données. 5. L'administrateur peut filtrer les données par période ou par agence.
Scénario alternatif	2.1 Données insuffisantes. 2.1.1 Le système affiche un message informatif. 3.1 Erreur de chargement. 3.1.1 Le système affiche un message d'erreur et propose de réessayer.

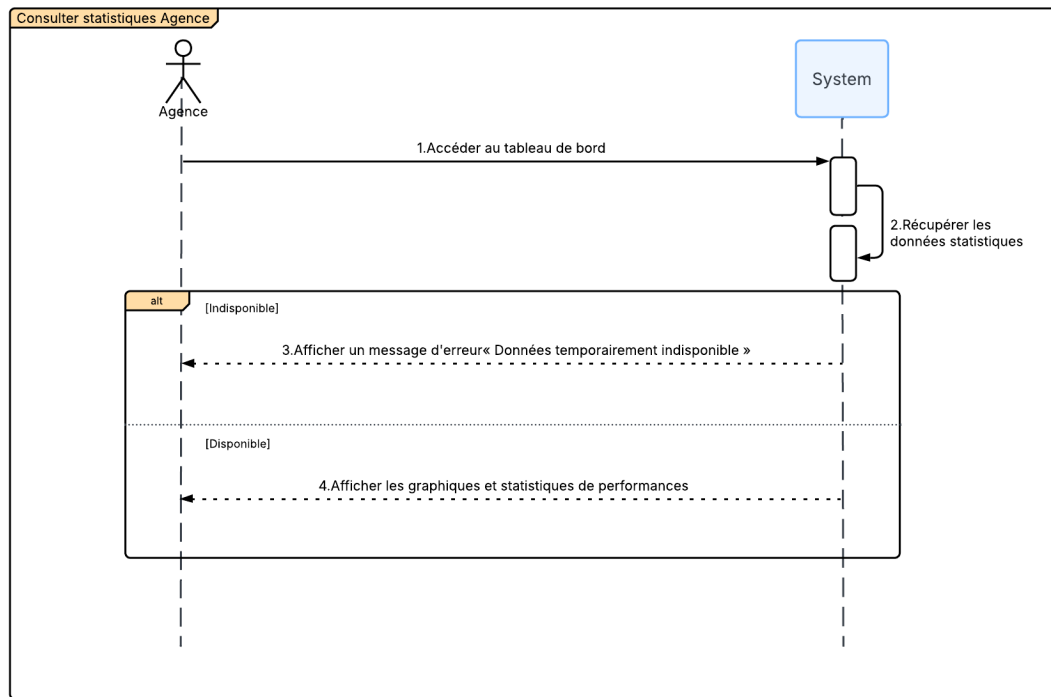


FIGURE 4.12 – Diagramme de séquence système de cas d'utilisation "Consulter statistiques (Administrateur)"

4.12 Diagramme de classe du deuxième sprint

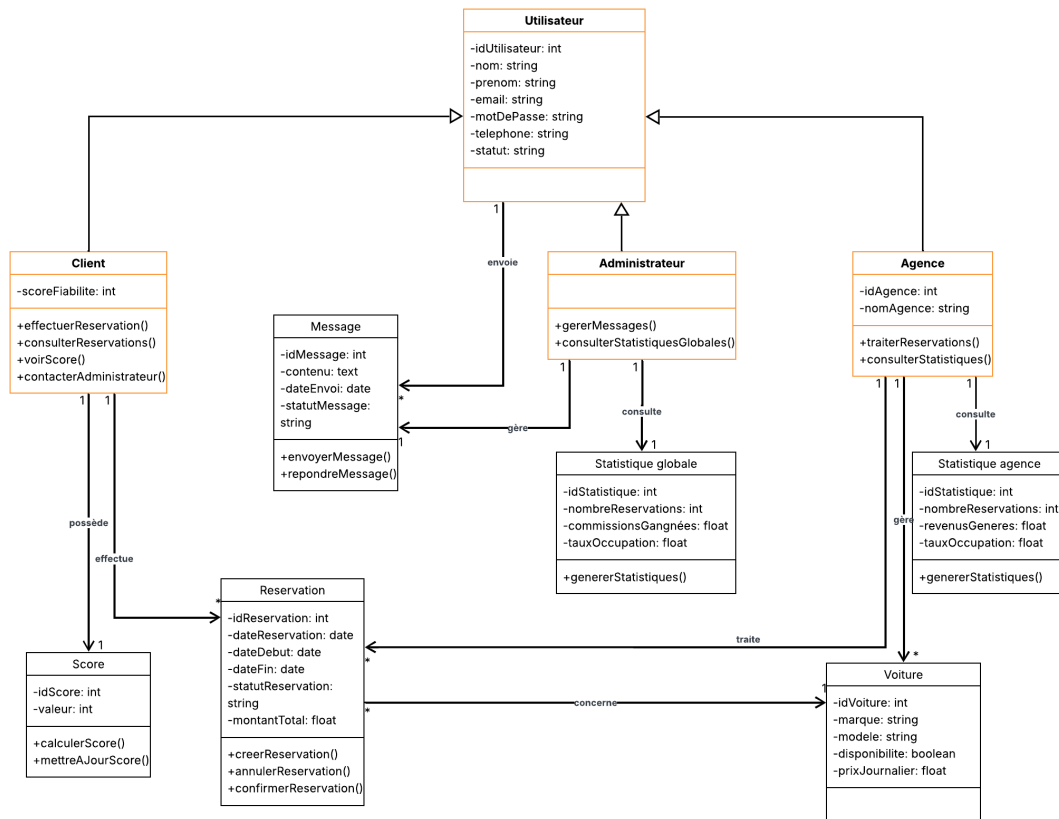


FIGURE 4.13 – Diagramme de classe du deuxième sprint

4.13 Conception des cas d'utilisation

Nous illustrerons dans cette section la conception de ce deuxième sprint à travers la réalisation d'un diagramme des classes pour chaque module, ainsi que d'un diagramme de séquence détaillé correspondant à chaque cas d'utilisation analysé.

4.13.1 Diagramme des classes participantes

4.13.1.1 Diagramme des classes participantes par acteur "Utilisateur"

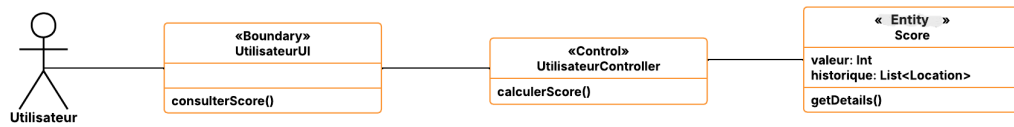


FIGURE 4.14 – Diagramme des classes participantes par acteur Utilisateur

4.13.1.2 Diagramme des classes participantes par acteur "Client"

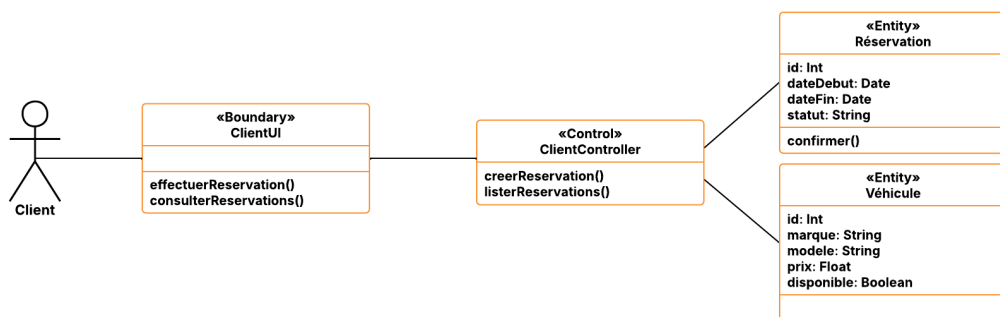


FIGURE 4.15 – Diagramme des classes participantes par acteur Client

4.13.1.3 Diagramme des classes participantes par acteur "Agence"

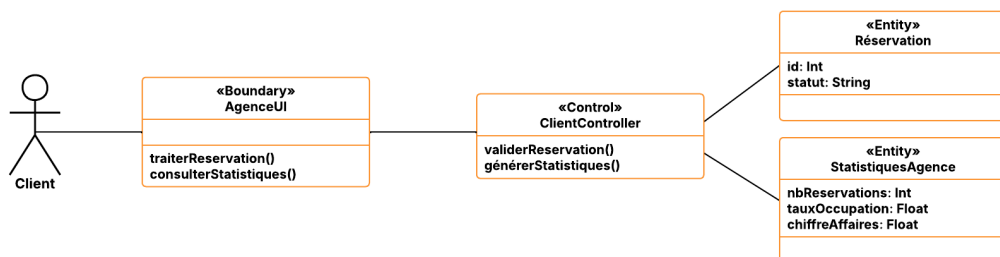


FIGURE 4.16 – Diagramme des classes participantes par acteur Agence

4.13.1.4 Diagramme des classes participantes par acteur "Administrateur"

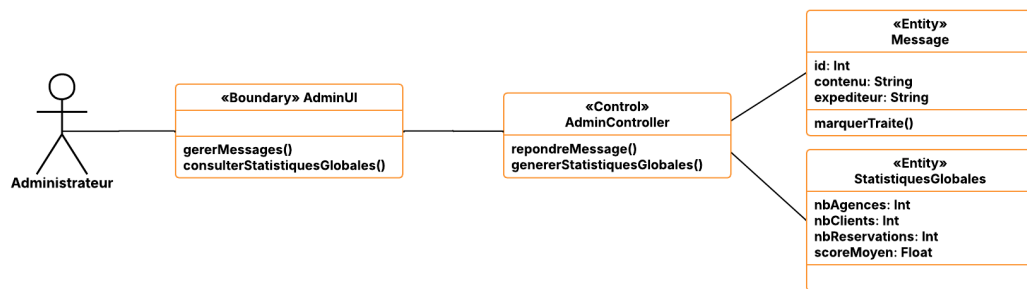


FIGURE 4.17 – Diagramme des classes participantes par acteur Administrateur

4.14 Diagrammes de séquence détaillés du deuxième sprint

4.14.1 Diagrammes de séquence détaillés de l'acteur Utilisateur

4.14.1.1 Diagrammes de séquence détaillé du cas d'utilisation " Voir score client "

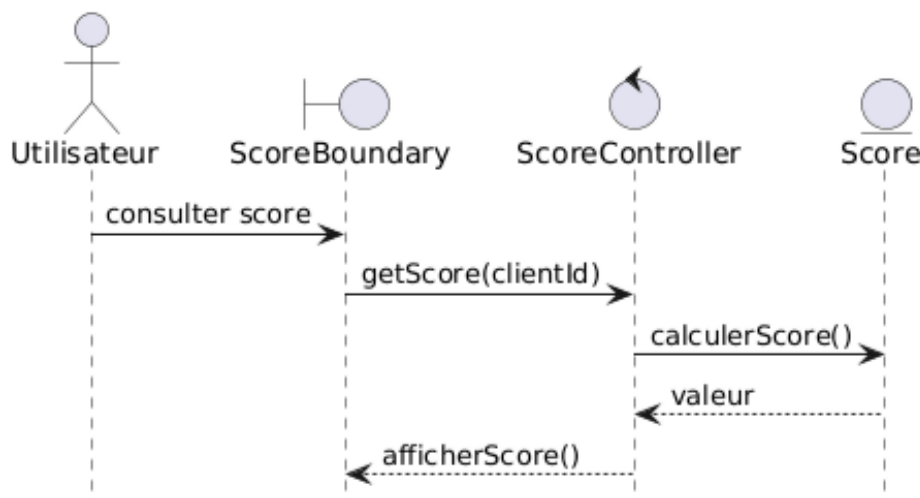


FIGURE 4.18 – Diagramme de séquence détaillé du cas d'utilisation " Voir score client "

4.14.2 Diagrammes de séquence détaillés de l'acteur Client

4.14.2.1 Diagrammes de séquence détaillé du cas d'utilisation " Effectuer réservation "

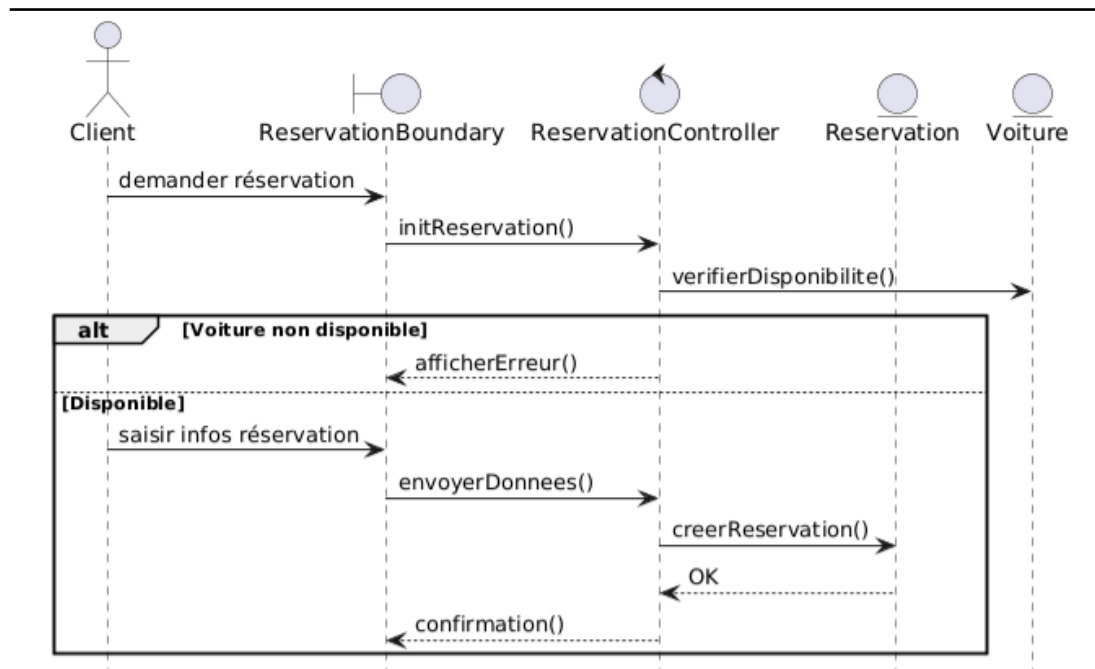


FIGURE 4.19 – Diagramme de séquence détaillé du cas d'utilisation " Effectuer réservation "

4.14.2.2 Diagrammes de séquence détaillé du cas d'utilisation " Consulter reservation ”

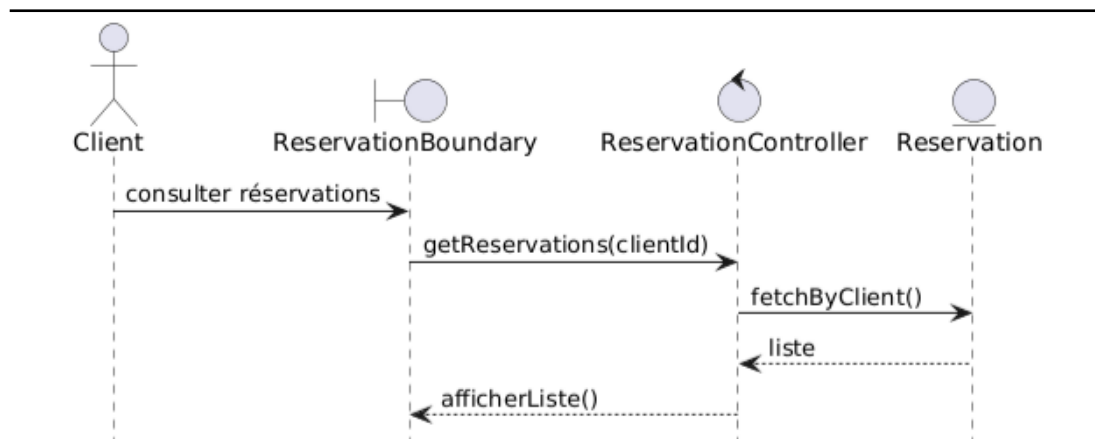


FIGURE 4.20 – Diagramme de séquence détaillé du cas d'utilisation " Consulter réservations en cours ”

4.14.3 Diagrammes de séquence détaillés de l'acteur Agence

4.14.3.1 Diagrammes de séquence détaillé du cas d'utilisation " Traiter réservations "

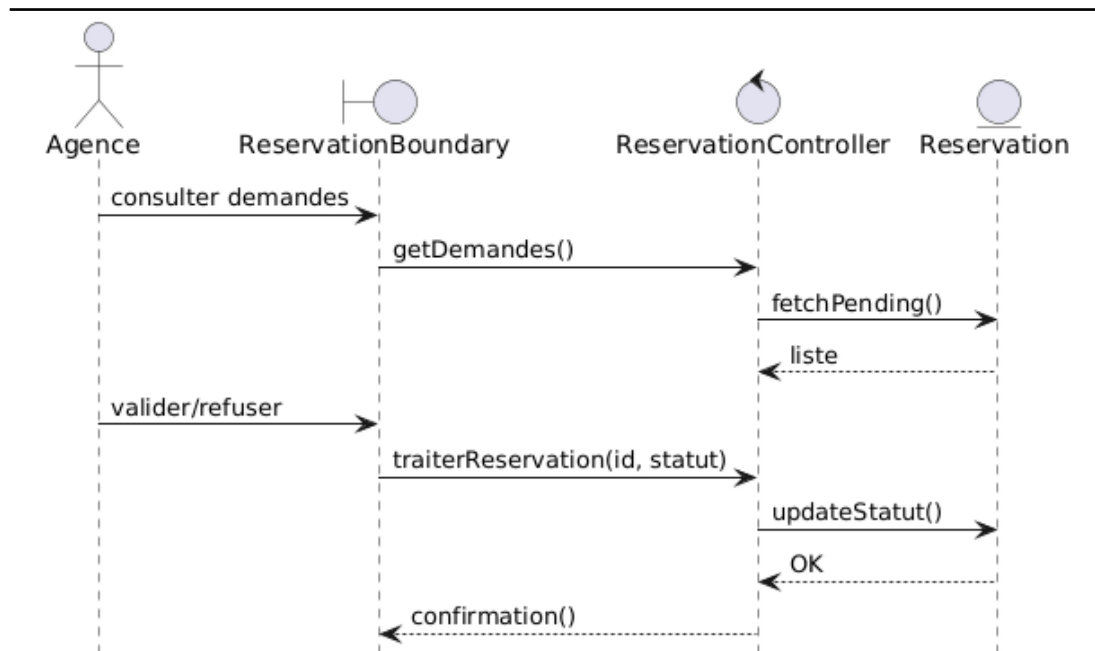


FIGURE 4.21 – Diagramme de séquence détaillé du cas d'utilisation " Traiter réservations "

4.14.3.2 Diagrammes de séquence détaillé du cas d'utilisation " Consulter statistiques agence "

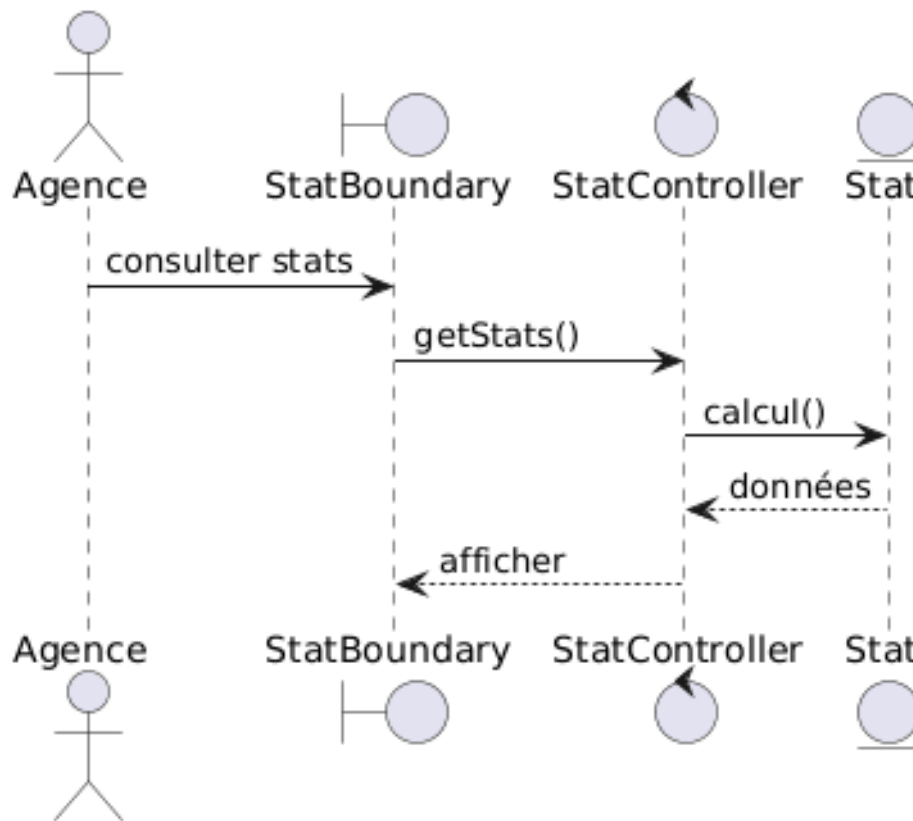


FIGURE 4.22 – Diagramme de séquence détaillé du cas d'utilisation " Consulter statistiques agence "

4.14.4 Diagrammes de séquence détaillés de l'acteur Administrateur

4.14.4.1 Diagrammes de séquence détaillé du cas d'utilisation " Gérer messages "

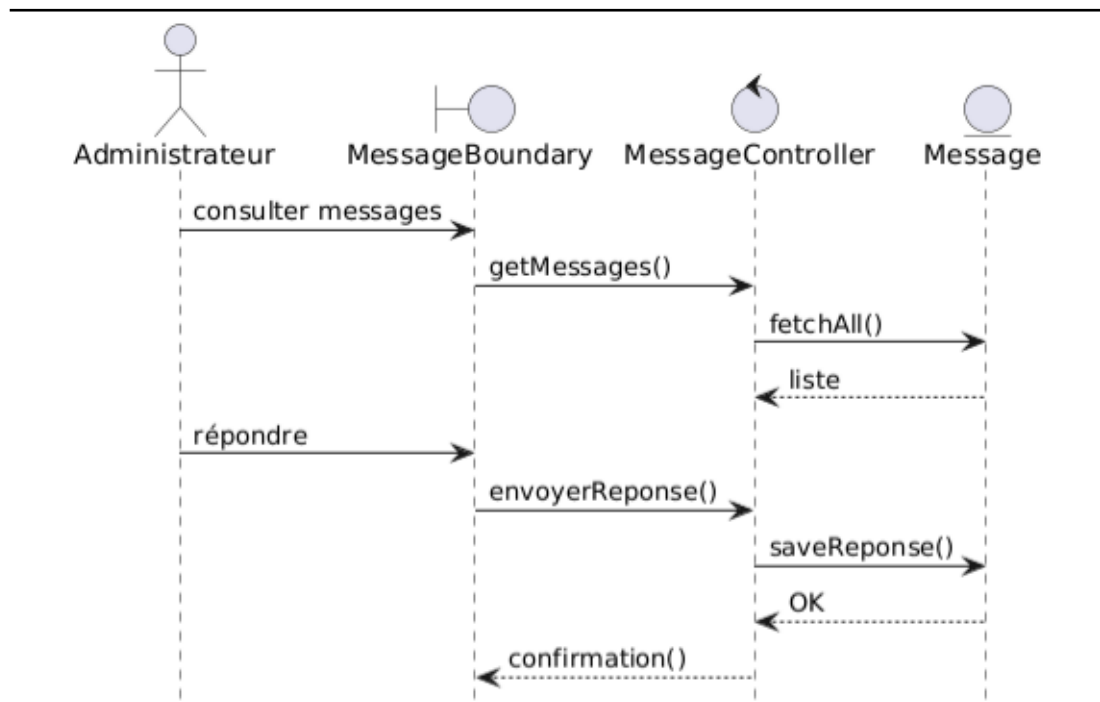


FIGURE 4.23 – Diagramme de séquence détaillé du cas d'utilisation " Gérer messages "

4.14.4.2 Diagrammes de séquence détaillé du cas d'utilisation " Consulter statistiques administrateur "

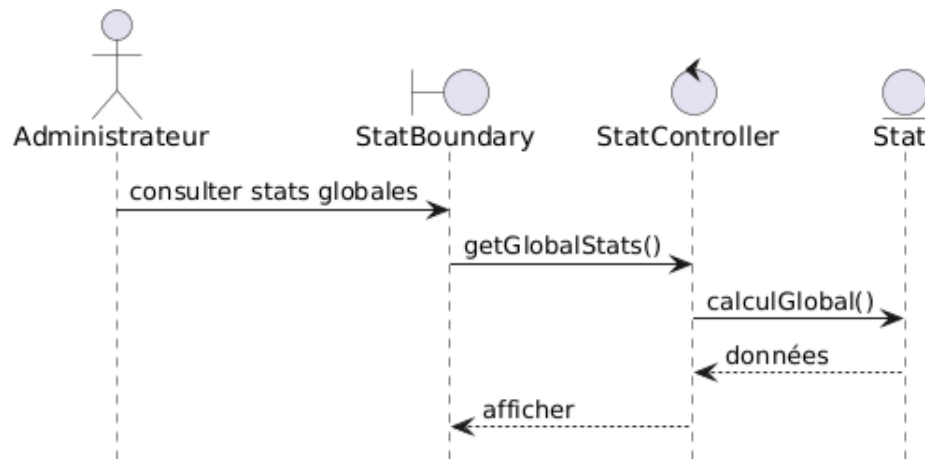


FIGURE 4.24 – Diagramme de séquence détaillé du cas d'utilisation " Consulter statistiques administrateur "

4.15 Diagramme de classe du deuxième sprint

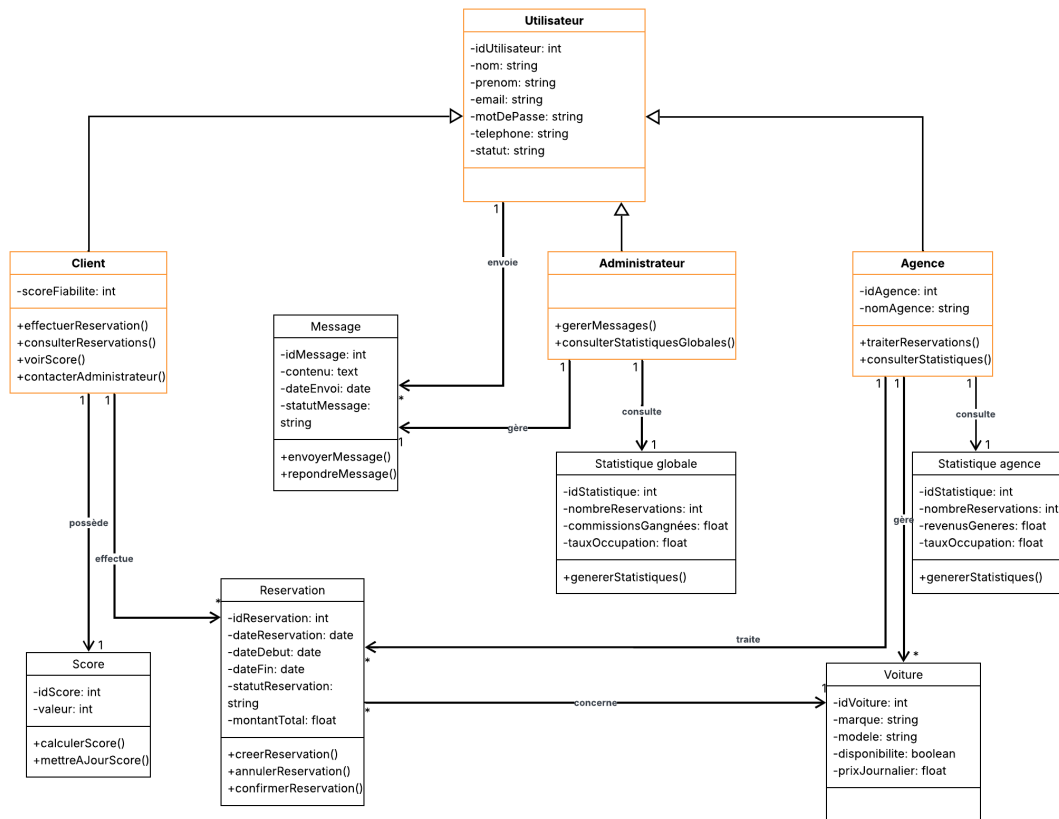


FIGURE 4.25 – Diagramme de classe du deuxième sprint

4.16 Implémentation

Le module d'implémentation du deuxième sprint présente la mise en œuvre technique des fonctionnalités développées, en détaillant leur réalisation au niveau du code et leur intégration dans l'application.

4.16.1 Tables de la base de données

4.16.1.1 Tableau « reservations »

TABLE 4.11 – Tableau « reservations »

Champs	Type	Libellé	Contraintes
id	BigInt UNSIGNED	Identifiant unique de la réservation	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
user_id	BigInt UNSIGNED	Identifiant du client	FOREIGN KEY (users), NOT NULL
vehicle_id	BigInt UNSIGNED	Identifiant du véhicule	FOREIGN KEY (vehicles), NOT NULL
start_date	Date	Date de début de location	NOT NULL
end_date	Date	Date de fin de location	NOT NULL
pickup_location	Varchar(255)	Lieu de récupération	NULLABLE
dropoff_location	Varchar(255)	Lieu de retour	NULLABLE
base_price	Decimal(10,2)	Coût base de location	NOT NULL
discount_amount	Decimal(10,2)	Montant remise appliquée	NOT NULL, DEFAULT 0.00
additional_charges	Decimal(10,2)	Frais supplémentaires	NOT NULL, DEFAULT 0.00
total_price	Decimal(10,2)	Prix total à payer	NOT NULL
platform_commission_rate	Decimal(5,4)	Taux commission plateforme	NOT NULL, DEFAULT 0.0800
platform_commission	Decimal(10,2)	Commission plateforme	NOT NULL, DEFAULT 0.00
agency_payout	Decimal(10,2)	Paieement agence	NOT NULL, DEFAULT 0.00
paid_amount	Decimal(10,2)	Montant déjà payé	NOT NULL, DEFAULT 0.00
remaining_amount	Decimal(10,2)	Solde impayé	NOT NULL, DEFAULT 0.00

Le tableau suivant présente le dictionnaire des données du deuxième sprint, en précisant pour chaque table les attributs définis, leurs libellés ainsi que leurs types.

4.16.1.2 Tableau « contact_messages »

TABLE 4.12 – Tableau « contact_messages »

Champs	Type	Libellé	Contraintes
id	BigInt UNSIGNED	Identifiant unique du message	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
name	Varchar(255)	Nom du contact	NOT NULL
email	Varchar(255)	Email du contact	NOT NULL
phone	Varchar(255)	Numéro téléphone	NULLABLE
subject	Varchar(255)	Sujet du message	NOT NULL
message	Text	Corps du message	NOT NULL
admin_reply	Text	Réponse de l'admin (snapshot)	NULLABLE
replied_by_email	Varchar(255)	Email de l'admin qui répond	NULLABLE
replied_at	Timestamp	Date de réponse	NULLABLE
is_read	TinyInt(1)	Message lu	NOT NULL, DEFAULT 0
submitted_at	Timestamp	Date de soumission	NULLABLE
read_at	Timestamp	Date de lecture	NULLABLE
created_at	Timestamp	Date de création	NULLABLE
updated_at	Timestamp	Date de modification	NULLABLE

4.16.1.3 Tableau « client_reliability_scores »

TABLE 4.13 – Tableau « client_reliability_scores »

Champs	Type	Libellé	Contraintes
id	BigInt UNSIGNED	Identifiant unique du score	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
user_id	BigInt UNSIGNED	Identifiant du client	FOREIGN KEY (users), NOT NULL
total_reservations	Int	Nombre total de réservations	NOT NULL, DEFAULT 0
completed_reservations	Int	Réservations complétées	NOT NULL, DEFAULT 0
cancelled_reservations	Int	Réservations annulées	NOT NULL, DEFAULT 0
late_returns	Int	Retours en retard	NOT NULL, DEFAULT 0
payment_delays	Int	Retards de paiement	NOT NULL, DEFAULT 0
damage_incidents	Int	Incidents dommages	NOT NULL, DEFAULT 0
total_unpaid_amount	Decimal(10,2)	Solde impayé total	NOT NULL, DEFAULT 0.00
reliability_score	Int	Score fiabilité (0-100)	NOT NULL, DEFAULT 100
risk_level	Enum('low', 'medium', 'high', 'blocked')	Niveau risque	NOT NULL, DEFAULT 'low'
admin_notes	Text	Notes administrateur	NULLABLE
last_calculated_at	Timestamp	Dernière recalculation	NULLABLE
created_at	Timestamp	Date de création	NULLABLE
updated_at	Timestamp	Date de modification	NULLABLE

4.16.2 Réalisation des interfaces du deuxième sprint

Nous présentons dans cette section les interfaces graphiques développées lors du deuxième sprint.

4.16.2.1 Interface contact administrateur :

FIGURE 4.26 – Interface contact administrateur

4.16.2.2 Interface formulaire réservation :

FIGURE 4.27 – Interface réservation

4.16.2.3 Interface statistiques globales :

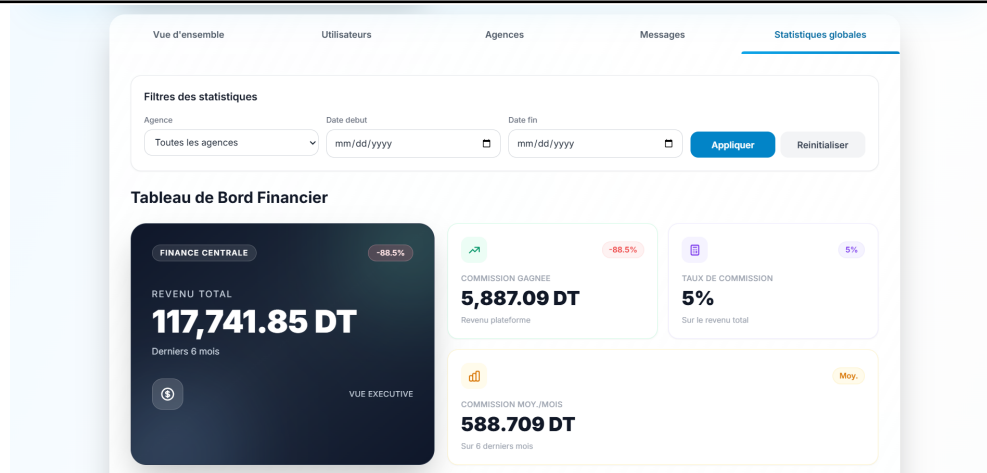


FIGURE 4.28 – Interface statistiques globales

4.17 Test

4.17.1 Test unitaire

4.17.1.1 Test unitaire du cas d'utilisation "Voir score client"

Cette section se concentre sur la reprise d'une partie du code pour le cas "Voir score client".

— Code source du cas d'utilisation "Voir score client"

```

9  class ClientReliabilityScoreTest extends TestCase
11     public function test_calculate_score_reduces_with_penalties()
14         Config::set('pfe.reliability_scoring', [
15             'cancelled_penalty' => 5,
16             'late_return_penalty' => 10,
17             'payment_delay_penalty' => 15,
18             'damage_penalty' => 20,
19             'blocked_threshold' => 30,
20             'high_risk_threshold' => 50,
21             'medium_risk_threshold' => 80,
22         ]);
23
24         // Create a partial mock of the model to avoid DB save
25         $scoreModel = $this->getMockBuilder(ClientReliabilityScore::class)
26             ->onlyMethods(['save'])
27             ->getMock();
28
29         // Configure attributes to simulate bad behavior
30         $scoreModel->cancelled_reservations = 2; // 2 * 5 = 10
31         $scoreModel->late_returns = 1; // 1 * 10 = 10
32         $scoreModel->payment_delays = 0; // 0 * 15 = 0
33         $scoreModel->damage_incidents = 1; // 1 * 20 = 20
34
35         // Prevent actual saving
36         $scoreModel->method('save')->willReturn(true);
37
38         $calculated = $scoreModel->calculateScore();
39
40         // Expected: 100 - (10 + 10 + 0 + 20) = 60
41         $this->assertEquals(60, $calculated);
42
43         // Risk level: 60 < 80 => 'medium'
44         $this->assertEquals('medium', $scoreModel->risk_level);
45         $this->assertFalse($scoreModel->isBlocked());
46         $this->assertFalse($scoreModel->isHighRisk());
47     }
48 }

```

FIGURE 4.29 – Test unitaire du cas d'utilisation "Voir score client"

4.18 Conclusion

Dans le cadre de ce chapitre, nous avons mené le Sprint 2 selon une séquence logique couvrant l'analyse des cas d'utilisation et la conception. Les fonctionnalités développées — effectuer et suivre des réservations, visualiser le score de fiabilité, traiter les réclamations, consulter les statistiques et gérer les messages — constituent le cœur métier de la plateforme et assurent une expérience complète pour l'ensemble des acteurs du système.

La phase suivante sera consacrée à la présentation des résultats obtenus, des technologies utilisées, ainsi qu'au déploiement de la solution.

Chapitre 5

Phase de clôture

5.1 Introduction

Ce chapitre présente la phase de clôture du projet, en mettant l'accent sur la finalisation technique de la plateforme, les choix d'outils utilisés, ainsi que les résultats obtenus à l'issue des deux sprints. L'objectif est de synthétiser le travail réalisé et de montrer la cohérence entre les besoins définis au départ et la solution livrée.

5.2 Environnement technique et outils adoptés

Pour concrétiser la plateforme web de gestion de location de véhicules, nous avons retenu un ensemble d'outils permettant d'assurer la robustesse, la maintenabilité et l'évolutivité de la solution.

5.3 Technologies de développement

Les technologies utilisées dans ce projet sont présentées dans le tableau suivant :

TABLE 5.1 – Technologies utilisées

Technologie	Description
 Laravel	Framework PHP utilisé pour développer le backend et exposer les API REST.
 React	Bibliothèque utilisée pour construire l'interface utilisateur côté client.
 MySQL	Système de gestion de base de données relationnelle du projet.
 Vite	Outil de build et serveur de développement pour le frontend.
 Tailwind CSS	Framework CSS utilitaire utilisé pour styliser rapidement l'interface.

5.4 Logiciels utilisés

Les logiciels utilisés dans le cadre de ce projet sont présentés dans le tableau suivant :

TABLE 5.2 – Logiciels utilisés

Technologies	Description
 Lucidchart	Outil de modélisation UML utilisé pour les diagrammes de conception.
 Visual Studio Code	Editeur de code utilisé pour le développement de l'application web.
 Overleaf (LaTeX)	Plateforme de rédaction utilisée pour la préparation du rapport.
 Microsoft Teams	Outil de visioconférence utilisé pour les réunions et les échanges à distance.
 XAMPP	Environnement local regroupant Apache et PHP pour l'exécution et les tests de l'application.
 phpMyAdmin	Interface web de gestion de la base de données MySQL.
 Git/GitHub	Outils de gestion de versions et de collaboration sur le code source.

5.5 Mise en production et organisation du déploiement

La phase de déploiement a consisté à préparer une version stable de l'application, incluant la configuration des variables d'environnement, la sécurisation des accès et la vérification du bon fonctionnement des modules principaux.

Une attention particulière a été portée à :

- la cohérence entre les environnements de développement et d'exécution,
- la fiabilité des échanges entre frontend et backend,
- la validation finale des fonctionnalités métier (réservation, réclamations, statistiques et messagerie).

5.6 Choix technologiques

5.6.1 Architecture retenue dans notre projet

Dans notre application, nous avons adopté une architecture API Laravel (backend) + SPA React (frontend). Le backend suit principalement une logique MVC enrichie par une couche Service, afin de séparer clairement les responsabilités et de faciliter la maintenance.

- **Modèle (Model)** : gère les entités métier et l'accès aux données (utilisateurs, agences, véhicules, réservations, paiements, etc.). Les modèles représentent la structure des données et les relations entre entités.
- **Contrôleur (Controller)** : reçoit les requêtes HTTP, orchestre le traitement et renvoie la réponse. Dans notre projet, les contrôleurs sont volontairement légers : ils délèguent la logique métier aux services et s'appuient sur une gestion centralisée des erreurs.
- **Vue** : dans une API REST, la « vue » est remplacée par des réponses JSON standardisées. Côté client, c'est l'interface React qui joue le rôle d'affichage et d'interaction utilisateur.

5.7 Conclusion

En conclusion, ce chapitre a permis de formaliser la phase finale du projet en présentant les outils mobilisés, les choix techniques retenus et les résultats atteints. Cette étape confirme la cohérence entre la planification initiale et la solution réalisée, tout en ouvrant la voie à des améliorations futures de la plateforme.

Conclusion

Ce rapport est le reflet de notre stage de fin d'études effectuée au sein de **Yes Internet**, dans le cadre de l'obtention de notre licence en Business Computing à l'**Ecole Supérieure de l'Economie Numérique (ESEN)**.

Durant cette période, nous avons eu l'opportunité de mettre en pratique nos acquis académiques à travers le développement d'une plateforme web de gestion de location de véhicules. L'objectif était de proposer une solution centralisée, fiable et évolutive, capable de faciliter la gestion des agences, des clients, des véhicules, des réservations et des paiements.

Pour répondre aux exigences du projet, nous avons suivi une démarche structurée. Nous avons d'abord réalisé une analyse de l'existant, puis identifié les besoins fonctionnels et non fonctionnels des différents acteurs. Ensuite, nous avons adopté la méthodologie SCRUM afin de planifier et exécuter le travail en itérations : construction du backlog produit, planification des sprints, implémentation progressive et validation des fonctionnalités.

Sur le plan technique, notre solution s'appuie sur les outils principaux suivants : Laravel, React, MySQL, Tailwind CSS et Vite. Nous avons également utilisé Git/GitHub pour le versioning, Lucidchart pour la modélisation, Visual Studio Code pour le développement et XAMPP pour l'environnement local. Ces choix nous ont permis de maintenir une architecture claire, une bonne qualité de code et un suivi rigoureux du projet.

Ce stage a été très enrichissant, car il nous a permis de renforcer nos compétences techniques et organisationnelles, notamment en conception, en développement web full stack, en tests et en gestion du temps. En perspective, la plateforme peut être améliorée par l'intégration de fonctionnalités complémentaires, telles que des notifications en temps réel, des tableaux de bord plus avancés et des mécanismes de supervision sécurisée, afin d'optimiser davantage l'expérience utilisateur et la performance globale du système.

